

Technical appendix: probability-proof workflows and trajectories

Shuo Deng

Kenneth W. Shum*

Revision 28

15 September 2026

Results updated through 14 September 2026

Revision 28 · 15 September 2026 · Results updated through 14 September 2026; earlier records retain their dates

Companion to *How probability proofs are completed*.

Evaluation update in Revision 23. Revision 23 incorporates the completed second batch, old L3 diagnostics and repairs, and a common mathematical-outcome assessment of both batches. Earlier development and historical analyses retain their dates. [Appendix Q](#) gives the new methods, item-level reconciliation, timing and evidence. No main solving run was added in this revision; independent caller checks test equivalent interfaces in existing deliveries.

Abstract

This appendix documents the two bodies of evidence used to evaluate ProbabilityTheoryFormalization’s mathematical completion, review and maintenance. Historical review records and corresponding code revisions support reconstruction of distinct judgments, qualification of mathematical targets, follow-up after failure, saved-code checks and detailed accounts of proof assembly and caller migration. The eleven-group configuration comparison supplies experimental deliveries, execution trajectories and resource observations. Its methods specify candidate selection, the materials shown to post-submission evaluators, the handling of disagreement and malformed results, and the corrected criteria used for the original 12 September panel. The 13 September continuation and responsibility audits retain their separate identities and costs.

Across both bodies, the detailed accounts distinguish mathematical work, recorded opinions, observable responses and accepted artifacts. Per-run measurements, task-level missingness, historical outcome tables, mathematical arguments and unresolved source questions remain available for examining the report’s reasoning. The original archive locators are retained, but access to these manuscripts is not access to every underlying record. The scope of this editorial pass is stated in F.2.

Navigation. Provenance and histories: A–B · Historical methods: C · Saved-code study: D · Glossary and sources: E · Contributions: F · Detailed trajectories: G · Observation windows: H · Source indexes: I–K · Historical mathematics: L · Measurements: M · Experimental methods: N · 13 September evidence: O · Timing reconstruction: P · Common assessment of both batches: Q · Detailed cases moved from the main text: R · Second-batch tool feedback and proof development: S · Evidence locations for expanded processes: T · Main report

*Corresponding author: wkshum@cuhk.edu.cn.

For historical mathematical work and maintenance, start with B and L; C and D contain the reconstruction, statistical and saved-code methods and results. For the original experiment, start with N.1 and N.6, then read the relevant cases in G and measurements in M. O gives the later continuation and audits. E.2 distinguishes the previously packaged supplement and later record locators from the wider historical archive. Both reading routes support the project questions in [main Section 1](#).

Structural edit in Revision 24. Full arguments from the former Chapters 4–6 are retained in [Appendix R](#). At that revision, main-text Chapter 4 introduced the types of problems agents face, retaining experimental data and verdicts from Revision 23.

Revision 25 process study. [Appendix S](#) adds full tool-call coverage, fixed-start process records and functional prototypes for the 33 second-batch runs. Detailed cases relocated in Revision 24 remain available; common outcomes and timing data are unchanged.

A Provenance and chronology

This appendix distinguishes evidence of the project design from evidence of actual proofs, judgments and maintenance. It explains what each source establishes and how event dates are located, so historical mechanisms and experimental implementations are assessed within their own versions and observation windows.

A.1 What each source can establish

Material	Supported observation	Interpretation limit
Code and documentation in commits	Interfaces, rules, and execution paths present in a version	A commit date need not mark first implementation or use; one commit may consolidate several changes
Candidates, support files, and diffs	Changes to statements, premises, proof bodies, imports, and calls	A shorter file need not contain less proof; identical main files can have different dependencies
Review, build, and application receipts	What was checked, the recorded judgment, and the program result	Model review is not human ground truth; review acceptance differs from successful state application
Operating instructions and handoffs	Actions requested of a person or agent	A request does not establish execution; a retrospective summary may omit the original motivation
Frozen analyses and case investigations	Counts under stated inclusion rules, code identity, and mathematical interpretation	Purposive cases are not random samples; recheck compatibility is not a new mathematical review

The evidence package retains source copies, content hashes, and original paths. Where code and actual use can be located but the discussion that introduced a rule cannot, the report describes the code and use without inventing a motivation. [E01-E21]

A.2 Dates, versions, and hashes

Dates come from Git metadata, timestamps inside review packages, or ledger events. Dates in rescue-directory names and modification times after copying are not treated as proof or review times. Request creation and result writing mark different stages; mixing them can reverse the apparent order.

Candidate versions, semantic-review versions, and mathematical-check versions have separate numbering. Reviews 29 through 32 for `thm_9_5`, for example, bind candidates 38 through 41. Mathematical check v10 and semantic review v10 for `prob_14_8` are different objects. Version numbers do not count independent model runs, and repeated failures do not estimate mathematical difficulty.

Hash comparisons preserve the original investigation’s distinction between raw bytes and normalized line endings. When a historical review reports a combined build but no separate command output survives, that remains an attributed historical claim. The historical audit numbers in C and D come from the frozen *qualified-v0.7.1* analysis and generated bindings used by *v0.7.5*. They are restored from the saved reports, not newly sampled or bootstrapped for this revision. Arithmetic cross-checks of reported values do not constitute rerunning the original analysis. [E18, E19]

A.3 Why these cases explain mechanisms rather than frequencies

The cases cover different design problems: proof accumulation in inversion, closure of the moment-generating-function argument, Gamma/Dirichlet support used by two consumers, the central-limit-to-coupon-collector chain, consumer migration after a total variation repair, target revisions, and upstream changes behind an identical main file. Earlier investigations selected these cases deliberately, sometimes knowing that a later candidate passed. They explain what changed. They do not estimate how often a phenomenon occurs across all tasks, and they are not held-out tasks for a future experiment.

A.4 Dated changes and their evidence status

Located date	Change present or recorded in the materials	Meaning of the date
March 30	Orchestration, decomposition, library retrieval, error feedback, and experience injection already exist	Inspected early baseline commit, not the date of invention
April 8	Old-orchestrator decomposition and task-package validation coexist	The two routes had not completed their transition
April 10	User requests a redesigned semantic review of source faithfulness	Design request, not implementation completion
May 14	Active entry points remove the old direct-generation route	State confirmed by a consolidation commit, not a controlled experiment
May 17	Bridge categories separate translation from proof debt	Code and documentation change together
May 22	Code contains obligation updates and propagation to the parent ledger	Available program branch, not proof that every case used it
June 15	Absorbed obligations require renewed checks of the parent and support files	Timestamped ledger event
June 19	A checkpoint states the boundaries among Math Gate, parent completion, and support	Saved policy milestone; some execution predates it
June 21	Shared Dirichlet density code is layered, with compatibility exports	Organization commit, not first mathematical completion
July 8	Modules for one task and their task-status boundary receive explicit family classifications	Classification milestone, not the first use of the word family
July 19	Probability-condition repair and consumer migration enter successive reviews	Actual case with object bindings
Read on September 10	Unified project contract continues to separate builds, semantic review, and application	Current reference point, not a new migration event

All dates refer to 2026. This chronology locates evidence; it is not a performance curve. [E01-E04, E08, E15]

A.5 Version basis for the system description

The inherited architecture account uses retained local snapshots of the repository README, architecture, workflow, review criteria, status contract and workspace-state documentation, indexed as V01–V06. The preparation recorded uncommitted README changes, so those snapshots—not the commit alone—define its local source basis. [26]

Revision 11 supplements that account with seven public documentation files read on 12 September 2026 at one fixed repository revision. They explain source-unit planning, the repair and scheduling decisions, canonical-state maintenance, textbook–Mathlib interfaces and the isolated demonstration. E.3 maps these documents to the relevant sections; the accompanying source guide retains the public revision, original file identities and selected excerpts. [34–36]

Documentation describes the operating design. The actual experimental configurations remain established by their frozen instructions, calls and receipts in N.1 and N.6, and historical mechanisms by their dated records. The public documentation is not substituted for those execution sources. [30,31]

B Additional proof and record histories

These cases retain the mathematical scope, ownership changes and event distinctions supporting main Sections 1, 5 and 6.

B.1 Obligation subtasks performed real mathematical work

OBL was the project’s label for proof-obligation subtasks. Diagnoses decomposed a parent’s gap into specific duties, and each subtask could have candidates, imports, and review records; the history also contains nested obligations. In `thm_7_8`, some results for integrability, existence of a Riemann-Stieltjes integral, squeezing, a common limit, and downstream adaptation returned to the parent file. Continuity on a compact set supplied integrability; function extension and an existing theorem supplied integral existence; the parent assembly then used these results. [E04]

In `prob_14_8`, deeper obligations proved local steps involving convergence of complex moment-generating functions on the real axis, a sequence of accumulation points tending to zero, and the implication from characteristic-function convergence to convergence in distribution. The parent still lacked the full derivation of characteristic-function convergence from the original assumptions. A subtask could therefore pass while its parent remained incomplete.

B.2 Absorbing subtasks and ending completion propagation

The saved ledger timestamps absorption of obligations for `thm_7_9`, `ex_14_4_2`, and `prob_14_8` at June 15, 2026, 02:10:44 UTC. It requires the parent and support files to be rebuilt, independently reviewed, and applied. Saved code from May 22 shows that obligation updates and proof-debt clearance could previously propagate to the parent ledger without necessarily producing a new parent semantic review. [E02, E04]

After absorption, useful mathematics remains in ordinary support or parent files, but an old obligation’s pass no longer automatically supports parent completion. The finite absolute-truncation proof for `thm_7_9` survives as chapter support. Its stated scope excludes the reverse bound, dominated convergence, filter handling, and the whole parent theorem. Decomposition remains available; final responsibility rests with the textbook task and the proof collection it actually imports. Old obligation packages remain historical evidence. [E11]

B.3 Mathematical content, ownership, and task identity

Question	Relevant distinction	What the classification does not establish
What mathematics does a declaration provide?	Representation conversion, proved auxiliary result, or outstanding proof work	A support label does not mean the statement is proved
Who maintains and explains it?	Material for one parent versus support with several actual consumers	A shared directory does not establish more efficient reuse
Does it count as a textbook task?	Parent textbook task versus auxiliary implementation module	Splitting a file does not complete another problem

June maintenance instructions favor preserving the parent’s final entry point and separating a large proof into layers owned by that task. Shared support becomes a candidate after a second actual consumer appears. This avoids prematurely generalizing task-specific parameters, temporary obligation wrappers, or a particular proof route. The history does not compare earlier, later, or absent extraction. [E03]

The word `family` also changes meaning across materials. An early maintenance queue may use it for related tasks; later `family_member` labels identify non-task modules split from one textbook task; the current catalog also uses book-number families. A keyword’s first occurrence cannot date all these mechanisms.

B.4 Two consumers of Gamma/Dirichlet support

Both `prob_1_4` and `ex_1_2_2` use the Gamma/Dirichlet work. The former develops a general normalized distribution and projected density. The latter handles coordinate charts, the Beta special case, and connections to its own source steps. A June 21 commit separates the density implementation into product-density, simplex-chart, and projected-density layers, leaving compatibility re-exports in the old Density file. `ex_1_2_2` calls the shared distribution identification while retaining its Beta branch and task statement. [E03]

This is evidence of actual reuse with task-specific proofs still present, although it does not quantify time saved. A later review asks the named `prob_1_4` theorem itself to include the already proved density conclusion. The subsequent parent theorem adds that result as a fourth conjunct. This changes where the public theorem states its promise; it does not make every earlier acceptance demonstrably wrong.

B.5 Modules for one task and already completed proofs

The Riemann-Stieltjes proof of `thm_1_1` moves from a mixed bridge to Darboux upper and lower sums, bad intervals, common refinements, and a limit argument. Its implementation modules later become `family_member` files under the same parent, which remains the review root. Candidate 43 for `thm_9_5` keeps the kernel argument in the main file; formal file 74 imports `thm_9_5_kernel` and retains the public inversion theorem. Reviews at both endpoints judge that the core proof need not be supplied as a public premise. The change relocates an existing proof. [E03, E18]

File separation can preserve a completed large proof, extract reusable material, or maintain an interface. These operations differ from a Math Gate intervention that changes a proof route. Counting them all as difficult tasks solved through replanning would erase that distinction.

B.6 Completion stages in `ex_3_1_4` and `thm_14_7`

When `ex_3_1_4` moves from fail to inconclusive, the later candidate has added interval measurability and uniqueness of the canonical extension. The label remains inconclusive because the request binds its obligations to placeholder nodes. The next pass record has a byte-identical main file but replaces those bindings with concrete obligations O1-O9. Code progress, review labels, and registration conditions change at different times.

For `thm_14_7`, candidate_v8 passes on May 27 under `open_math_debt`; the byte-identical `official_snapshot_v5` fails the next day for missing textbook proof. The written rules match, but the bound context and the stages of candidate review and official-snapshot review differ. An earlier acceptance of `source_step/open` nodes as proof debt also conflicts with the original prompt restrictions. `COMPLETED_WITH_PROOF_DEBT` denotes completion with outstanding

proof work, while `state_transition=completed` in an application receipt identifies an adoption path. Neither means a debt-free textbook proof. [E18]

B.7 thm_9_7: registered and reviewed candidates differ

The earlier request registers candidate `v3`, but its result records an override and actually fails `v4`. The later request and result both refer to `v5` and pass it. The actual `v4-to-v5` diff restores 57 lines of distribution-layer compatibility exports while retaining the core proof for arbitrary order. Each judgment has substantive content. The registered `v3-to-v5` relation would nevertheless assign `v4`'s failure to `v3`, so it is excluded from direct comparison. Its endpoints were not silently replaced to create another pair. [E18, E19]

B.8 Program events in prob_1_9 and thm_13_1

The minimal check for `prob_1_9` rejects a hash because the request embeds LF line endings while the disk file uses CRLF. Byte-level line-ending conversion confirms the same mathematical source, and both requests embed the same main code. This is a binding failure; it neither refutes the integration-by-parts proof nor adds one.

Five anomalies for `thm_13_1` show how mixing request timestamps and result-write timestamps can reverse the execution order. An earlier result does not reach the expected location. Before receiving its substantive criticism, the caller redispatches the unchanged candidate, and the second review does not use the first opinion. This sequence contains no observed repair prompted by the first failure. One endpoint in the same-main-file comparison is also an acceptance written by the original author within the same workflow; the other is a separately dispatched, read-only review. Their roles do not support interpreting them as two independent judgments under fixed conditions. [E18]

B.9 The thm_14_8 report-correction sequence

The original June 22 review is written but cannot be applied because required status fields are missing. The caller then explicitly supplies pass and exception categories, and the same reviewer corrects the fields. Application rejects the correction because the review basis changed. Under a new request, the same reviewer reassesses the byte-identical candidate. The final receipt permits an exception and explicitly does not adopt it as a completion result. This sequence consists of a judgment, report corrections, and a related reassessment; it contains no successive improvements to the candidate proof. [E18, E20]

B.10 prob_14_10: local preparation precedes upstream completion

Before its upstream theorem is complete, `prob_14_10` already proves polynomial-integral convergence, uniform exponential approximation, and error control, and constructs the upstream inputs. When it later changes from non-pass to pass, the main candidate changes only one call to select the second conjunct of the complete `prob_14_8` result. The upstream versions at the endpoints nevertheless differ: one contains private axioms, while the other is the single-file version completing the parent theorem. [E07]

Local preparation, completion of the upstream basis, and final delivery occur at three different times. The actual call demonstrates use of the lemma without measuring saved work. A one-line consumer change also does not imply that the reviewer merely changed its opinion.

B.11 An unrecovered source decision for def_3_6

The source sentence for `def_3_6` discusses closed bounded intervals in the real numbers. Later Heine-Borel context uses compactness for subsets of the real numbers. The later review accepts closedness and boundedness of real subsets as the chapter's representation, retaining closed intervals as a one-way instance. The material supports that limited historical acceptance. It does not establish that closed and bounded sets are compact in arbitrary topological or metric spaces, or recover the missing second-version source approval. Target comparability remains unknown. [E18]

C Review-history reconstruction, qualified comparisons and follow-up

The historical analysis asks what the project’s review archive can establish about judgment identity, mathematical comparability and recorded progress. It uses the frozen `qualified-v0.7.1` study and its saved numerical bindings. Its units differ from the 33-entry configuration panel: reconstruction identifies judgments, qualification identifies comparable transitions, and follow-up identifies fixed failure origins and admissible steps. The sections below state each operation and its results. [E18, E19]

C.1 Reconstructing judgments without counting representations as new reviews

The 1 September 2026 snapshot contains 3,820 database rows concerning 453 task objects. This is the snapshot’s archival boundary, not every project run or the current catalog’s completion denominator.

Raw record category	Count	Interpretation
Distinct direct-evidence signatures	3,275	Signature identity based on result-evidence and review-input digests
Duplicate references	60	Repeated references, not additional independent opinions
Object-binding references	404	Links to reviewed objects
Collector claims	80	Statements made by collection machinery
Application-receipt reference	1	Reference to an application event
Database rows	3,820	Total in the stated snapshot

A direct-evidence signature is keyed by exactly the result-evidence digest and review-input digest. Task, adapter, candidate, label and context are consistency checks within a group. The normalizer converts returned review records into the stored result format. Removing 33 normalizer-generated inconclusive signatures that lack original-request bindings leaves 3,242 reviewer-generated signatures. Eleven Kenneth custom semantic reviews are separately stratified by source, leaving 3,231 ordinary-task signatures. [E18, E19]

Qualification and representation merging then follow different branches:

Branch	Calculation	Unit and operation
Signature qualification	$3,231 - 30 = 3,201$; $3,201 - 2 = 3,199$	Remove program/test records and two unqualified endpoints; unit remains the signature
Ordinary representation groups	$3,231 - 251 = 2,980$	Merge representations using explicit production relationships
Qualified judgment groups	$3,201 - 251 = 2,950$; $2,950 - 2 - 1 = 2,947$	Merge representations, exclude endpoints and merge one report-correction sequence
Verified-input subset	2,914 of 2,947	Qualified judgment groups with verified inputs

The thirty program/test exclusions comprise twelve runs without a configured reviewer, seventeen application or normalization failures, and one injected test. The 251 representation merges comprise 223 raw/normalized pairs, twenty-two application pointers and six same-delivery pointers. Therefore 3,199 is not an earlier stage of the 2,980 count: these numbers apply different operations to a common boundary.

Text similarity is not sufficient grounds for merging. Three text-identical copies without production pointers remain separate, and thirty-four candidate-identity pairs remain unresolved. One judgment can have several saved representations, and one candidate can receive several judgments. The reconstruction avoids turning a normalization failure into a mathematical rejection or a raw/normalized pair into two independent reviews. The raw counts and their unit definitions, rather than a single apparent number of review attempts, are the result of this part of the audit.

C.2 What was bound beyond the main file

Among 3,189 recoverable inputs, 721 contain submitted file bundles, 1,547 contain hashes of non-main objects and 2,442 identify at least one non-main object. Together they provide 15,788 object observations with source-field locations. These categories can overlap. Of the original 2,645 directed relations, ninety-eight retain the same main file while changing non-main objects bound at both endpoints; thirteen of those move from fail to pass. No relation in that investigation preserves all objects needed to establish identity of the complete reviewed state. [E18, E19]

The archive also records 574 task-dependency links, with 318 tasks in its largest connected component. This graph does not define independent statistical clusters. All thirty-nine subtasks have parent-input bindings. Of 125 located inputs, 117 directly embed template review histories; repeating such text does not establish a new independent review of the child’s source, code and obligations.

After representation merging there are 2,571 relations, fourteen still undirected. Within the research boundary, the 2,475 directed relations have the following structural classifications:

Relation category	Count
Candidate-to-official main-file match	241
Candidate main-file change	465
Later candidate with the same main file	100
Textbook-source-related relation	207
Official-object follow-up	1,172
Unresolved activity mode	290
Total directed relations in this boundary	2,475

These are structural relationships, not independent improvements or causal links. In particular, same-main-file comparisons are not automatically reviewer-agreement observations. The target-qualified analysis below asks whether the intended mathematical task and candidate binding are comparable while allowing actual code revisions; it does not assert that the entire review input stayed byte-identical.

C.3 Constructing the 461-comparison set and measuring label change

Selection begins with 465 relations classified as candidate main-file changes. Excluding one failed application leaves 464. Separating two support-assembly relations leaves 462 managed task-version relationships. Excluding the `thm_9_7` registered/reviewed-object mismatch leaves 461 directly bound comparisons across 248 tasks. [Appendix B.7](#) explains why the mismatched earlier version cannot silently be substituted with the version actually judged. [E18, E19]

The transition table counts relationships, with earlier labels as rows and later labels as columns:

Earlier / later	Pass	Fail	Inconclusive	Partial
Pass	103	12	1	0
Fail	205	117	5	1
Inconclusive	15	0	1	0
Partial	1	0	0	0

There are 116 earlier and 324 later pass labels, a net increase of 208. The binary pass indicator records acceptance versus another recognized status for this calculation; it does not make fail, inconclusive and partial semantically interchangeable. Unrecognized labels are not silently assigned a score.

The saved analysis applies three selections and two weighting conventions:

Selection	Comparisons / tasks	Task-weighted change [reference interval]	Comparison-weighted change [reference interval]
All directly bound main-file comparisons	461 / 248	57.42 [51.96, 62.79]	45.12 [38.52, 52.16]
Both endpoints labeled pass or fail	437 / 234	56.06 [50.43, 61.61]	44.16 [37.38, 51.51]
Earlier findings list nonempty	363 / 205	77.52 [72.49, 82.32]	60.06 [50.23, 70.39]

All changes and interval limits in this table are **percentage points**. Reference intervals are the saved whole-task resampling summaries specified in C.9. They do not correct retrospective selection, cross-task dependence or missing semantic evidence.

Twenty-four comparisons contain an inconclusive or partial endpoint. Sixteen move from one of those statuses to pass; one moves in the reverse direction. Recorded review-contract fields differ in forty-five relations and source-statement fields in two; matching recorded fields does not establish identical effective requirements. The 775 distinct endpoints appear 922 times, with 147 reused endpoints, forming 314 maximal chains. Their total net change remains 208. Shared endpoints and telescoping changes prevent treating every relation as a fresh independent improvement.

For task k and comparison e , let $A_{ke,0}$ and $A_{ke,1}$ indicate a recorded pass at its earlier and later endpoints. With n_k comparisons in task k ,

$$d_k = \frac{1}{n_k} \sum_e (A_{ke,1} - A_{ke,0}), \quad \hat{\mu}_{\text{task}} = \frac{1}{S} \sum_k d_k, \quad \hat{\mu}_{\text{comparison}} = \frac{\sum_k n_k d_k}{\sum_k n_k},$$

where S is the number of tasks. The first overall mean weights tasks equally after averaging within each task; the second weights each comparison equally. Both describe changes in recorded labels within the selected archive, not the fraction of textbook tasks completed or the effect of review.

C.4 Qualification and detailed-reading coverage change what the trend can explain

C.4.1 How target comparability was decided

The question is whether the **accepted source mathematical objective** is preserved between the two endpoints. The proof route, implementation, support organization or review policy may change while that objective remains the same. An added formal parameter or changed theorem header is a reason to inspect the relation, not by itself proof of a changed target. A source decision accepting a genuinely different statement or convention, by contrast, creates a target boundary. [E18, E19; 32]

The semantic decisions came from **AI archival readings, unblinded to the earlier and later verdicts**. The recorded reading basis includes the source text, both endpoint inputs and results, public declarations, full embedded candidate code, diffs and review obligations. The reader records the before/after target and the reason for retaining a relation or treating it as a boundary. These readings establish a historical qualification decision; the program separately checks its relation membership, bound objects and source references.

The frozen revision accepts either an explicitly sourced earlier target decision or a saved substantive reading tied to that relation. It does not accept a screen reporting merely that no material change was found. Twelve such weak decisions were replaced by substantive readings in the retained historical revision, with their superseded decisions

preserved. A positive decision remains subject to the independent object-binding exclusion; it does not certify unchanged review conditions or the truth of the code. [32]

Disposition in the 462-relation frame	Count	Operational meaning
Affirmative target comparability	129	A saved substantive decision confirms the same source mathematical objective
Confirmed target boundary	2	A saved decision establishes a changed accepted statement or source convention
Unknown comparability	331	No target-specific substantive decision establishes comparability for that relation

Thus the explicit decision table has 131 decisions, not 462 completed semantic adjudications. The other 331 are unknown in the full frame; they are not 331 confirmed changes or necessarily unread records in every other part of the investigation. The analysis has no comprehensive independent expert adjudication of these AI readings.

The following examples reproduce decisions in that same frame:

Example	What the saved decision uses	Consequence
A fail-to-fail relation for coupon collection, prob_14_11	A changed target header was screened, but reading the source, complete endpoint code and review obligations attributed the difference to route, support, debt or policy while retaining the objective	Affirmative, subject to the other eligibility checks
A fail-to-pass relation for independent sums, prob_14_7	The source omitted independence of the displayed limits; the passing statement exposes <code>hLimitIndep</code> under an explicit source correction, supported by the counterexample in L.4.1	Confirmed target boundary
A fail-to-pass relation for compactness, def_3_6	The relation has no target-specific substantive decision; screening alone cannot establish equivalence of the accepted source objectives	Unknown; fixed-target follow-up cannot cross it

These are decisions about individual relations, not permanent classifications of task names. In particular, another prob_14_11 transition changes the centering target, as explained in L.4.2. The original decisions are retained in the evidence supplement. [32]

C.4.2 Qualified label change and detailed-reading coverage

Excluding the one registered/reviewed-object conflict from the 129 affirmative relations leaves 128 comparisons across seventy-nine tasks. They contain twenty-eight earlier passes and forty-five later passes, a net increase of seventeen. [E18, E19]

Weighting for the target/object-qualified subset	Change in recorded pass	Saved reference interval
Task weighting	20.46 percentage points	[9.49, 31.65] percentage points
Comparison weighting	13.28 percentage points	[4.93, 22.86] percentage points

This positive change remains a result of the qualified subset. The subset was obtained through purposeful investigation rather than random restriction of the broad sample, so its difference from the broad estimate cannot be attributed

to changed targets alone. Unknown target comparability means that affirmative evidence was not established, not that a changed target was demonstrated.

The detailed anomaly investigation supplies a separate coverage view. It contains 1,018 relationships: 994 substantive mathematical or relational readings and twenty-four program or binding comparisons. Sixty of the substantive relationships lack recoverable endpoint inputs. Its intersection with the 461-comparison set is:

Coverage of the broad comparison	Comparisons	Earlier pass labels	Later pass labels	Net change
In the detailed anomaly queue	163	32	33	1
Outside that queue	298	84	291	207
Broad comparison total	461	116	324	208

Nearly all broad net growth lies outside the detailed anomaly queue. The case explanations therefore cannot explain that growth at the same semantic depth. This is a statement about coverage, not a finding that the remaining relationships are wrong.

A further same-candidate view contains 1,380 object-checked comparisons, including 270 label changes. Content coding covers 128 of those changes, leaving 142 uncoded. Those **128 coded label changes are not the 128 target/object-qualified comparisons** above. Context, dependencies and review stage can change while a main file or candidate is unchanged, so the same-candidate view does not produce a single fixed-evidence reviewer-agreement rate.

C.5 Fixing failure origins before deciding which paths can be followed

Within the 462-relationship frame, 364 relationships have nonempty earlier findings. Linking identical shared endpoints produces 259 maximal segments: segments that cannot be extended within those relationships, not a subset selected for being unusually long. A segment is retained only when its first endpoint is explicitly labeled fail, and that endpoint becomes its one fixed origin. Later failures do not restart the trace; the method does not search inside other segments for additional convenient origins. This gives 230 origins across 182 tasks. Only then are target comparability and valid binding required for each observed step. [E18, E19]

Construction stage	Unit	Operation
364 relationships	Earlier/later pairs with earlier findings	Select the starting relation material
259 maximal segments	Connected sequences sharing identical endpoints	Assemble the retained segments
230 failure origins in 182 tasks	Fixed starting positions	Retain explicitly failing segment starts
100 observed follow-up steps	Qualified transitions	Follow only while target and object evidence permit

The 363-comparison findings-nonempty subset in C.3 is defined after selection of the qualified 461-comparison set. The 364-relationship starting frame here is defined before follow-up qualification. The different counts belong to different operations.

The saved segment-length distribution is retained because it explains how the connected starting material is formed:

Relationships per segment	Segments	Relationships contributed
1	209	209
2	38	76
3	1	3

Continued on next page

Relationships per segment	Segments	Relationships contributed
4	5	20
5	3	15
12	2	24
17	1	17
Total	259	364

Follow-up stops at first acceptance, the saved segment's end or before the first ineligible comparison. An origin that exits before a single qualifying step remains in the denominator. The procedure therefore describes how much evidence remains after a fixed failure origin rather than dropping paths that cannot be followed.

C.6 Exits and the changing risk set

Strict follow-up observes twenty-five first acceptances before exit and 205 exits before observed acceptance. The reasons are:

Exit reason	Before first step	After observed steps	Total
No affirmative target-comparability decision	159	27	186
Confirmed target or source-convention change	1	1	2
Registered and reviewed objects conflict	1	0	1
Saved segment ends	0	16	16
Total	161	44	205

An exit marks the end of admissible observation, not the eventual mathematical failure of the task. In particular, the 186 missing-comparability exits must not be reported as 186 confirmed target changes.

Follow-up step	Paths at risk	First acceptances	Exits	Continue
1	69	22	32	15
2	15	3	5	7
3	7	0	4	3
4	3	0	2	1
5	1	0	0	1
6	1	0	0	1
7	1	0	0	1
8	1	0	0	1
9	1	0	0	1
10	1	0	1	0

At each step, paths at risk equal acceptances plus exits plus continuations. The sum of risk-set sizes is 100 observed steps. The 161 pre-first-step exits are not among those steps but remain among the 230 origins. A later step is contributed only by a path that has not already passed and still has qualifying evidence.

C.7 First-versus-later acceptance and the selection check

Quantity	Numerator / denominator	Estimate	Saved whole-task reference interval
Acceptance among qualified first steps	22 / 69 first steps	31.88%	[21.43%, 43.14%]
Acceptance among qualified later steps	3 / 31 later steps	9.68%	[0.00%, 28.00%]
Acceptance observed before evidence exit	25 / 230 fixed origins	10.87%	[7.02%, 15.07%]
Mean observable follow-up	100 steps / 230 fixed origins	0.4348 steps per origin	[0.3198, 0.5702] steps per origin

The saved later-minus-first contrast is -22.21 percentage points, with reference interval $[-36.94, -1.90]$. That contrast is not a diminishing-returns estimate. Only fifteen tasks supply later steps. Restricting retrospectively to those tasks, their nineteen first steps contain only one acceptance. The later-step group therefore differs in its task composition and survival in observation, as well as in revision stage. [E18, E19]

This check is the substantive interpretation of the table: the broad first-step group is not an appropriate counterfactual for the selected group that continues. Long paths can also contribute several later steps. Neither the negative contrast nor the reference interval removes those differences. The observations establish the acceptance and trace lengths actually seen; they do not identify the effect of one additional revision.

C.8 Broader reachability does not repair the missing fixed-target evidence

Keeping the same 230 origins but permitting broader relationships yields:

Follow-up rule	Observed steps	Paths reaching a recorded pass	Distinct pass endpoints, where reported
Strict target/object-qualified follow-up	100	25	Not separately reported here
Historical main relations with earlier findings and known boundaries applied	326	203	Not separately reported here
All main relations	327	204	204
All qualified activity relations in the research boundary	391	222	210
Qualified activity relations across the archive	396	224	211

The broader rules answer a broader record-reachability question. Reaching more pass records is observable and useful for mapping the archive; it does not establish that the same mathematical target was preserved along every newly admitted step. The strict analysis's exits cannot be filled with these passes as though only missing timestamps had been repaired. [E18, E19]

C.9 Resampling specification and retained limits

The saved comparison analysis uses 20,000 bootstrap resamples of whole tasks with replacement, retaining all comparisons and both endpoints of a sampled task. It uses NumPy PCG64 and seed sequences $[20260906, r]$, with $r=0, 1, 2$ identifying the three comparison selections, and linear interpolation for the 2.5 and 97.5 percentiles. The process summaries likewise resample the 182 whole tasks while retaining zero-step exits and qualified rows. These specifications describe the recorded computation; it was not rerun for this English revision. [E18, E19]

The intervals assume independent, exchangeable task clusters and the same historical selection mechanism, assumptions not established by this archive. Shared dependencies, purposive investigations and incomplete semantic cov-

erage remain. The report consequently calls them reference intervals and retains point estimates, units and selection rules beside them. It does not reinstate the previously withdrawn paired t, GEE, Wald, McNemar or related significance claims.

The historical analysis produces a reconstruction of judgment identity, positive recorded-label change in selected sets, a narrower comparability-qualified result, a detailed-reading coverage analysis and a longitudinal selection check. The methods and tables above define each result. E19 identifies the original frozen numerical bindings; these historical observations remain separate from the A/B/C panel.

D Saved-code checks, definition witnesses and evaluation-method comparison

The saved-code study checks frozen candidates beyond their historical review labels: compilation, existence of the named target, permitted axiom dependencies, direct use of the conclusion as a premise and a selected definition’s behavior. The following sections retain the pair selection, execution programs, measured and unavailable states, and comparison of evaluation methods. [E18, E19]

D.1 Pair selection and the meaning of holdout

Selection starts from the original 465-relation frame across 250 tasks. Six pairs are purposively selected during method development: thm_14_3, def_12_1, thm_14_7, thm_14_8, prob_5_6 and def_10_5. An exclusion list contains 107 task identifiers, eighty-five occurring in the 250-task frame. Among the remaining 165 tasks, the procedure chooses a representative relation by the minimum seeded relation hash and forms four strata by task family and recorded requirements. Two tasks are selected per stratum. [E18, E19]

Set	Tasks	Selection rationale or stratum
Development	thm_14_3	Response explicitly did not execute a review
Development	def_12_1, thm_14_7	Partial progress requiring interpretation beyond labels
Development	thm_14_8	Changed completion target
Development	prob_5_6	Requirement and interface changes
Development	def_10_5	Explicit response to earlier comments
Holdout stratum 1	def_8_4, def_2_1	Definitions/examples with changed recorded requirements; eight eligible tasks
Holdout stratum 2	def_13_2, ex_3_1_2	Other definitions/examples; fifty-five eligible tasks
Holdout stratum 3	prob_8_2, prob_5_2	Theorems/problems with changed recorded requirements; twenty-one eligible tasks
Holdout stratum 4	prob_10_1, thm_2_6	Other theorems/problems; eighty-one eligible tasks

The requirements stratum is computed from two recorded comparisons: task/content and the contract for essential source proof steps. If either records a different value, the task enters the changed-requirements stratum. Otherwise it enters the remaining stratum, which includes noncomparable fields. “Other” therefore does not mean that identical effective mathematical requirements were established. This metadata rule controls sampling, not the semantic target qualification in C.4. [32]

Holdout selection does not inspect labels, source bodies, apparent improvement or execution success. Pair identities are fixed before later qualification. Missing, inapplicable and failed results are retained rather than replaced by easier pairs. The term holdout describes that selection procedure; it does not establish a representative random benchmark or guarantee that every selected task was wholly unexposed earlier.

D.2 Compilation and extraction are separate outcomes

The saved historical runner uses Lean 4.31.0. It first compiles each whole file, then seeks the unique non-private declaration whose final name component exactly matches the task identifier. It prints the target’s type and axiom dependencies, including transitive dependencies. It does not select an unrelated theorem merely because that theorem is easy to check. [E18, E19]

Of twenty-eight files, seventeen compile and sixteen yield the named target. All sixteen measured targets pass both saved axiom policies. The additional compiling file does not contain the required target declaration. Pair-level scoreability for this target-based check is:

Pair status	Development tasks	Holdout tasks	Scoreable endpoints
Both endpoints	thm_14_3, thm_14_8	def_8_4, def_2_1, prob_5_2, thm_2_6	12
One endpoint	Later prob_5_6	Later def_13_2, later ex_3_1_2, earlier prob_10_1	4
Neither endpoint	def_12_1, thm_14_7, def_10_5	prob_8_2	0

Thus six pairs have two scoreable endpoints, four have one and four have neither. A failed build does not provide an axiom score of zero: no eligible target was measured through that execution.

The two axiom policies are deliberately chosen checks. The permissive policy rejects `sorryAx`; the stricter policy permits only `propext`, `Classical.choice`, `Quot.sound`, or no axioms. They are not the project’s early and late historical policies. Passing the stricter policy implies passing the permissive one for an extracted target. Neither establishes that the declaration faithfully expresses the textbook problem.

Within the eight holdout pairs, five compile at both endpoints and one at neither. `def_13_2` changes from compilation failure to success; `prob_10_1` changes from success to failure. Both earlier and later sides therefore have six of eight compiling files, despite the individual changes. This is why a zero aggregate change does not imply unchanged artifact quality.

D.3 The historical runner and generic pair checker are different programs

The generic pair checker receives two supplied sources and a preselected declaration. It compares candidates only after their printed declaration types match in a fixed environment. It does not decide whether the selected type is the right textbook proposition or compare every semantic behavior.

The historical runner instead compiles each of the twenty-eight frozen endpoints, extracts the exact named target and assembles the four policy cells directly from endpoint results. It has no cross-endpoint type-equality gate. Historical findings use this latter program, whereas the constructed validation scenarios in D.8 use the former. Success on both historical endpoints must not be presented as evidence that a type-equivalence check was run. [E18, E19]

D.4 A direct conclusion premise: all pair states

A separate AI-assisted reading checks whether the public premises already provide the entire required conclusion. The reading records locations and identities, but has no independent expert audit. It targets a specific direct shortcut rather than all possible statement weakening or indirect relocation of proof work. A code-inspection result can therefore be available even where the saved compilation environment fails. [E18, E19]

For this check, the two rule sets come from the earlier and later endpoints’ saved requirements. Their effective prohibitions can be identical. In the table, tuple order is $q_{00}, q_{01}, q_{10}, q_{11}$: earlier code under each rule, followed by later code under each rule. A valid 1 means the direct-conclusion-premise defect was not found by this check; a valid 0 means the direct defect was found.

Task or tasks	Four-cell status	Reason or interpretation
thm_14_3	All uncertain	Bound inputs omit definition bodies needed to decide whether a premise restates the conclusion
def_12_1, def_10_5, def_8_4, def_2_1, def_13_2	All inapplicable	Selected main exports are definitions, not theorems
thm_14_7, prob_5_6, prob_8_2, prob_5_2, thm_2_6	(1, 1, 1, 1)	Neither version's main theorem has the direct conclusion premise
thm_14_8	Inapplicable, invalid, invalid, 1	An earlier proof-debt exception has no valid cross-version mapping
ex_3_1_2	Inapplicable, inapplicable, 1, 1	Earlier export is a definition; later named theorem has no public premises
prob_10_1	(0, 0, 1, 1)	Later source removes the full-conclusion premise but fails to compile in the saved environment

This direct-premise matrix contains fifty-six potential candidate/rule cells: twenty-seven measured, twenty-three inapplicable, four uncertain and two invalid. Only six complete numerical four-cell tables are available. These statuses are not alternate spellings of failure; in particular, a definition is not a theorem that failed a premise check.

The earlier prob_10_1 theorem takes precisely the desired equivalence as a premise and returns it. The later source removes that premise, which both rules prohibit. The code reading therefore supports removal of the direct defect. Its failed saved-environment build remains a separate negative result; the original dependency environment was not fully reconstructed. The report does not turn a visible source correction into certification of the complete repaired artifact.

D.5 Formal witnesses for the meaning of countable

The source text for def_2_1 adopts a convention requiring a bijection with all natural numbers. The earlier predicate uses the broader library concept `Countable`, which includes finite sets. The empty set distinguishes them: it satisfies the broader predicate but admits no bijection with the naturals, because zero has no preimage. The later candidate implements the selected bijection convention and gives the broader concept a separate name. [E18, E19]

The saved Lean witnesses check six propositions, expressed here in ordinary mathematical language rather than presented as newly executed code:

Witness	Checked proposition
1	The earlier predicate accepts the empty set
2	The written reference predicate rejects the empty set
3	The later predicate rejects the empty set
4	The later predicate agrees with the reference for every set
5	The earlier predicate does not agree with the reference for every set
6	The earlier and later predicates differ

Dependencies are empty or confined to the permitted foundational axioms. These are six propositions about one definition, not six independent tasks. The witnesses establish precise relations among formal predicates. Interpreting the task text as the selected formal reference is an AI-assisted source judgment without independent expert certification; the checks also do not certify unrelated content in the later file.

This result is nevertheless more informative than a generic statement that the definitions differ. It gives a distinguishing instance, establishes agreement with an explicit reference for the later predicate, and separates the formal proof of those facts from the interpretation that chose the reference.

D.6 What the four-cell comparison asks

Let $q_{ij} = g(C_i, R_j; E)$ be a valid score for candidate C_i under rule R_j in a fixed environment E , with $i, j \in \{0, 1\}$.

Candidate	Rule R_0	Rule R_1
Earlier C_0	q_{00}	q_{01}
Later C_1	q_{10}	q_{11}

The complete table can answer six related contrasts:

$$\begin{aligned}
D_{C|R_0} &= q_{10} - q_{00}, & D_{C|R_1} &= q_{11} - q_{01}, \\
D_{R|C_0} &= q_{01} - q_{00}, & D_{R|C_1} &= q_{11} - q_{10}, \\
\Delta &= q_{11} - q_{00}, & I &= q_{11} - q_{10} - q_{01} + q_{00}.
\end{aligned}$$

Two contrasts hold a rule fixed while changing code, two hold code fixed while changing the rule, and the remaining two concern the total diagonal change and interaction. A contrast is answerable only when actual measurements and verified logical relationships uniquely determine it. Unavailable cells are not filled with zero. The six questions share inputs and are not six independent task outcomes. [E18, E19]

The fair fixed-standard baseline evaluates both candidates under R_1 and uses verified rule equality or implication when available. For the axiom check, a strict-policy pass already determines the permissive-policy pass. For the direct-premise check, effective prohibitions can be identical. Extra cells add information only if they determine answers that this logically informed baseline cannot already determine. The comparison is therefore not against a deliberately uninformed baseline.

D.7 Historical answerability results and the bounded negative finding

Evidence set	Questions	Historical labels	Fixed R_1	Four cells	Four cells plus Shapley
Axioms: six development pairs	36	0	13	13	13
Axioms: eight holdout pairs	48	0	27	27	27
Direct premise: six development pairs	36	0	12	12	12
Direct premise: eight holdout pairs	48	0	25	25	25

Across the inspected historical pairs and attributes, the four-cell method and fair fixed-standard baseline determine the same answers to the same questions. A Shapley allocation distributes differences calculated from the same observations; it supplies no new input information. The result is a methodological negative finding for these checks, not an accuracy ranking or proof that four-cell evaluation never helps. [E18, E19]

A zero in the historical-label column requires a separate explanation. A broad semantic acceptance is not automatically a measurement of the present axiom or direct-premise attribute. When no valid mapping is established, the current comparison method cannot answer the specified numerical question from the label alone. It is not claiming that every historical reviewer abstained or was wrong.

D.8 Constructed validation scenarios are not historical successes

Seven deliberately constructed proof scenarios about $n + 0 = n$ check whether the generic comparison program handles cases with known arrangements. Across forty-two related questions, historical labels answer four, the fixed standard seventeen, the four-cell method twenty-six, and adding Shapley still twenty-six. The per-scenario gains over the fixed baseline are $[5, 0, 4, 0, 0, 0, 0]$. [E18, E19]

Scenario	Deliberate arrangement	Result preserved by the validation
1	Admitted proof becomes an external-axiom proof; earlier rule permissive, later strict	Both diagonal cells fail, while the permissive candidate gain and strict rejection remain visible
2	Same candidate change; both rules permissive	Candidate gain is one; rule difference is zero
3	External-axiom proof becomes a completed proof; earlier rule permissive, later strict	Permissive scores tie; the strict score improves
4	Admitted proof becomes an external-axiom proof; both rules strict	All four cells are zero
5	Completed proof compared with noncompiling code	Noncompiling endpoint has no axiom score
6	Later code proves another proposition	Target mismatch makes the comparison invalid
7	Completed proof compared with a missing file	Missing-material state is retained

The scenarios can reuse source files and outputs. They demonstrate that extra evaluations can be informative when the arranged rule/candidate relationships require them, while leaving the historical no-additional-information result unchanged. They are neither seven independent probability tasks nor seven new formalization successes. Previously withdrawn significance claims and Boolean teaching examples are not reinstated.

D.9 What these checks add to the system assessment

The saved-code study establishes concrete, limited properties: whether a file compiled, whether it supplied the intended named declaration, what axioms that declaration depended on, whether a direct conclusion premise was removed, and how two definitions behaved against a written reference. Different execution programs and unavailable states remain explicit. The rule comparison then asks whether added evaluation work actually resolves additional questions, rather than assuming that a more elaborate table must be more informative.

Full execution artifacts are indexed in the historical evaluation release and the E18/E19 frozen sources. The pair selection, measurements, witness statements, formulas and negative-result tables needed to interpret the study are included above.

E Glossary and source navigation

E.1 Terms and symbols for lookup

These definitions collect terms already introduced where they are needed. Project labels classify roles; they do not certify mathematical correctness.

Term	Meaning in this report
Lean / Mathlib	The proof assistant / its mathematics library
Source unit / task plan	A bounded textbook section or Problems section / the plan recording tasks and their dependencies
Staging / canonical corpus	Draft and temporary working artifacts / the official Lean task files updated through acceptance

Continued on next page

Term	Meaning in this report
Dependency / downstream caller	A result or interface a proof relies on / code using a result that may need repair when its interface changes
SQLite state	A rebuildable index of tasks and retained evidence, used for scheduling and maintenance
Task / parent	A textbook obligation / the original task owning a decomposed proof
Candidate / delivery	A version of formal code / the code and accompanying submission material
Solver root	A top-level attempt whose role calls and continuations share one execution clock
Main panel	The 33 selected configuration–task deliveries, distinct from all 41 development roots
Author / coordinator / host	AI implementation role / AI work-organizing role / a historical label whose actor or infrastructure referent must be determined from the record
Outer experiment agent	AI session in Codex managing experimental runs; distinct from the coordinator organizing proof work inside C
Host machine / runtime	Supporting software and infrastructure, rather than an AI author or coordinator; an unresolved intervention issuer remains unresolved
Sol / Astra / medium	Recorded gpt-5.6-sol / gpt-6-astra configurations; medium is a reasoning-effort setting, not a capability tier
Endpoint	One review or candidate at one end of a specified comparison
Semantic review	A model assessment of source faithfulness and proof obligations
Common evaluation / adjudication	Post-submission experimental assessment / a further judgment resolving specified disagreements
Proof spine	Essential steps of the supplied mathematical argument whose implementation is being checked
Application / ledger	Validating a bound review and applying eligible work to its task / the project’s state records
Binding / hash	A link to the specified object / an identifier of file content under a stated byte rule
Signature	The pair of result-evidence and review-input digests used in archive reconstruction
Bridge / translation	Historically broad auxiliary connection / the representation-conversion category
Proof debt / proof_debt_support	Mathematical work still outstanding / its support classification
OBL	A proof-obligation subtask, with its own historical records
Math Gate	A proof-route check controlling the next allowed actions
Support	Auxiliary code owned by one task or used by multiple actual consumers
Family / family_member	Depending on the source, related tasks or a book-number group / an implementation module of one parent
Public premise	An assumption a caller must supply to use an exported theorem
Named target	The non-private declaration selected by the exact task-name rule
sorryAx	Lean’s admitted-proof placeholder axiom
propext, Classical.choice, Quot.sound	The foundational axioms permitted by the saved stricter policy
$C; H; \text{sn}(\emptyset)$ in Appendix L.3	The June mathematical configuration (not experimental arm C); external upstream proof; first-row standardization scale
$N, m_N, p_{N,i}, G_{N,i}$	Number of coupon types; target count; new-type probability; geometric waiting stage

Continued on next page

Term	Meaning in this report
Z_N, a_N, b_N	Exactly standardized sum; relative scale; scaled mean error
$A_{ke,0}, A_{ke,1}$	Recorded-pass indicators at the two ends of comparison e in task k
q_{ij}	A valid evaluation score for candidate i under rule set j in a fixed environment
Evidence exit	The point beyond which a specified follow-up lacks qualifying evidence
Reference interval	A descriptive whole-task resampling range with the stated, unverified assumptions

E.2 Source identifiers, included evidence and local archive access

The report and appendix contain the methods, results, mathematical arguments and selected case evidence used in the assessment. The full `report_revision_12.zip` delivery additionally includes the targeted evidence supplement introduced in revision 10, the public-documentation excerpts added in revision 11, and selected existing candidate and service records supplied for Revision 12. The original source files retain their bytes and identifiers. The [evidence guide](#) and root-level `evidence.html` provide portable entry points; the latter opens in a browser after the bundle is extracted. The report manuscripts or their rendered PDFs do not contain these separate files. The Revision 20 package includes the English and Chinese manuscripts and PDFs, editorial checks, the retained evidence supplement and later verification records. The original relative evidence paths are preserved; archive-only materials still require the corresponding archive.

The bundle contains original opinions and records, not a new expert certification of their mathematical judgments. It does not contain the whole private textbook corpus, all historical candidates and logs, or the production database. Experimental records are provided for this private review; no broader publication permission is inferred. Public documentation remains separately identified in [E.3](#).

E01–E21 identify historical source groups, R the initial trajectories, N the second edition and U the 15:49 increment. Q/X/Y/Z/I retain their original package identities; a citation locates evidence, not an additional experiment. The wider local archive resolves these identifiers through `SOURCE_LINKS.json`, `ARCHIVE_REFERENCES.json` and its original manifests. Those locators require that archive and are not public download addresses. The supplement’s own A/B/C/D/E source codes are file labels, not the experimental configurations.

ID	Source	Use in this report
E01	Supplementary investigation of the early workflow	Components, active entry points, commits, and execution limits
E02	Mid-period obligation, support, and Math Gate investigation	Checkpoint correspondence, timing, and control boundaries
E03	Bridge, support, and family history	Type split, shared support, and implementation families for one task
E04	Obligations and parent targets	Absorption events, subtask responsibilities, and successive mathematical checks
E05	Successive <code>thm_9_5</code> proofs	Reviews 29-32, candidates 38-41, and later file separation
E06	Downstream timing for <code>thm_9_6</code>	Calls, connection to the final derivation, and route requirements
E07	<code>prob_14_8</code> and <code>prob_14_10</code> investigations	Partial results, a global limit, and consumer bindings
E08	<code>def_8_5</code> and consumer migration	Probability conditions, witness representation, and migration scope
E09	Upstream confounding in <code>thm_11_8</code>	Identical main files, different upstream content, and missing earlier version

Continued on next page

ID	Source	Use in this report
E10	Current workflow contract	Math Gate, builds, review, and application responsibilities
E11	Current state contract	Parent completion types, retired obligations, and formal boundaries
E12	Local mirror of the coauthored paper	Workflow background; no certification of identity with the public version
E13	v0.7.5 entry point and typesetting sources	Layout reference and earlier report identity
E14	Original early materials and Git objects	April request, baseline code, early runs, and May entry points
E15	Current project AGENTS.md	Present responsibilities and authority rules
E16	Current Math Gate implementation	Program blocking, risk triggers, and older pilot identifiers
E17	Candidate-one design notes	Scope of the unexecuted local comparison
E18	Complete Chinese LaTeX, README, and CHANGELOG for v0.7.5	Substantive earlier content and removal decisions
E19	Generated numerical bindings and frozen analysis outputs	Earlier report numbers and their source-field paths
E20	Central limit, coupon-collector, and target-boundary investigations	Mathematical chain and counterexamples
E21	Shared entry point and integration notes	Integration requirements, historical corrections, and execution scope

The original manifest filenames and archived locations remain in `manuscript/evidence/README.md`, an unchanged historical guide. The Revision 12 evidence guide distinguishes those locators from files supplied in that bundle. References introduced in revision 10 map to the following included records; exact run identifiers and fingerprints remain in that evidence layer.

Reference	Evidence used here	What is reproduced in this appendix
[30]	Supplement A01–A06, A10–A21, A30–A40, AP01–AP11 and selected original opinions	Common-evaluation rules, inputs, session settings and stage-specific corrections in N.6
[31]	Supplement B01–B12 and the run index	Candidate selection, version roles, input correspondence, allowances and missingness in N.1/N.5
[32]	Supplement C01–C07 and E01–E02	Target-decision procedure and examples in C.4 ; metadata sampling rule in D.1
[33]	Supplement D01–D05	Review-recovery sequence and unresolved session attribution in G.6

The supplemental files above are under `manuscript/evidence/r9_rewrite_evidence/`. For the bounded Revision 12 additions, `verification/MAINTENANCE_SCOPE.csv` and `CANDIDATE_DELIVERY_INDEX.json` locate the L3/L7 deliveries; `L1_MAP.json` and `H1_MAP.json` locate the recorded transfer and binding recoveries. `verification/L2_SERVICE_AUDIT.csv` maps the six services in G.18 to their full identifiers, original feedback, recorded caller context and candidates. The copied Z015 annotations and Z016 coverage record are under `verification/supporting_records/archive/`. [37]

These maps are editorial navigation and identity checks, not new mathematical verdicts or evidence of production landing. Some underlying source records remain in Chinese; G.18 presents the six-service findings in the corresponding English and Chinese appendices. All printed package paths are relative to the extracted bundle. Absolute paths inside preserved records remain historical locators, not public download addresses. Packaging and verification times are not new experimental observations. The original comparison cutoff remains unchanged; the separately dated 13 September evidence is indexed in O.

E.3 Public project documentation

The following documents supply the project overview and public software entry points in the main report. They were read at the same fixed public repository revision on 12 September 2026. Links below resolve to that revision; the [evidence guide](#) links to the unchanged documentation source guide, selected verbatim passages and file identities in `manuscript/evidence/project_docs/`, separately from the experimental evidence.

Document	Content used in the report	Reference
Architecture	Source-to-task lifecycle; Python, Lean and SQLite roles; staged work and the canonical corpus	[34], [36]
Phase 2 workflow	Local repair versus route diagnosis; Math Gate; batch planning; review application	[34]
Workspace state	Rebuildable evidence index; current-code coverage; canonical updates and recovery	[34]
Dependency decision trail	Source-unit plans, declared reliance, problem support and reasons for imports	[34], [35]
Interface dependency policy	Textbook definitions, translation theorems and subsequent Mathlib use; notation versus code dependency	[35]
Semantic review criteria	Source-to-code mapping, listed callers and the distinction between infrastructure and task proof	[35]
Workflow demonstration	Isolated exercise of production functions, real compilation and recorded teaching reviews	[36]

The overview uses these documented mechanisms; the case histories supply evidence of actual mathematical work and failures. The demonstration’s default review replay is not a new model-quality measurement. The documented source-reading stage did not execute the demonstration or the production workflow. The source account is retained from the earlier manuscripts; this edition does not add a new repository inspection.

F Contributions and editorial scope

F.1 AI and human roles

AI assisted historical candidate generation, revision, review, archive reconstruction, source reading, calculation checks and bilingual writing. Shuo Deng designed and implemented task organization, workflows, review mechanisms, evidence maintenance and research analysis, and coordinated the assistance. Kenneth W. Shum contributed clarification of the textbook’s mathematical intent. Historical runtime reviews and retrospective readings are separate events.

The single-evaluator direct-premise check, target-comparability judgments and case interpretations have source trails, but the archive lacks a comprehensive independent human-expert adjudication. Role separation and hash binding make provenance inspectable; they do not make model opinions formal ground truth.

The coauthored paper *From Lecture Notes to Lean: Formalizing a Textbook on Probability Theory* provides project and mathematical background; its local mirror is indexed as E12. It does not substitute for the frozen configurations or establish their comparative performance.

F.2 Current edition and earlier changes

Revision 23. Revision 22 is the editing baseline. Section 1 is preserved verbatim; Section 2 retains the project and role explanation with necessary additions for the two batches and common assessment. Sections 3-5 and the discussion incorporate new outcomes, diagnostics, repairs and evaluation corrections; Appendix R.3 retains its historical analysis. This edition reconciles evidence and criteria for 66 items, runs two original-environment caller checks, and checks bilingual text and PDF output. Q.6 locates the outputs. The Revision 22 and earlier notes below are historical editorial records, not checks newly performed in this edition.

Revision 22 (14 September 2026). This edition implements the reading revisions to Chapters 3 and 4: direct reporting of completion, consistent current pass/fail labels, times for all eleven groups with original-round and cumulative effort distinguished, and revised explanations of tool-error counts and the L2 case. Appendix P records timing reconstruction and comparability checks. Both languages and both appendices are updated. This edition uses the existing experimental results through 13 September and does not yet incorporate the second batch.

Earlier editions. Revision 19 added Shuo Deng as first author and Kenneth W. Shum as corresponding author; it retained Revision 18’s substantive text and results. Revision 18 had checked A-M1 completion and timing and B-H1’s scope-specific judgment, and corrected the presentation of the completed A-L7 continuation; see O.5. Revision 12 had improved evidence access and presented the six L2 service assessments. Their original release notes and check records remain with those editions. A check reported for an earlier edition describes that edition, rather than a new verification in the present one.

The companion main text is *How probability proofs are completed*. The two PDFs explain the methods and findings; the separately preserved evidence package contains the underlying indexed records. E.2 explains access to those materials.

G Mechanism, mathematical-repair and criterion evidence

This appendix reuses completed case analysis and exact sources while retaining distinct observation windows. The corrected 12 September comparison follows Y001-Y003. The 13 September continuation and responsibility audits in O separately update the corresponding conclusions; earlier submission times, absent common evaluation or original labels describe their contemporary states only. M2 repair is in G.14.1, two written L7 counterexamples in G.16, and the uniform-assessment denominator in N.4.

G.1 L5: a substantial helper product without an established adoption chain

L5 concerns maximal coupling for continuous densities. The parent delegates the common minimum density, residual densities, joint measure, marginals, mismatch probability and related work. Although the assigned role is called a support author, its mathematical scope is much broader than one small lemma. [R02, R03; T03]

The helper produces a 534-line candidate and a passing build receipt. The parent receives host paths and hashes, cannot find the text or patch inside its own container, and continues local search and implementation. Helper and parent candidates have different naming and organization. Similar mathematical routes may result from the same textbook and parent request; they do not identify an adopted implementation.

The investigation therefore changes the earlier handoff label and the framework’s adoption criterion. It also distinguishes the outer transfer diff from the inner patch that changes the target file. A transfer package can exist and a patch can be checked without establishing that the target candidate incorporated it. This is a correction to the analysis itself, based on the relation among the actual artifacts rather than their reported status.

The helper’s 554.094 seconds of process time remains incurred work. The earlier accounting also records 2,520,780 input and 20,310 output tokens for that helper. No established adoption chain does not mean zero cost, and it does not prove the absence of every indirect contribution. These measurements belong to that recorded helper and must not be added again to a root account that already includes it. [R02]

G.2 Earlier B-L2: short duration does not explain the failed single review

The earlier B version searches variance and covariance interfaces, works around missing patch tools, writes proofs, and builds both targets and the bundle. It voluntarily reports completion after about 256.532 seconds, with about 6,943 seconds of its budget remaining; no timeout kill explains that ending. [R02, R03; T04]

The common single review contains the important comparison. A uses `variance_sum'`; B uses `variance_fun_sum'`; both use uncorrelatedness to cancel off-diagonal covariance terms. Saved library code shows that the latter wraps a conversion from the former. A's reviewer explicitly permits the general expansion, while B's reviewer cites that level of expansion as a failure reason.

At that earlier observation point, the analysis establishes inconsistent criteria, not a complete corrected evaluation of the whole B bundle. The sequence “short run, failed review” does not establish that premature stopping caused an omission. Nor does the short span establish superior efficiency. Current L2 acceptance and the later r4 candidate are reported separately in G.15; this earlier runtime and provisional judgment are not substituted for them.

G.3 B r3: a completed service can return no mathematical product

The earlier lead-author innovation run requested support on conditional-expectation interfaces. A recursive search timed out and stopped the helper's container; the final reply reported no valid patch or compilation, despite the parent tool labeling the transfer completed. The author continued the proof independently. When later Chinese checking returns suffered encoding faults, it switched the next request to ASCII English. A failed recursive help request stopped before model startup and therefore was not another executed helper-model call. G.9.1 gives the integrated mathematical and checking sequence. [R02, R03; T05]

The initial inventory ended before the parent and final general reviewer returned. Later records established submission, all four real solver roles, technical checking and common dual-review acceptance. The completed record supplements that earlier observation window rather than erasing its faults. [R09–R11, R15, R16]

G.4 Initial C: a role summary omits executed checks

Initial C executed a coordinator, a writer and two checking roles, although its status summary listed only the author. Both checks produced opinions but failed during export, and the coordinator submitted while disclosing that it had not obtained independent checking. G.9.2 retains the candidate, sequence, remaining budget and later evaluation of the same artifact. This accounting discrepancy shows why a summary of visible roles cannot replace the actual dispatch and return records. [R02, R03, R12, R17; T06]

G.5 The L1 starting point: why innovations form a martingale

The same-problem comparison uses textbook problem `prob_13_9`. On a probability space, an observation process satisfies $X_0 = 0$, with each positive-time observation measurable and integrable. Let $\mathcal{F}_0$ be trivial and $\mathcal{F}_n = \sigma(X_1, \dots, X_n)$ represent observations through time n . An innovation is the observation minus its conditional prediction from past information:

$$Y_0 = 0, \quad Y_{n+1} = X_{n+1} - \mathbb{E}[X_{n+1} \mid \mathcal{F}_n], \quad S_0 = 0, \quad S_n = \sum_{k=1}^n Y_k.$$

The agent must define these objects, prove integrability and adaptedness of innovations and partial sums, and derive the martingale conclusion from the source premises. Public mathematics libraries and general support are available, but callers must not be asked to supply those intermediate conclusions. The deliverable is actual Lean definitions and proofs with dependencies. [N01, N03–N06]

The central mathematics has two steps. Linearity and self-conditioning give zero conditional innovation mean. The partial-sum recurrence and adaptedness then give the martingale identity:

$$\mathbb{E}[Y_{n+1} \mid \mathcal{F}_n] = 0, \quad \mathbb{E}[S_{n+1} \mid \mathcal{F}_n] = \mathbb{E}[S_n + Y_{n+1} \mid \mathcal{F}_n] = S_n \quad \text{almost everywhere.}$$

The Lean implementation must also handle inclusion between measurable spaces, integrability, indexing, and the distinction between function equality and almost-everywhere equality. Main [Appendix R.2](#) and [Appendix G](#) explain how the workflows performed this work, beyond their final pass labels.

G.6 A: natural filtration and recovery of review delivery

A’s entry instructions required the author to proceed through generated materials, building, independent review, and application. Within that process, the author chose search terms, library interfaces, and proof implementations. A concrete distraction appeared at the start: although the prompt specified `prob_13_9`, it also contained the medium-task instruction to complete `thm_13_17` before `thm_13_18`. The author queried those identifiers, could not find them in the private catalog, and returned to the only schedulable task, `prob_13_9`. This detour followed a prompt error; it does not establish that A’s normal workflow necessarily incurs the same search. [N01: author prompt line 3; first bridge lines 18-32]

The engine’s generated searches returned some irrelevant material. The author then read library sources for natural filtrations, adapted processes, and conditional expectation. A request to a support author failed while copying the parent workspace, before a helper model executed. The main author continued independently. Two initial file-writing attempts encountered a missing patch command and a missing Python alias, so subsequent build checks did not check the intended new proof. Once the proof was actually written, Lean identified three specific interface errors: the zero-function integrability lemma needed explicit arguments; the natural-filtration adaptedness lemma had received an extra argument; and a function-equality theorem had been used as though it supplied an almost-everywhere hypothesis. The author read the exact signatures, corrected these uses, and obtained a successful build for candidate four. [N01: first bridge lines 38-57]

A used Mathlib’s `Filtration.natural`, which includes observations from zero through n . It accounted for time zero by using $X_0 = 0$ to prove that the initial filtration was trivial. It then proved integrability and strong adaptedness of innovations and partial sums, and derived zero conditional innovation mean using subtraction and self-conditioning. The final theorem applied the general zero-conditional-increment martingale criterion after proving $S_{n+1} - S_n = Y_{n+1}$. Its public assumptions concern the positive-time observations; measurability at zero is derived internally. This route performs the required mathematical work. Its use of a general martingale criterion, instead of B/C’s local interface, is not evidence of an omitted obligation. [N03: lines 33-142]

The remaining process was largely blocked by independent-review execution and return handling. Of seven general-review requests, one was rejected at entry; five reviewer sessions have completion receipts and actual pass opinions; the last started but has no completion receipt. The support request mentioned above failed before execution. The five opinions did not reach a readable author-side result file through the normal path. Workspace exports failed, later readonly checks rejected returns, and one successful review with inline source materials still returned only a host path and hash. The author searched the local workspace and found no response file. It consequently waited, retried, or reported that it could not apply a review, although a retrospective analyst can read the completed opinions. [N01-N02]

The host recorded recovery of an existing review JSON into the result path, reporting equal source and destination hashes. The controller later observed a task pass without resetting the root deadline. The retained review JSON matches the referenced reviewer’s final public output. However, the session named in the recovery receipt differs from the session in that review directory’s actor receipt and opening log event. The supplied evidence confirms the conflict but does not independently re-read the historical destination or establish which field caused the attribution error. [N02, N11; 33]

The supported sequence is usable-result recovery followed by an observed pass, with session attribution still unresolved. It includes host intervention, waiting and engineering faults in A’s recorded 5,151.280-second root duration. Those costs and the recovered review are not additional proof improvements.

G.7 B r4: readable feedback enters an interface revision

After implementing and checking an initial proof, B r4’s author requested general review. The return contained readable Chinese text. The reviewer passed the mathematics and noted one nonblocking interface issue: the final theorem required measurability and integrability at every natural-number time, whereas the task required them only at positive times. Since $X_0 = 0$, the time-zero properties follow immediately. This was an opportunity to align

the calling interface with the task, not a discovery that the proof assumed an extra mathematical conclusion. [N08: author lines 40-41]

The author then changed the final theorem to accept only positive-time premises, deriving all-time measurability and integrability inside the proof before calling the existing helper lemmas. A local name replacement introduced a typo; after correcting it, the author rebuilt the module and full entry point and repeated the axiom checks. The final target differs from the reviewed version. Specific advice, a parent-task return, the corresponding edit, and checks on the new version form a traceable sequence here. There is still no counterfactual experiment identifying the advice’s net causal contribution. [N08: lines 42-49 and final code from line 201]

B r4 involved two real roles and a 714.466-second root duration. Its own common evaluation reviewers both passed it; that result is not borrowed from B r3. Helper lemmas retaining all-time premises do not undo the final interface repair: the final proof actually derives their required inputs from the source assumptions. [N08-N09, N11]

G.8 C r3: review enters the submission decision

This C r3 submission sequence was captured in the 11:50:33 supplement; its source window remains in [Appendix H](#). New C r3’s coordinator read c1, ran a build, and requested general review. Unlike initial C, coordination record 34 returned the complete Chinese review JSON, its pass opinion, and the result that the readonly workspace was unchanged. The coordinator’s subsequent public message explicitly acknowledged the build and review passes. Its next action submitted the same c1 with no unresolved items. The retained final target is the same submitted candidate. [N10: coordination lines 27-36; public events lines 54-65]

The sequence establishes opinion production, inclusion in the return, readable exposure, explicit acknowledgment, and subsequent submission. There was no new writer assignment or code version after review. This case therefore shows feedback entering a submission decision, not feedback repairing code. The actual roles were coordinator, writer, and general reviewer; no mathematical-consultation call occurred. That is evidence of optional-service use in one run, not evidence that omitting mathematical consultation is generally better. [N10-N11]

New C r3’s root duration was 708.714 seconds, with 4,691.286 seconds remaining at submission. This solver supplement covers submission and internal review; its common evaluation is separately recorded in main [Section 3.1](#) and [Appendix K.9](#). The closeness of 708.714 and B r4’s 714.466 seconds does not establish equivalence or justify a repetition count for subsequent experiments.

G.9 Earlier B/C: later acceptance does not remove an earlier information gap

The next two runs belong to earlier versions and explain what the current improvements address. Later evaluation does not retroactively give a model access to an opinion that had not arrived when it submitted.

G.9.1 B r3: the author finished the proof; readable review requested no substantive repair

B r3’s author first read the task, source, and public interfaces, then requested help finding exact conditional-expectation lemmas and a proof route. The support author’s recursive search timed out and its container stopped; it delivered no valid patch. The returned status said completed and some text was corrupted, but neither fact establishes completed mathematical help. The main author continued searching for subtraction, integrability, and adaptedness interfaces, tested types in a small checking file, and implemented the task using the one-indexed observation filtration. [N04-N05]

The mathematical structure matches A’s. B used the last coordinate of the finite-history map to prove measurability of X_{n+1} , then filtration monotonicity to lift conditional-expectation measurability to the next time. Partial-sum properties followed by recursion. Zero conditional innovation mean used subtraction and the repeated-conditioning lemma `condExp_condExp_of_1e`; the final theorem constructed the fields of the local martingale definition `def_13_7`. A’s general criterion and B’s field-by-field construction discharge the same final mathematical responsibility. [N04: final code lines 44-186]

The main repairs concerned finite-history coordinate types and finite-sum representation. The author tested the direction of the arguments to `Measurable`, adjusted the finite index, and obtained a buildable version. A mathematical-review request then failed while encoding the return, so the tool delivered no opinion text. The author subsequently requested general review in English and received a readable no-defects response covering indexing, integrability,

adaptedness, zero conditional mean, and absence of extra obligations. Feedback arrival is established here, but the opinion identified no substantive defect to repair. [N04: lines 16-35]

The author then tried to remove an incidental simplifier warning, introduced a local structural error while editing, and restored the induction step and successful build. The final candidate independently received a dual-review pass at the common evaluation. The sequence is therefore a small cleanup after a passing opinion, not a mathematical gap repaired by that opinion. The failed mathematical-review return must not be treated as feedback the author read. B r3 involved four real roles and a 940.836-second root duration, already including time spent waiting for help. [N04-N05, N09, N11]

G.9.2 Initial C: the coordinator saw the candidate but not the checking opinions

C's coordinator first read the task, source, common rules, and supporting filtration, martingale, and history-map code. It then gave the writer a concrete assignment: reuse the one-indexed natural filtration and local martingale interface and prove the innovation and partial-sum properties. The writer had tools for searching and implementing proofs; the coordinator organized the work through registered candidate, build, review, and submission services. C's autonomy is thus described at specific decision points: the coordinator selected services and their order, while the writer selected lemmas and local implementations. [N06: coordination lines 1-18]

The writer encountered a missing patch command and patches with invalid formatting or mismatched context. Once the candidate was written, actual Lean errors concerned finite-sum evaluation, measurability arguments, and zero-function representation. It repeatedly inspected local lines and lemma types, repaired the target, built the module, and then built the aggregate entry point. The final code used the last history coordinate to prove observation measurability and derived the filtration's inclusion in the ambient measurable space from the positive-time premises. Innovations, partial sums, and zero conditional mean each had actual lemmas, followed by a field-by-field proof of the concrete martingale conclusion. Tool failures and Lean repairs interleaved; each unsuccessful return is not a failed mathematical strategy. [N06: writer lines 12-49 and final code]

After receiving c1, the coordinator read the complete target file, its explanation, and the aggregate entry point. It called the fixed build service and received a pass, then requested general review and mathematical checking. Both checking models actually ran and produced, respectively, passing or viable-route opinions. But readonly-filesystem export failures meant the coordinator received errors instead of those opinions. It checked status, read the build result, and submitted c1 as believed complete while listing the failed independent checks. Root duration was 941.191 seconds, with 4,458.809 seconds remaining at submission. [N06: coordination lines 21-38; N07]

This was an explicit submission with remaining budget and acknowledged checking-delivery failure. It cannot be described as submission after successful independent review, nor can remaining budget alone establish unjustified abandonment. Later common evaluation reviewers both passed the same candidate. That is evidence about the submitted candidate under later evaluation; it does not change what the coordinator knew when submitting. [N06-N07, N09]

G.10 Different work paths through the same mathematical task

The table expands acceptable final candidates into different concrete processes.

Comparison point	Initial A	B r3	Initial C
Core mathematics	Natural filtration, innovation and partial-sum properties, general martingale criterion	One-indexed history filtration, the same properties, local martingale definition	One-indexed history filtration, the same properties, local martingale definition
Main decision maker	Author constrained by review gates and controller state	Author arranges search, help, checks, and submission	Coordinator arranges writer, checks, and submission; writer implements
Feedback arrival	Several opinions produced; author-side delivery blocked; host recovery	Support failed; mathematical-review encoding failed; general-review text arrived	Both checking models finished; coordinator received export errors
Submission or ending	Controller observed pass after recovery	Author submitted; later dual-review pass	Coordinator submitted with missing-review disclosure; later dual-review pass

A shows how a review gate and delivery failure can interact. B shows an author completing the proof without a useful support patch and choosing another review. C shows a coordinator making decisions across separate writing, checking, and submission services. All depend on actual tool capability and information delivery; role names alone do not establish what happened.

These observations do not identify the net benefit of one organization. A had prompt and runtime-repair interference; B r3 already incorporated baseline revisions; C used an additional Astra coordinator; and timing and early endpoint conditions were not fully matched. B r3, initial C, and B r4 have identical endpoint category bindings for task, source, and upstream materials, supporting a comparison of their mathematical obligations. This does not retrospectively equalize their solving conditions. A's later dual review is reported separately in main [Section 3.1](#) and [Appendix K.9](#); it does not erase the initial trial's provisional single-review status. [N01-N11, N13; R06-R08]

G.11 M1: complete assistance adoption and delivery-note revision

For a martingale X and stopping time T , the stopped process is $X_n^T = X_{T \wedge n}$. The first target proves that this process remains a martingale; the second obtains, under the specified conditions,

$$\mathbb{E}[X_T] = \mathbb{E}[X_0].$$

A bounded stopping time can be handled in finite time. An almost-surely finite stopping time with a uniformly bounded process, or an integrable stopping time with bounded increments, requires passing finite stopped-expectation identities to the limit. The integrable control below permits dominated convergence. For potentially infinite stopping times, the code must define a valid representative instead of using an undefined X_∞ .

M1 requires two connected results. The agent must first define a stopped process and prove that it remains a martingale, then establish the stopped-expectation conclusion in three cases: bounded stopping time; almost surely finite stopping time and a uniformly bounded process; and integrable stopping time with bounded increments. The third case requires constructing an integrable bound rather than asking the caller to supply the desired domination. For example,

$$|X_{T \wedge n}| \leq |X_0| + cT,$$

connects almost sure convergence with expectation. The Lean implementation must also handle infinite stopping times, measurability, and integrability. [U01, U02]

B r4 delegated the complete two-task implementation to a helper author. The helper produced a patch in an isolated copy; the principal author received and read it, applied the target changes, built them, and requested mathematical review. The two files reconstructed by the patch match the code finally evaluated line for line. This establishes a complete production, delivery, application, and validation sequence. The principal author still performed assembly, tool recovery, and checking; the work cannot be reduced to one short answer. [U02]

A detail matters for accounting: the principal author's first composite build command exited with code 0, but its standard error said that the build command was not found. The author then located the absolute executable path and completed a real successful build. An analysis based only on the outer exit code would misclassify the earlier tool failure as successful validation. [U03]

C r3 also delegated the whole two-task group, read the returned candidate, built it, and checked the targets in sequence. A second candidate followed, but a comparison of all visible files shows that only the delivery document changed; the mathematical source was identical. The coordinator requested missing public declarations and premise descriptions. The internal reviews had not identified a mathematical defect, so the revision concerns delivery documentation rather than a corrected proof error. [U04]

C's two target reviews reused one review session. They are two actual calls, not two independent reviewers. The later common evaluation has its own two evaluations. This distinction affects role counts and independence claims without automatically negating the value of internal target-by-target checks.

All three M1 candidates passed their post-submission evaluations, establishing observed completion of this group's mathematical requirements under all three organizations. That does not establish equivalence: A's initial run had different runner failures and recovery, C added a coordinator using a different model, and matching endpoint outcomes do not equalize costs or stopping rules. [U01]

G.12 M3: construct almost-sure convergence on a common space

The construction sends the same uniform random number through different quantile functions. For distribution functions F_n , F and U uniform on $(0, 1)$, it can be written as

$$\begin{aligned} Z_n &= F_n^{-1}(U), & Z &= F^{-1}(U), \\ Z_n &\stackrel{d}{=} X_n, & Z &\stackrel{d}{=} X, & Z_n &\longrightarrow Z \text{ almost surely.} \end{aligned}$$

The inverses are generalized inverses; the proof handles discontinuity levels, measurability, and endpoints. The conclusion concerns newly constructed variables, not almost-sure convergence of the original X_n on its original space.

M3 is the Skorokhod representation theorem, textbook theorem 10.8, rather than the separate subsequence result for convergence in probability. Given convergence in distribution of real-valued random variables, it must construct variables on a common probability space, preserve the distributions of both the sequence and its limit, and prove almost sure convergence. The final implementation uses generalized inverse distribution functions on the unit interval, establishes the marginals, and removes countable exceptional levels and endpoints to obtain convergence. A helper working on the full task produced code that the principal author actually adopted. Adoption is established through the relation between the helper product, principal-author actions, and final code. [U05]

The case also distinguishes two kinds of evaluation failure. The second common-evaluation review was recorded as inconclusive because its confidence field had an invalid type, followed by a passing adjudication. There was no corresponding mathematical counterexample. The failed-format call still belongs in the resource account. [U05, U06]

Specifically, the confidence field contained an object where the interface required a number. Adjudication considered the inconclusive result and candidate evidence; this should not be retold as discovery and repair of a mathematical counterexample. The format problem did not produce a new proof repair in the candidate. [U06]

G.13 H1: inversion enters uniqueness and a standard-Cauchy application

The five duties are the bound $|e^{ix} - 1| \leq |x|$, inversion, uniqueness via inversion, a Cauchy-average application, and an integer-valued criterion. The narrative first reconstructs A's mathematical reuse: its complete candidate passed the original adjudication, while the two original reviewers disagreed over standard Cauchy versus a general parameter

family. The later subsection reports uniform scope reassessment of all three candidates, without treating the old adjudicator’s reading as the uniquely established task meaning. [Q057-Q060; U13][Z025, Z026]

Completed conclusion	Actual use in a later target	Current scope
Characteristic-function inversion	The uniqueness proof calls inversion	Internal delivery; interval endpoint atoms are handled
Characteristic-function uniqueness	The standard-Cauchy average application calls uniqueness	Internal delivery; public parameters fix standard Cauchy
Integer-valued criterion	Imports the uniqueness file but does not call the theorem	A separate terminal proof route; internal delivery

Reuse therefore means a locatable invocation in the later derivation. An import graph only records a possible code dependency.

The most informative H1 evidence shows how an intermediate proof entered later work. The author first requested the complete inversion proof while also restricting that support request to read-only work. The response identified the Dirichlet-integral gap without a usable patch; that alone cannot establish an inability to solve the whole task. The author narrowed the next request to that integral’s limit and uniform bound, obtained built support, incorporated it into the draft, and then requested completion of the remaining inversion argument. [U07]

The third helper completed the remaining proof, but file export failed. Only after the host recovered the original product could the principal author read and apply it, build it, and obtain independent internal review. The mathematical work existed while its delivery was temporarily blocked. Final success included host recovery and must not be described as uninterrupted autonomous completion. [U08]

The final inversion code contains finite integral interchange, the oscillatory kernel’s different pointwise limits, dominated convergence, and half-mass endpoint terms:

$$\lim_{T \rightarrow \infty} \frac{1}{2\pi} \int_{-T}^T \frac{e^{-iat} - e^{-ibt}}{it} \phi_\mu(t) dt = \mu((a, b)) + \frac{1}{2}\mu(\{a\}) + \frac{1}{2}\mu(\{b\}), \quad a < b.$$

The subsequent uniqueness theorem actually invokes this inversion result. It obtains equal interval probabilities at endpoints without atomic mass, then uses dense endpoint selection and a measure-determination argument to establish equality in distribution. This is stronger evidence than an import: the earlier theorem’s conclusion enters the later derivation. [U08, U09]

The first uniqueness review returned a pass, but the application gate rejected it because its input hash did not match. The same reviewer corrected the binding and associated explanation; the principal author then applied it successfully. The candidate source did not change. This is binding validation and recovery, not a mathematical proof repair. [U09]

The two newly completed terminal applications further specify the chain:

- **Cauchy sample average.** The code derives the standard Cauchy characteristic function $e^{-|t|}$ through Fourier integration, then uses the independent-sum product formula and scaling to show that the average and a single term have the same characteristic function. It explicitly invokes the newly completed uniqueness theorem. The public theorem currently fixes the standard Cauchy distribution. The later adjudication accepts this implementation under the frozen scope while retaining the objection; the code must not be described as covering arbitrary location and scale parameters. [U14]
- **Integer-valued criterion.** The other application proves that $\phi_X(2\pi) = 1$ if and only if X is integer-valued almost surely. It uses the real part of a unit-modulus complex exponential being at most 1, and equality of the integral to 1 forcing that real part to equal 1 almost surely. This yields a complex exponential equal to 1 almost surely. Although the file imports uniqueness, its proof body does not invoke that theorem. The two applications follow different actual dependency routes. [U15]

By 15:49, all five targets have internal review-registration receipts. Their candidate hashes, applied-source hashes, and final joint-build output hashes agree. The last two applications completed at 15:20:48 and 15:31:31. The final joint build establishes simultaneous availability of the five targets in one source state. This fixes the code state of internal delivery at that time; the later whole-group evaluation is updated at the start of this section. [U13]

The supported process finding is that this run reached complete internal delivery through re-scoping assistance, reusing completed analytical support, and recovering blocked transmission. There is no comparison showing what

would have happened without help or recovery. Nor does this case establish that narrower requests are always better: B’s M1 and A’s M3 also adopted products delegated at a broader task scope.

G.13.1 H1 core completion and extension scope

Current acceptance requires all five targets and interprets the unparameterized Cauchy application using the textbook-supported standard-Cauchy core. The complete A same-root continuation, B and C all pass. A’s earlier partial snapshot remains incomplete and is not conflated with the later complete candidate. [Y001-Y003]

Complete candidate	Current core	Delivered Cauchy scope
A same-root continuation	Pass	Standard Cauchy
B r4	Pass	Zero location, arbitrary positive scale
C r3	Pass	Arbitrary location and positive scale

These scope differences remain mathematical results, but unstated extensions no longer define core failures. Chapters 1 and 6 supply standard-Cauchy semantics. Table 9.1 on printed page 154 additionally gives $b/[\pi(1 + b^2x^2)]$, supporting B’s centered scale extension. Exercise 9.6 on page 161 does not explicitly quantify location. Lack of an arbitrary location parameter therefore does not by itself refute B’s completion of the accepted core. [X003, X012, Y001]

The three candidates previously received six independent initial reviews and three necessary adjudications. B’s raw adjudication already accepted the standard scope mathematically, but a TeX quotation escape failed peripheral validation. The repair handles the determinable correspondence and reuses that opinion without new model calls. Arbitrary-location generalization remains supplementary coverage. [Z025, Z026, Y005]

This process also exposed missing evaluator inputs: seeing only anonymous source code cannot establish that the author omitted delivery notes. Before the necessary adjudications, each candidate’s actual original submission materials were supplied; no author documentation was invented. This shows how an incomplete evaluation view can create apparent failure, but does not establish an exhaustive audit of every other candidate’s documentation. Historical labels and pre-repair tables are retained in [Appendix K.8](#).

G.14 Mathematical repair: derive a premise inside the proof

Assistance adoption, documentation changes, and review-registration recovery explain collaboration. The next two cases instead show how public mathematical obligations were completed. They involve both build feedback and source correspondence, requiring more explanation than saying that the code was fixed.

G.14.1 T01: M2 contains search, interface tests, and a substantive proof repair

M2 concerns the strong law under fourth-moment conditions. M2 corresponds to textbook theorem `thm_11_7`: independent random variables with a common mean and a uniform raw fourth-moment bound have sample means converging almost surely to that mean. Setting aside the code’s indexing convention, the mathematical requirement is:

$$\mathbb{E}X_i = m, \quad \sup_i \mathbb{E}|X_i|^4 \leq c \quad \Rightarrow \quad \frac{1}{n} \sum_{i=1}^n X_i \longrightarrow m \quad \text{almost surely.}$$

The formal task also specifies measurability, fourth-power integrability, independence, and public declarations. The agent must turn those requirements into definitions and proofs that Lean can check. It can use the supplied general mathematical library, search for lemmas, test their interfaces, write auxiliary lemmas, and revise its implementation. The deliverable is a code version and its dependencies. [R02, R14; T01]

In this run, the author searched for strong-law, independence, and integral-product interfaces, read library implementations, and wrote small files to check exact types. It then implemented a fourth-moment expansion for finite sums, used independence to cancel terms, derived a tail-probability bound, and applied Borel-Cantelli. The first seven of eight initial build checks failed; the eighth passed.

However, the candidate that passed the eighth build still required the caller to supply a bound on the centered fourth moment, involving $X_i - m$. The textbook supplied a bound on the raw fourth moment, involving X_i . The code proved a useful intermediate theorem but still left part of the assigned work to its caller.

A general reviewer identified the gap. In later versions, the author proved the pointwise inequality and its integral consequence:

$$(x - m)^4 \leq 8(x^4 + m^4), \quad \mathbb{E}(X_i - m)^4 \leq 8(c + m^4).$$

Version eleven moved this conversion inside the proof. Its public theorem accepts the raw-moment assumption directly and uses the earlier centered-moment proof as an auxiliary theorem. Internal re-review of version eleven passed, and the final file matches it byte for byte. There was no common evaluation at the original cutoff; the subsequent existing dual-review pass is recorded in main [Section 3.1](#) and [Appendix K.9](#). [R02, R14, R18]

This example explains why the study needs code, build output, and review comments. A successful build confirms that Lean accepts the written declaration and proof. Comparison with the textbook can reveal that the declaration asks for an extra premise the proof should derive. Comparing code versions establishes whether a later revision actually closes that gap.

The strong-law revision above can be traced through actual work. The author did more than emit a proof draft: it searched library interfaces, read implementations, switched to `python3` when `apply_patch` and `python` were unavailable, and used Lean feedback to repair function representations, integrability combinations, and types. The seven failed builds still returned a zero outer shell status, while their output explicitly reported failure. Shell status alone would therefore misclassify the proof checks. [R02, R03; T01]

The final raw-moment bridge is also a code change. It establishes that a public-premise gap was identified and closed within this feedback and control chain. The first prompt explicitly prohibited help and required the author to return after preparing a review request. Neither the absence of help nor that return establishes an autonomous judgment that further work was unnecessary. The supplied textbook route also constrained method selection; there is no evidence that the author compared all strong-law proofs before choosing it.

G.14.2 T02: L3 removes extra premises that the proof should derive

Version six of A's `prob_7_6` asked the caller to provide everywhere nonnegativity and measurability of the limit function. Version ten removed both public requirements. It derived almost-everywhere nonnegativity from the given convergence and nonnegativity conditions, then obtained the required almost-everywhere strong measurability from integrability. At points where convergence holds, a convergent sequence of nonnegative values has a nonnegative limit; the proof should carry that step. [R02, R03; T02]

The record contains the specific criticism, commands reading the feedback, changes to the declaration and proof body, and a re-review bound to version ten. This is stronger than a review simply preceding a pass. However, the chain also contains host resumption, binding repair, diagnosis, a source decision, and a mathematical check. It does not assign the entire repair to one reviewer.

Old B's final code for the same problem still publicly assumes almost-everywhere nonnegativity of the limit; the interface review then treated that as a gap. Moving the premise into the proof was an actual interface change. Under this edition's common outcome criterion in Q.2, a redundant nonnegativity premise directly derivable on the same input does not alone block completion. The old A/B failures remain due to the missing tail scope. Historical review judgments and the present outcome criterion are retained separately.

G.15 L2: one standard for general covariance expansion

Current L2 acceptance uses G, which permits general mathematical infrastructure. A, early B, B r4 and C all pass. Candidates must still derive zero cross terms from the supplied uncorrelatedness assumptions and complete the mean, variance, Chebyshev and limit steps for the weak law. Directly assuming the target remains forbidden. [Y001-Y003]

A calls `variance_sum` and C calls `variance_fun_sum`; the latter is obtained directly from the former in the library. Both use an expansion valid for generally correlated families and then handle off-diagonal terms:

$$\text{Var}\left(\sum_i X_i\right) = \sum_i \sum_j \text{Cov}(X_i, X_j), \quad \text{Cov}(X_i, X_j) = 0 \quad (i \neq j).$$

Original evaluation permitted A’s use but rejected C’s corresponding behavior. Matching source and contract did not ensure matching interpretation. The correction removes this inconsistency: C changes from fail to pass, and A remains accepted. [Q017-Q027, Q098-Q107, Y002]

Candidate	Current G standard	Supplementary S: self-expansion
A	Pass	Incomplete coverage
Early B	Pass	Incomplete coverage
B r4	Pass	Pass
C	Pass	Incomplete coverage

S additionally requires the candidate or permitted support to expand the centered square and apply expectation linearity itself. B r4 performs that work. A and C do not deliver the complete self-expansion, although C separately proves that an individual centered-product integral vanishes. The retained obligation-level assessment therefore distinguishes their gaps rather than inferring identical proof coverage from a common overall label. These are route requirements under S, not mathematical counterexamples to their G-standard proofs. [Z020, Z021, X018; 30]

The ten earlier L2 evaluations and valid adjudications supplied sufficient evidence. Neither the 05:17 supplement nor the 09:22 repair added L2 model calls. G is now the explicit primary standard; S is supplementary. The choice was made after observing results and does not establish configuration superiority under an originally fixed criterion. [Y001, Y007]

B’s earlier support export failure remains a process observation. Its final r4 candidate connects the supplied uncorrelatedness definition and performs its own expansion, so the final interface is no longer unresolved. Support adoption and final mathematical correctness are separate questions. [Q081-Q086; U17, U18]

G.16 L7: the earlier stop and contract issues in the repair

Dated scope. This section retains the evidence available at the 12 September cutoff, including the earlier separate A repair. The successful 13 September resumption is a different artifact and stage; see O.2.

B/C currently complete reasonable repaired tasks, while the original and separate repaired A candidates retain delivery gaps. This section explains the A stop and insufficient assumptions before distinguishing their outcomes under corrected acceptance. Runs, scopes and resources remain separate. [Y001-Y003]

The L7 mathematical gate identified an interface revision and unresolved obligations. The outer host subsequently prohibited all edits, assistance, and new reviews and sealed the run. These were separate decisions at different levels. The original root budget was 7,200 seconds, with approximately 3,668 seconds remaining at sealing, so budget exhaustion does not explain this stop. Later H1 completion does not prove that L7 would necessarily have finished if continued. [U10]

The three endpoint opinions accepted saved theorem 10.10 but handled unfinished theorem 10.11 and the partial scope differently. The adjudicator’s direct prompt did not inherit the ordinary reviewers’ same partial-scope restriction. This partial-material evaluation ultimately records failure. All three opinions accept saved theorem 10.10, while the full two-target contract still lacks theorem 10.11. The original fail, the scope discrepancy, and the missing target must all remain visible. The record is not changed to pass, and scope discrepancy is not asserted to be the sole cause of failure. [U11]

The specific mathematical dispute preceding the stop follows. The singleton counterexample refutes a particular continuity interface; it does not complete every obligation of the second target.

A general review challenged a continuous-mapping claim in L7. It recommended replacing ambient continuity at points of a set with continuity relative to that set. The proposed interface does not suffice. On a one-point probability space, take:

$$S = \{0\}, \quad V = 0, \quad V_n = \frac{1}{n+1}, \quad f(x) = \begin{cases} 0, & x = 0, \\ 1, & x \neq 0. \end{cases}$$

The function is continuous relative to the singleton S . The variables V_n converge to V , but $f(V_n) = 1$ for every n , so they do not converge to $f(V) = 0$. The approximating variables take values outside the set; restricting the function’s behavior on the set is insufficient. This directly checkable counterexample resolves the proposed change without ranking the reviewers’ confidence. [R02, R03; T07]

The counterexample refutes that modification, not every outstanding criticism of the original candidate. Other textbook-route obligations remain. In the final author turn, the host explicitly prohibited code changes and help, and required a recorded stop. The result retains an internal pass for the first target and an incomplete second target. It cannot be described as the author autonomously abandoning the entire problem.

G.16.1 The repair candidate: an omitted duty and insufficient assumptions

The later A-L7 repair delivers both target files and passes internal review and building, but both original common reviewers fail it; it remains incomplete under current acceptance. Both accept the coordinate equivalences and continuous-mapping convergence control. They note that the returned vector-convergence predicates carry no measurability field and that the candidate does not separately prove each composite output to be a random vector. This is an actual omission relative to the written contract, which technical success cannot erase. [Q069-Q076]

The contract simultaneously assumes continuity only on a set S containing the limit almost surely and demands measurability of all composites. Even ambient continuity at each point of S is insufficient. Take the Lebesgue probability space $\Omega = [1, 2]$ and a non-Lebesgue-measurable subset E , and define

$$f(x) = \mathbf{1}_E(x), \quad S = \{0\}, \quad V_1(\omega) = \omega, \quad V_n(\omega) = 0 \ (n \geq 2), \quad V(\omega) = 0.$$

The function f vanishes near 0 and is continuous there. Every input variable is measurable and the sequence converges pointwise to V , whose value lies in S . Yet $f(V_1) = \mathbf{1}_E$ is not measurable. This one initial term does not affect convergence but already violates the demanded measurability of every output. This is a mathematical counterexample to sufficient task conditions, not a new Lean-verified experiment.

This is distinct from the preceding relative-continuity counterexample: the earlier example refutes an inappropriate continuity substitution; this one shows that ambient continuity still leaves measurability outside S uncontrolled. The executor and independent method audit read the frozen source and bounded chapter context without finding a traceable convention that all deterministic functions are measurable. This is not an exhaustive search of the published book. The corrected task adopted at 09:22 explicitly adds Borel measurability of f and retains ambient continuity at each point of S . It evaluates existing reasonable repairs without new solving. [Z030, Z031]

G.16.2 Current acceptance: B/C complete reasonable repaired tasks

Current L7 acceptance permits reasonable, disclosed and fully proved repairs. The mapping is Borel measurable and ambient-continuous at each point of S , with the limit in S on a full-probability event or almost everywhere. Candidates must actually deliver measurability of every composite and both convergence results; public support theorems can carry these conclusions. [Y001]

Candidate	Current outcome	Delivered scope
A original partial	Incomplete	Entire continuous-mapping branch absent
A separate repair	Incomplete	Numerical convergence accepted; composite measurability not delivered
B r4	Pass on repaired task	Public support delivers measurability and both convergence results; also assumes S measurable
C r3	Pass on repaired task	Complete repair using measurable preimage or almost-everywhere membership; separate numerical core

B's measurability assumption on S is a stronger sufficient repair than C's formulations. The difference remains explicit. Current acceptance asks for a reasonable sufficient repair, not a unique or weakest assumption, so it no longer maintains B's failure on that basis. Permission to add conditions does not automatically supply A's missing proofs; both A candidates remain incomplete. [Y002, Y003]

Original literal-contract rejections and formatting-related uncertainty remain historical records. Current outcomes follow corrected acceptance and sufficient existing mathematical evidence, without new solving. They do not imply that all three configurations originally worked from the same corrected inputs.

G.16.3 How the targeted reviews support corrected acceptance

The two independent reviews completed at 05:17 are valid and agree on all ten obligations, requiring no adjudication. They accept C’s coordinate equivalences, norm and convergence bridges, almost-sure numerical core without mapping measurability, and complete corrected in-probability proof. The original delivery explicitly discloses function-level Borel measurability rather than directly assuming composite convergence or measurability. [X053, X058, X010]

Current evidence	Both independent reviews
Mathematical correctness under exact public assumptions	Pass
Reasonableness of the condition repair	Pass
Disclosure of changes and scope	Pass

Both reviews also noted noncompliance with the original literal conditions. The acceptance repair completed at 09:22 explicitly accepts the reasonable corrected task, so C-L7 now passes. Formatting repair alone would not resolve the old scope disagreement; current acceptance relies on both evaluator repair and corrected evaluation design. [Y001-Y003]

Printed textbook pages 176–177 and the measurable-composition theorem on page 58 were supplied retrospectively, not backdated into solver input. The local statement does not explicitly assume global mapping measurability, while measurable composition requires measurable maps between compatible spaces. This is neither an exhaustive convention search nor author confirmation. [X037]

Attribution remains specific: an external reviewer of A’s repair also identified insufficient conditions, so C is not the sole discoverer. A’s numerical in-probability control without mapping measurability is substantive work; B also delivers a reasonable repair. C explicitly discloses its repair and independently retains the numerical core under weaker assumptions. Identification, repair delivery and external recognition are different events. [X003, X012]

G.17 L3: retaining the same eventual-tail requirement for each candidate

Later audit. The 13 September check establishes that A’s mathematical reviewer actually received the tail wording and B used a fresh internal review session. It updates responsibility attribution without changing the scope labels below; see O.3.

Current L3 acceptance uses T, the eventual-integrability formulation expressly permitted by the original task. C delivers that public interface and remains accepted. A and B r4 instead require every sequence term to be integrable and provide no general tail wrapper, so both are incomplete. A changes from its original pass; correction is not confined to changes that increase acceptance. [Y001-Y003]

Candidate	Current T standard	Delivered scope
A	Incomplete	Missing eventual-tail wrapper and recovery of the original-sequence limit
Early B	Incomplete	The corresponding tail wrapper is absent
B r4	Incomplete	Missing eventual-tail wrapper and recovery of the original-sequence limit
C	Pass	Delivers the task-permitted eventual-tail interface

All-term and eventual integrability differ. On $\Omega = (0, 1]$, let $f_1(x) = 1/x$ and let all later terms and the limit be zero. The nonnegative extended integrals converge to a finite limit although the first term is not integrable. A tail reduction can discard the finite prefix and recover one-norm convergence of the original sequence, but that wrapper is absent from A/B’s actual public delivery. [X003, X012, Y002]

This gap does not refute their negative-part control, one-norm argument under the stated assumptions, or density, total-variation and Gaussian branches. The missing work is an actual interface and derivation, not a rejection of

all existing mathematics. Conversely, because the original task permits T, C is not required to add the stronger E wrapper deriving eventual integrability from extended integrals. [Y001]

The four candidates previously received twelve consistency evaluations. The raw A and B r4 adjudications already identified the scope gap but were blocked by a quotation or newline-escape mismatch. This repair resolves determinable correspondences against the actual context and reuses the original reasoning and votes; these main outcomes are no longer held unresolved by formatting. The old F/E/T table is retained in [Appendix K.8](#). [Z022, Z023, Y005]

The case exposes two evaluation problems: the original assessment treated the same extra premise inconsistently, and reassessment briefly imposed a wrapper beyond the task’s permission. The correction retains A/B’s real delivery gaps while accepting C’s completed, permitted scope.

G.18 The six-service audit: validity, exposure and adoption are separate results

The saved targeted audit covers exactly six L2 services: two A general reviews, one B support-author call, one B general review and two C target reviews. The audit was recorded on 11 September at about 21:54 Beijing time; this revision presents its existing annotations and supporting records, not six new assessments. The original task and service requests remain separate from the later G/S interpretation in G.15. G permits general covariance expansion; S requires the additional candidate-built expansion. [16,21,37]

The table uses the first eight characters of each original dispatch identifier, unique within these six records. Full episode, root, dispatch and session identifiers, original feedback, candidate paths and source locations are retained in `verification/L2_SERVICE_AUDIT.csv` and the evidence guide. **Unverified exposure is not verified non-reading.**

Service and short identifier	Saved validity assessment under the later readings	Full-text exposure	Adoption or mathematical repair established
A variance review, 50194432	General covariance expansion and cross-term cancellation are supported under G; S expansion is not delivered.	Unverified	Passing opinion; no requested proof repair. Intermediate caller edits were not fully reconstructed.
A weak-law review, bb680eb5	Local mean/variance, Chebyshev and limit reasoning is supported; the variance dependency retains its G/S restriction.	Unverified	Passing opinion; no proof repair claimed. Final-code correspondence does not establish every intermediate action.
B r4 helper, 6361bff3	The saved audit does not independently validate the isolated patch. Final B’s G/S pass cannot be transferred to the helper product.	Unverified after recovery	Export was recovered, but actual reading, patch adoption and resulting obligation repair remain unknown.
B r4 joint review, f5e4ae15	Both reviewed targets are supported under G and S.	Verified	Author acknowledges validation before submission. Both reviewed target files equal the final files byte for byte; this is validation, not a proof repair.
C r3 variance review, 8ce74337	General expansion and cross-term cancellation are supported under G; full S expansion is not delivered.	Verified	Opinion text reached the coordinator. No proof repair claimed; all intervening edits were not reconstructed.
C r3 weak-law review, abbc1cbd	Local weak-law reasoning is supported; its variance dependency retains the G/S restriction.	Verified	Opinion text reached the coordinator. No proof repair claimed; a complete caller-edit chain was not reconstructed.

What changed in the analysis. B’s helper did produce model output, although a permissions failure blocked export. Engineering recovery later copied the product without another model call or container. Recovery establishes delivery, not adoption. The B joint review is a different service: its returned text, the author’s acknowledgment and the byte-

identical reviewed/final targets establish checking of an existing result before submission. They do not establish that the recovered helper patch created that result. [Z015, Z016]

What the six results support. Three services have verified full-text exposure and three remain unverified within this subset; these are not new estimates for all 127 services. Five are passing opinions without a requested mathematical repair, so unchanged code must not be scored as failed repair. The helper’s unresolved validity and adoption remain unknown. For A/C, the stronger S route requirement also does not make the supported G proof or every local weak-law step false. The audit therefore establishes a specific validation chain and specific observation limits, not an overall valid-feedback adoption or repair rate. N.4 retains the full inventory boundary.

H Observation windows and historical resource accounts

The unified main-panel spans use first dispatch to last role return in M.1. The historical accounts below preserve their original ending events and observation windows. They provide evidence about particular trajectories but are not substituted for missing unified measurements or added to totals that already contain the same calls.

H.1 Source windows identify when material became available to the analysis

The first inventory was read on 11 September 2026 between 09:18:50.564 and 09:18:57.960 Beijing time. The later primary window ran from 11:40:14 to 11:40:24, while C r3 was still in progress. A supplement captured its completed solver sequence at 11:50:33, followed by a bounded endpoint supplement at 11:56:54. The U increment used the per-file window 15:49:07-15:49:16 and reused case checks completed at 15:04. These are non-atomic collection windows with per-file identities, not claims that every source represented one simultaneous system state. [R02, R04; N01-N13; U12]

Later evidence does not retroactively change what an author could read at submission. Nor does a completed helper receipt establish that the parent task was complete at that collection time. The Q/X/Y/Z windows and final acceptance cutoff are retained separately in K.

H.2 L1 versions: time and token boundaries

Run and ending event	Root seconds	Input tokens, including cache	Output tokens, including reasoning
Initial A: first dispatch to controller ending	5,151.280	At least 6,156,275	At least 65,876
B r3: first dispatch to author return	940.836	2,664,038	30,165
Initial C: first dispatch to explicit submission	941.191	2,887,828	35,702
B r4: first dispatch to author return	714.466	2,355,921	22,349
C r3 supplement: first dispatch to explicit submission	708.714	2,570,670	27,730

Tokens sum actual solver-role receipts and exclude the later common evaluation. A’s second author turn lacks usage fields and the last reviewer lacks a completion receipt, so its usage is a lower bound and cannot define a complete cost ratio. B r4 includes 2,222,592 cached input tokens; C r3 includes 2,354,304. Those subsets are not added again. [N11]

Root duration already includes waits for assistance. Author return, role-process end, explicit submission, container stop and sealing remain different events. C r3 starts its root clock at first actual dispatch rather than the earlier outer launch. The much-discussed ratio near 7.2 uses approximately 5151, 714 and 709 seconds with different ending events. It is not a uniformly measured efficiency ratio and is not substituted for the current main-table comparison.

H.3 M1, M3 and H1: detailed role receipts

This earlier account ends at the author’s last return or the coordinator’s explicit submission, as recorded for each run. Assistance waiting is already included. [U13, U16]

Run	Completed calls / distinct sessions	Root seconds	Input, including cache	Cached input	Output
B r4, M1	3 / 3	1,058.725	4,426,338	4,187,264	35,358
C r3, M1	5 / 3	1,594.905	8,445,442	8,098,048	63,518
A, M3	3 / 3	1,267.653	7,508,872	7,132,032	47,040
A, H1 original and continuation	16 / 12	10,929.620	95,568,298	93,436,160	271,009

The output counts respectively include 7,761, 15,274, 15,802 and 77,033 reasoning tokens; those are not additional outputs. All sixteen registered H1 solving-side calls have completion receipts and share the original root clock across continuation. At that window, the roughly three-hour-two-minute span leaves about six hours fifty-eight minutes of the original ten-hour allowance. The subsequent common evaluation remains separately accounted. Sixteen completed calls are not sixteen independent mathematical experiments.

These historical seconds are not identical to every M.1 unified span. In particular, C-M1 retains 1,594.905 seconds to the historical root endpoint here, while M.1 reports 26.79 minutes to its last-role-return endpoint. The endpoint definitions differ; this edit does not reconstruct their per-event difference from raw timestamps. B/C-M1’s recorded time and input usage are both higher for C under the available respective accounts, but the difference cannot be assigned exclusively to coordination. The historical table retains its original entries; A-M1’s time reconstructed with the common start and endpoint definition appears in P.

H.4 The analysis definitions were refined from observed failures

The earlier case investigations refined three parts of the trajectory framework. Help use was divided into code revision, route selection, validation decisions and submission decisions. Failure locations were separated into request entry, model execution, artifact export, return encoding, parent reading and code application. Records also distinguished the actor permitted to choose the next action, the information that actor could see, and later host interventions. [N01-N13; R01-R03]

These refinements followed the seven-case investigation and same-task analysis; they were not preregistered. L5’s changed adoption classification and the L1 difference between interface repair and unchanged-code submission are examples of the evidence changing the analytical definitions. The older inventory remains in the historical archive; the resource boundaries and analytical refinements used in this report are retained here.

I R and N source navigation

The tables retain source identities needed by citations. R resolves through the R manifest in E.2; N through the N manifest’s file identifiers and snapshots. Earlier editorial-check counts and dispatch exchanges are not repeated. R03 locates M2 version eight, its review and the version-eleven change; R14 is the final version-eleven code and R18 its closeout identity.

ID	Content and purpose
R01	Framework v1: questions, units, evidence types, autonomy, and missingness.
R02	Seven-case pilot analysis: claims C01-C14, costs, and alternative explanations.

Continued on next page

ID	Content and purpose
R03	70 evidence references: original paths, lines, call identifiers, fixed copies, and excerpts.
R04-R05	First-inventory mechanical audit and delivery checks: coverage counts, hashes, and references.
R06-R07	Original four-trial report and later corrections to eligibility and timing interpretation.
R08	C's simplified coordination design: roles, fixed services, information, and comparison limits.
R09	Bounded execution closeout: results, faults, and candidate engineering repairs.
R10	Revised-B solver accounting: four real roles, time boundaries, public events, and tokens.
R11, R16	Revised-B final-evaluation accounting and original decision: dual review, identity, development eligibility, and cost.
R12, R17	C's revised final-evaluation accounting and original decision: later dual review of the same candidate and cost.
R13	The executor's revised-B raw-output audit, not independently repeated stream by stream for this report.
R14, R18	M2 version-eleven code and stable closeout receipt: public interface and time boundaries.
R15	Revised-B original solver result: submission and snapshot state.
R19	Timing definitions in normalized-results compiler v2 and corrections to earlier inferences.
R20-R21	Structured seven-case coding and condition, role, and session-version registry.
R22	Executor closeout state projection, not a live process check by this report.

ID	Contents
N01	A prompts, searches, checks, requests, and public messages across both author turns
N02	A review text, recovery receipt, controller status, and identity-field conflict
N03	A's original sealed final Lean proof
N04-N05	B r3 final proof, author actions, and actual outputs of three helping roles
N06-N07	Initial C coordination, writer implementation, and two real checking roles
N08	B r4 Chinese review, corresponding revision, checks, and final proof
N09	Separate common decisions for B r3, initial C, and B r4
N10	C r3 closeout: returned opinion, explicit acknowledgment, submission, and target
N11	Role receipts, usage, missing fields, and solver scope
N12	C r3's ongoing fragment in the primary window
N13	New A and C r3 endpoints, target-byte bindings, and M2's status decision

J U source navigation

U codes map to the 15:49 increment's original identifiers and then to original paths, frozen bytes, lines and checksums through claim-level evidence in [E.2](#). Original identifiers in this table belong only to that increment; its N01 is not this report's N01.

This edition	Increment identifier	Checked material
U01	K02	M1 task bindings, three endpoints, and differing conditions
U02	K03	B M1 delegation, patch delivery, adoption, and validation
U03	K04	B M1 outer exit code and actual build recovery
U04	K05	C M1 reused review session, documentation change, unchanged code
U05	K06	M3 inverse-distribution construction and actual adoption
U06	K07	M3 confidence-format inconclusive outcome and adjudication
U07	K08	H1 narrowed help scope and adopted integral support
U08	K09	H1 inversion code, export failure, and host recovery
U09	K10	Actual uniqueness invocation and review-binding recovery
U10	K12	L7 mathematical gate, host stopping, and remaining budget
U11	K13	L7 partial scope, disagreement, and adjudication
U12	N01	Latest inventory, increment, and 24-root coverage
U13	N02	H1 five internal review registrations and final joint build
U14	N03	Standard-Cauchy application and uniqueness invocation
U15	N04	Integer-valued criterion without uniqueness invocation
U16	N05	H1 complete registered role account and shared root clock
U17	N06	Latest B L2 helper export failure
U18	N07	B L2 helper correspondence with the given definition
U19	N08	C L2 bridges, reuse, and ongoing state
U20	N09	17 endpoint decisions and comparison eligibility

K Task and acceptance-version index

K.1 Task inputs

Frozen H1, L2-L7, M2 and M3 instructions are I17957, I17975, I17996, I18005, I18015, I18042, I18063, I18089 and I18099 respectively. Common rules are I05757 and M2 source I18088; L1/M1 retain N01-N06, R06 and U01. [E.2](#) and the archive-reference file locate the settings manifest.

K.2 Reading the version index

This index separates execution versions, capture windows and acceptance criteria. A later capture can preserve an earlier run, and a later criterion can change the interpretation of an unchanged candidate. [K.8](#) retains original and intermediate labels; [K.9](#) identifies the corrected acceptance used in the main panel. [N.1](#) explains which executions supply that panel.

K.3 Production contracts

P01/P02 preserve the status contract and review criteria; V01–V06 are the inherited local document snapshots. [A.5](#) separately identifies the public documentation used to expand the system overview. Experimental rules and production mapping remain as stated in [N.1](#) and [N.2](#).

K.4 Q capture window

Q comes from 142 selected files captured around 20:00 on 11 September, including 23 decisions and related candidates, opinions and contracts. This is a historical non-atomic capture, not new experimentation. Revision 7 `source_manifest.json` retains full identities.

K.5 Z final first-round package

The Z package closes at 03:07:37 on 12 September and records the previously executed sixteen additional solver runs and 31 consistency evaluations. Key files are Z002 unified status, Z004 attempts, Z008 versions, Z011 costs, Z015-Z016 semantics/adoption and Z020-Z026 criterion results. `FINAL_PACKAGE_INDEX.json` supplies paths and delivery hashes.

K.6 All 41 solver roots

Version stratum	Actual roots	Common endpoint roots	Original selected label counts
A: separate vector-convergence repair	1	1	fail: 1
A: earlier development	9	9	pass: 8, fail: 1
A: later runtime	2	2	pass: 2
B: capability revision	1	1	pass: 1
B: original development	5	0	no_common_label: 5
B: later transport-capable implementation	11	11	pass: 8, fail: 3
C: initial coordinator	1	1	no_common_label: 1
C: later coordinator runtime	11	11	pass: 9, fail: 1, unresolved: 1

These are version denominators, not a conversion of absent common labels into failures. Earlier provisional evaluations occupy a separate cost category. Root existence, candidate completeness, endpoint execution, label validity, and comparative eligibility are different fields. [N.1](#) explains the main-panel selection, and the accompanying evidence index retains the per-run correspondence. [Z002, Z004, Z008; 31]

K.7 X targeted evaluation

The X package closes at 05:17:25: X018 reuses L2; X053/X058 contain two L7-C opinions, X010 validity, X003/X012 scope, X020/X027 additional cost and X037 retrospective textbook context. `TARGETED_R2_PACKAGE_INDEX.json` locates all 59 files; these are not new assessments in this edit.

K.8 Original outcomes and scope-sensitive intermediate tables

These tables reproduce the pre-correction labels retained in revision 7. They describe historical evaluator states, not current outcomes. The main report and [K.9](#) use the corrected acceptance. In particular, a format-unresolved cell is not evidence that the candidate remains unresolved now. [Y001-Y003]

K.8.1 Original selected panel

Group	A selected candidate	B r4	C r3
L1	Pass	Pass	Pass
L2	Pass	Pass	Fail
L3	Pass	Fail	Pass
L4	Pass	Pass	Pass
L5	Pass	Pass	Pass
L6	Pass	Pass	Pass
L7	Fail	Fail	Unresolved

Continued on next page

Group	A selected candidate	B r4	C r3
M1	Pass	Pass	Pass
M2	Pass	Pass	Pass
M3	Pass	Pass	Pass
H1	Pass	Fail	Pass
Original totals	10 pass / 1 fail	8 pass / 3 fail	9 pass / 1 fail / 1 unresolved

L2 is sensitive to the general-expansion versus self-expansion rule; L3 to the required tail scope; H1 to the Cauchy parameter family; and L7 to task conditions and valid result processing. The other groups retain their recorded development-version coverage. No row implies matched-condition repetition.

K.8.2 H1 before the processing and acceptance correction

Candidate	Original endpoint	Standard-Cauchy assessment	General location-scale assessment
A original root with continuation	Pass	Pass	Fail
B r4	Fail	Unresolved: invalid quotation	Unresolved: invalid quotation
C r3	Pass	Pass	Pass

Current [G.13.1](#) accepts the textbook core for all three and separately records A's standard, B's centered-scale and C's location-scale delivery. The old quotation failures above are preserved as processing history rather than current mathematical failures.

K.8.3 L3 before the processing and acceptance correction

Candidate	Original label	F: termwise finite integrals	E: extended-integral source domain	T: task-permitted eventual tail
A	Pass	Unresolved format	Unresolved format	Unresolved format
Early B	Fail	Pass	Fail	Fail
B r4	Fail	Unresolved format	Unresolved format	Unresolved format
C	Pass	Pass	Fail	Pass

F assumes termwise finiteness; T is the task-permitted eventual-integrability interface; E additionally derives the tail reduction from the extended-integral source assumptions. Current [G.17](#) uses T, retains A/B's missing wrapper and does not add E as a new requirement for C. The historical F/E/T table records why the former labels and unresolved states cannot substitute for that current decision. The table retains the original intermediate labels; it is not the current acceptance table.

K.9 Y: corrected acceptance for the 12 September comparison

The Y package completes at 09:22:31: Y001 current criteria, Y002 main panel, Y003 interpretation, Y005 processing repairs, Y006 eighteen regressions and Y007 completion. It reuses opinions with zero new model evaluations, solver runs or builds; this edit retains that original comparison cutoff. The additional A-L7 continuation in [O.2](#) is a separate 13 September event.

L Historical mathematics supporting current claims

The following arguments explain proof responsibility, workflow evolution, coupon collection, target changes and successive inversion proofs. Historical statistics and code-check methods are in C and D. Textbook task names identify the mathematical histories, distinct from the experimental group labels introduced in main [Section 2.2](#).

L.1 How author-selected editing developed alongside explicit delivery control

The historical change concerns who chooses the next action. The early system assigns planning, retrieval and generation to prescribed stages. The later code agent selects searches and edits from current files and feedback, while entry programs still govern eligible submissions and acceptance. This is a supported account of changing responsibilities, not a measured performance comparison. [E01, E14]

L.1.1 What the early pipeline already contained

The inspected 30 March 2026 baseline, commit 976756b, contains orchestration, proof planning, library retrieval, candidate reranking and local checking. `TextbookOrchestrator` coordinates stages and `ProofArchitect` develops a proof plan. Retrieval uses sentence embeddings and a FAISS index, followed by language-model reranking. `ReflectionManager` injects prior experience into the initial task prompt, and repeated errors can broaden the search. Experience use and feedback are therefore present in the baseline, not later inventions inferred from their names.

An older API-pipeline script also has a rescue branch that regenerates the whole file. That branch must not be mistaken for the later package-based author’s normal response to semantic feedback. Component presence explains possible execution routes but does not measure their frequency, elapsed time or proof quality. [E01, E14]

L.1.2 Coexisting entry points and the transition to task-package editing

On 8 April, an old-orchestrator report decomposes `ex_1_2_3`, while a `def_5_1` task package and two validations are also recorded. Old decomposition and package-based work coexist. A validation at that point must be interpreted under its contemporary interface, not automatically treated as today’s full semantic review.

On 14 May, commit 8a2e563 removes the old direct-generation and orchestrator entry points. Ordinary work follows package preparation, draft editing, building and semantic review. Errors and review reasons guide local revisions, and the agent has discretion over the action sequence. Entry points retain control over admissible candidates and review materials. Separate review artifacts in May do not prove that every author/reviewer pair already used isolated instances; later independence requirements are not projected backward. The consolidation commit bundles changes and cannot isolate an editing-agent effect. [E01, E14]

L.1.3 Why source correspondence entered the review object

The 10 April design request explicitly distinguishes building from judging whether Lean faithfully expresses source TeX. It identifies interactive checking, temporary/final builds and keyword heuristics as insufficient for that semantic question and requests a review layer using review documentation and a local language model.

The requested package includes source statement and plan, candidate code, metadata, imports, dependencies and review rules. Structured outputs, caching, failure handling and registration conditions also enter the design. Later `def_5_1` artifacts contain candidate and input hashes, rule versions, source-to-statement mappings and verdicts. These records establish that explicit review inputs were used, not their accuracy against independent mathematical truth. [Appendix A](#) distinguishes requests, implementation and actual execution rather than assigning all three the same date. [E01, E14]

L.2 Divide the proof, but keep responsibility for the whole task

A long proof creates two practical questions: which smaller results should be proved next, and who checks that they add up to the textbook claim? `ToyApollo`’s obligations, support files, and route checks address different parts of this problem. Their names are easiest to understand through the work they organize.

L.2.1 An auxiliary name can hide two different jobs

The early word **bridge** covered both a proved conversion between representations and mathematics still lacking a proof. Suppose a textbook distribution and a library distribution use different definitions. Proving their equivalence connects the representations. Merely naming a missing theorem, assuming it as a premise, or declaring it as an axiom leaves mathematical work outstanding.

The `prob_1_4` history illustrates the distinction. An early proof depends on public axioms in `gamma_beta_bridge`. A later version removes that bridge but defines the normalized Gamma density as the desired Dirichlet formula and obtains the equality by simplification. The historical reviewer still asks for a derivation from the probability model. Subsequent versions supply the product distribution, normalization map, simplex support, projected density, and change of variables. Those arguments constitute the progress. [E03]

On May 17, commit `7cf8717` separates representation conversion (`translation`) from outstanding proof work (`proof_debt_support`). These are bookkeeping categories, not certificates for individual declarations. A reader must still inspect what a declaration assumes and proves.

L.2.2 What a proof-obligation subtask contributes

A **proof obligation**, labeled OBL in the project, is a smaller mathematical duty separated from a parent task. It can have its own candidate, imports, and reviews, and some obligations are divided again. In `thm_7_8`, such work establishes integrability from continuity on a compact set, obtains a Riemann-Stieltjes integral using function extension and an existing theorem, and supplies squeezing and common-limit arguments for the parent. These are substantive results, even when the parent still has work to do. [E04]

The inversion example in main [Section 1](#) and [L.5](#) makes the completion rule concrete: proving two of the three ingredients in `h_spine` leaves the third to the caller. It also shows that an internal obligation can be narrowed when the textbook argument needs less than the original plan requested.

Earlier code allowed obligation updates and clearance of proof debt to propagate into the parent’s ledger without necessarily creating a fresh parent semantic review. A June 15 ledger event absorbs obligations for `thm_7_9`, `ex_14_4_2`, and `prob_14_8` into parent or support files and requires renewed builds, independent review, and application. The useful mathematics survives; the old subtask’s acceptance no longer automatically establishes the parent’s status. [Appendix B](#) retains the exact timestamp and the limited scope of the surviving truncation result. [E02, E04, E11]

L.2.3 Checking the route before assembling the parent

Math Gate is a route check that controls the next stage of work. When triggered by risks such as hiding proof work in public premises, a mismatch with the source, or blocking status, it asks for a natural-language proof skeleton and an independent role’s check of the claim, route, and intended Lean statement. A recorded go permits the next stage. A stop blocks ordinary parent-task writing under that plan, while named support results may still be permitted.

This is an action rule, not a final theorem verdict. Scheduling and build entry points enforce it; prompt text alone does not describe the mechanism. The trigger rules include older pilot identifiers as well as risk categories and status text, so the gate did not apply uniformly from the start of every task. Final delivery still needs the appropriate build, semantic review, and registration. [E02, E10, E16]

Consider `prob_14_8`, which derives tightness and convergence in distribution from moment-generating-function convergence near zero. Tightness controls probability mass escaping far away. The chosen proof extends the moment-generating functions into a complex strip, extracts an analytic limit, identifies it, and obtains characteristic-function convergence on the imaginary axis.

Early candidates use private axioms. Later work proves tightness, analytic bounds, compactness-based extraction, subsequence results, and diagonal lemmas. Repeated stop records recognize these achievements but identify a remaining assembly problem: limits constructed on compact pieces must agree as restrictions of one global function. Later proofs construct that function, prove the agreement, and reconnect it to the diagonal argument. Semantic review 13 assesses the resulting derivation from the original assumptions. Review 14 assesses a main entry point importing support: the complete single-file proof existed before this file split. [E07]

The records therefore show useful support accumulating during stops. They do not show what the same agent would have achieved without the gate.

L.2.4 A definition repair also changes its callers

On July 19, `def_8_5` keeps its total variation formula but adds the source’s probability-measure condition to its public signature. A caller must now supply evidence that its inputs are probability measures. The repair plan identifies the definition owner, direct consumers, duplicate names, and the broader dependency closure. This extends the task from changing one definition to adapting the code that uses it. [E08]

For example, `def_10_5` says both that the measures are probability measures and that their distances converge. Lean needs the probability evidence available at the point where the distance is formed. The repair passes the same evidence through a dependent witness and a local instance (`letI`). Replacing “both conditions hold” with “if the first holds, then the second holds” would change the definition: non-probability inputs could satisfy the implication vacuously.

Five substantive reviews involve only two main-file contents. Consumer changes, failed registration for missing fields, and expanded checks account for further events. Acceptance of the definition alone does not finish the caller migration. The retained account uses reports and saved caller material; endpoint hashes do not independently verify every consumer.

L.2.5 Support files and task identity

A **support file** contains auxiliary code used by a task or by several tasks. June maintenance instructions favor layers owned by one parent until a second actual consumer justifies considering shared support. The Gamma/Dirichlet work has two such consumers: `prob_1_4` and `ex_1_2_2`. Its June 21 split separates product-density, simplex-chart, and projected-density layers while preserving compatibility exports. Each task still retains its own statement and specific arguments.

By contrast, splitting a finished proof into modules need not create reusable mathematics or solve a new task. The later `thm_9_5_kernel` contains the inversion argument previously held in the main file. Modules labeled `family_member` remain implementation parts of one textbook task, whose parent is the review root. Other historical uses of **family** mean related tasks or book-number groups; [Appendix E](#) distinguishes them.

The practical reading rule is to ask three separate questions: what mathematics is proved, who maintains and uses it, and which textbook task owns the final claim? A file count answers none of these by itself. [Appendix B](#) preserves the additional ownership and extraction cases, including the difference between a newly completed proof and a newly organized one.

L.3 Why the central limit theorem did not finish the coupon example

A general theorem and an example using it can have different unfinished steps. This case makes that distinction visible. The textbook theorem concerns sums of independent random variables; the example uses geometric waiting times to model collecting coupon types. The saved history separates a core limit theorem, a connection to the intended random variables, and the example’s final normalization. [E18, E20]

L.3.1 Start with the example’s mathematical target

There are N equally likely coupon types, and the target is to collect $m_N = \lfloor (N+1)/2 \rfloor$ distinct types. After i types have been collected, the probability that a new draw gives a new type is $p_{N,i} = (N-i)/N$. The saved formal model directly takes independent geometric random variables $G_{N,i}$, valued in the positive integers, with these probabilities. Think of each variable as the duration of one waiting stage.

The total modeled waiting time, its exact mean, and its exact variance are

$$T_N = \sum_{i=0}^{m_N-1} G_{N,i}, \quad \mathbb{E}T_N = \sum_{i=0}^{m_N-1} \frac{1}{p_{N,i}}, \quad \text{Var}(T_N) = \sum_{i=0}^{m_N-1} \frac{1-p_{N,i}}{p_{N,i}^2}.$$

The first sum adds the waiting stages. The other two use the mean and variance of a geometric variable; independence lets the variances add. As N grows, the leading terms of the mean and variance are $N \log 2$ and $N(1 - \log 2)$.

There is an important scope condition at this starting point. The saved proofs use the independent-stage model just described. The evidence does not separately establish its equivalence with the original sequence of independent

uniform coupon draws and that sequence’s stopping time. The following account concerns the proved chain inside the saved model; it does not silently supply this additional equivalence.

L.3.2 Three pieces of the argument

The project calls the general theorem `thm_14_8` and the example `ex_14_4_3`. For a fixed N , a **row** means the finite collection of random variables being summed. Different rows can have different lengths and distributions. A **canonical product model** places these variables on a product probability space, where their coordinate distributions and independence are built into the construction.

Piece of the proof	Mathematical job	Why the next piece is still needed
Core central limit theorem	Prove the limit in the canonical product model under the relevant conditions	A concrete independent row may have been defined on another probability space
Distributional connection	Show that the concrete row and the product model have the same joint distribution	The resulting theorem uses the row’s exact mean and variance
Textbook normalization	Replace exact mean and variance by the textbook’s asymptotic expressions	The example asks for this final normalized quantity

The distributional connection transfers a result between constructions of the random variables. The normalization step changes the numerical centering and scale of the sum. They solve different problems, even though both are needed to use the general theorem in the example.

L.3.3 The June configuration had incompatible assumptions

Four June pass snapshots contain the same configuration C . Here a configuration is the collection of data and assumptions supplied to the theorem, not a candidate version label. The exported conclusions assumed both C and an external central limit theorem proof H . The historical reviews allowed this upstream proof, which lay beyond the book’s proof scope, to remain an input while classifying the example itself as `textbook_example_completed`. That classification records the reviewer’s acceptance; it does not prove that an admissible C exists. In fact, some necessary conditions imposed on C could not hold together.

The configuration used $N = n + 2$, where n indexes the rows. At the first row, $n = 0$, so $N = 2$ and only one coupon type is needed. The only geometric stage has success probability $p = 1$: it always lasts one draw. Its variance is therefore

$$\frac{1 - p}{p^2} = 0.$$

The code’s total-variance identity then forces the square of $\text{sn}(\theta)$ to be zero. The symbol $\text{sn}(\theta)$ is the first-row standardization scale in the configuration, and the parent configuration also requires $\text{sn}(\theta) > 0$. No value can satisfy both requirements. Consequently no configuration C supplies the claimed instance of this conditional theorem, although useful moment estimates remain in the file.

A saved Lean probe checks the exact numerical definitions, establishes zero first-row variance, and derives the contradiction. It exits with code zero and uses only `propext`, `Classical.choice`, and `Quot.sound`, the foundational axioms named in the saved check. This is a formal check of those necessary conditions in the recorded environment, not a reconstruction of the entire June project. Lean can verify a conditional proposition without asserting that its premises are realizable. Allowing the external proof H does not remove the contradiction inside C , which is obtained without using H .

L.3.4 An August failure nevertheless contained the core proof

The August main file proves the Lindeberg and Lyapunov branches of the central limit theorem in the canonical product model. These are the two sufficient-condition routes implemented by the theorem. Its public interface no longer asks the caller for an external central limit theorem proof or an arbitrary Gaussian target.

The failed review explicitly accepts those core arguments. Its remaining demand is the second piece in the table: connect concrete independent rows to the product model and update the callers. The next version, dated August 5, adds that connection using the equality of the joint distribution with the product of the marginal distributions, together with the behavior of distributions under measurable maps. The parent theorem then receives a pass.

Thus the later failed candidate contains mathematical work absent from the earlier example acceptance that permitted an external upstream proof. The later requirements ask for the core theorem internally and for its connection to concrete independent rows. The sequence is intelligible once we read what each candidate proves and what each review asks for; the labels alone do not rank their mathematical quality.

L.3.5 The final step changes the centering and scale

When the parent theorem passes, the example already builds the concrete geometric rows and applies the theorem using each row's exact mean and variance. It uses $N = n + 3$ to avoid the first-row degeneracy. What remains is the third piece: reach the textbook's normalization. A subsequent example review fails the byte-identical body for this gap.

The general theorem concerns the exactly standardized sum

$$Z_N = \frac{T_N - \mathbb{E}T_N}{\sqrt{\text{Var}(T_N)}}.$$

This variable is obtained by subtracting the sum's exact mean and dividing by its exact standard deviation. The saved theorem gives $Z_N \Rightarrow \mathcal{N}(0, 1)$: convergence in distribution to the standard normal law, not an exactly normal distribution at each finite N . The textbook seeks the same limiting law after subtracting $N \log 2$ and dividing by $\sqrt{N(1 - \log 2)}$. To connect the two, write

$$\frac{T_N - N \log 2}{\sqrt{N(1 - \log 2)}} = a_N Z_N + b_N, \quad a_N = \frac{\sqrt{\text{Var}(T_N)}}{\sqrt{N(1 - \log 2)}}, \quad b_N = \frac{\mathbb{E}T_N - N \log 2}{\sqrt{N(1 - \log 2)}}.$$

Here a_N measures the change in scale, and b_N is the remaining mean error measured on the textbook's scale. It is enough for this change to become negligible: $a_N \rightarrow 1$ and $b_N \rightarrow 0$, with the corresponding transfer of convergence in distribution. Because the coefficients change with N , this transfer must also be justified.

Later candidates bound the mean error by $O(1)$, meaning that its magnitude stays bounded as N grows. They also prove $\text{Var}(T_N)/N \rightarrow 1 - \log 2$ and supply the distributional argument for the changing affine map. The example then passes. This explains how parent acceptance and example failure could coexist: they concerned different obligations. The later success completes that remaining step in the saved model and leaves the separate June contradiction unchanged.

L.4 Check the target and the reviewed object before comparing labels

A later pass may follow a mathematical repair, an accepted change to the problem, or a change in the material presented to the reviewer. The following cases explain how to tell these possibilities apart. The first two change the mathematical target; the third changes the dependency context while leaving the main file unchanged. [E09, E18, E20]

L.4.1 Independent approximations do not specify the coupling of their limits

In prob_14_7, each pair X_n, Y_n is independent, and the two sequences converge separately in distribution to X and Y . These marginal convergences specify the distribution of each limit separately. They do not specify how the displayed limits are related to one another.

Take each pair to be independent standard normal variables. Their sum has normal distribution with variance 2. Now let the displayed marginal limits be $X = Y = Z$, where Z is standard normal. Both marginal convergences still hold, but $X + Y = 2Z$ has variance 4. The asserted convergence of the sums therefore cannot hold for every choice of joint distribution of the limits.

An earlier formalization assumes that the limiting sum's distribution is the convolution of the marginals. A later source decision explicitly requires the limits to be independent; the final theorem exposes this as `hLimitIndep`. This repairs a missing condition through an accepted change to the target convention. It is not simply a new technical parameter proving the identical original claim. For follow-up that requires a fixed mathematical target, this decision marks a boundary.

L.4.2 A converging proportion may approach too slowly for the claimed centering

The coupon task `prob_14_11` has a related but different issue. A condition such as $m/N \rightarrow 1/2$ specifies a limiting proportion. A central limit statement must also control the centering error on a scale of order $\sqrt{N}$. The first condition alone gives no such rate.

To see the gap without rounding complications, take even $k \geq 4$, let $N = k^4$, and collect $m = N/2 + k^3$ coupon types. Then $m/N = 1/2 + 1/k$ tends to $1/2$. The exact mean of the modeled geometric waiting sum is

$$M(N, m) = \sum_{i=0}^{m-1} \frac{1}{1 - i/N}.$$

Each term is the expected waiting time for the next new type. Integral bounds give $-1 \leq M(N, N/2) - N \log 2 \leq 0$. The additional k^3 stages each have expected duration at least 2. Therefore

$$\frac{M(N, m) - N \log 2}{\sqrt{N(1 - \log 2)}} \geq \frac{2k^3 - 1}{k^2 \sqrt{1 - \log 2}} \rightarrow +\infty.$$

The centering error, measured on the proposed standard-deviation scale, can grow without bound even though the proportion converges. This is precisely the kind of error that [Appendix L.3](#) proves tends to zero for the more specific coupon example.

The failed candidate already constructs geometric times, independent product rows, and a local Lyapunov argument. The passing candidate uses the row's exact mean and standard deviation, proves positivity, and changes the final target under a separate mathematical-review decision. That historical acceptance also permits the upstream external proof `H:thm_14_8_ProofBeyondBook`; it must not be described as a wholly self-contained final proof. The revision establishes the accepted conditional, exactly standardized result, not the original fixed-centering conversion. The fail-to-pass relation remains in broad label comparisons but stops fixed-target follow-up.

L.4.3 An unchanged main file can be different review evidence

For `thm_11_8`, the July 24 and August 5 main files have identical content. Yet their bound versions of the upstream theorem `thm_11_5` have different hashes. The later review input also includes a clue from the preceding review, and the build outputs differ.

Only the later upstream source was recovered in the bounded search. We can establish that the reviewed inputs differ, but cannot fully reconstruct the mathematical difference between the two upstream versions. This is not a comparison of two reviewers evaluating unchanged evidence. The case shows why a main-file hash alone is too narrow a description of the reviewed object; it does not quantify any accuracy gain from hash binding.

L.4.4 Separate progress, judgment, and registration

Other saved cases make the distinction in [Section 1](#) operational. `ex_3_1_4` adds mathematical arguments before placeholder obligation bindings are repaired. `thm_9_7` registers one candidate while its earlier review actually judges an overridden version; the statistical analysis excludes that mismatched relation. `prob_1_9` encounters a line-ending hash failure with unchanged mathematical source. The June 22 `thm_14_8` sequence corrects missing report fields and reassesses unchanged code; its final receipt records an exception without adopting a completion result.

These are not interchangeable kinds of improvement. [Appendix B](#) retains the full version, role, and event-order details, as well as `prob_14_10`, whose small final caller edit follows substantial upstream proof work. It also retains `def_3_6`, where missing source approval leaves target comparability unknown. A missing decision stays unknown rather than being classified as either a confirmed target change or a completed repair.

L.5 Inversion: successive proof increments and a narrower sufficient auxiliary target

The `thm_9_5` sequence contains a conditional main argument before all of its analytical ingredients are internally proved. The initial argument already uses substitution, pointwise cases, dominated convergence and rescaling. Its public `h_spine` packages integrability, the right to interchange integrals and the Dirichlet-integral limit. Historical semantic reviews 29-32 bind candidates 38-41; these are separate version counters, not thirty-two independent trials of the final theorem. [E05]

Successive stage	Proof increment	Outstanding public support
Initial conditional argument	Main inversion proof conditional on analytical ingredients	Integrability, interchange and key limit
Product-integrability revision	Bounds and finite measure establish the needed product integrability	Interchange and key limit
Integral-interchange revision	Interchange proved for nonnegative truncation parameters	Key limit
Completed assembly	Error estimates prove the limit; previous results supply the other conditions	<code>h_spine</code> removed

The nonnegative-parameter restriction narrows an overstrong internal plan. The proof sends its truncation parameter to positive infinity, so that restricted auxiliary statement suffices for the unchanged textbook target. This is different from adding a public assumption that excludes intended inputs: the former avoids proving an unnecessary lemma, while the latter can leave required source scope uncovered.

The early non-pass judgments already acknowledge mathematical progress. Counting the final acceptance alone would lose those increments; counting each new support file as a newly solved task would overstate them. Later movement of a complete argument into `thm_9_5_kernel` is separately documented in [B.5](#). That organizational change does not move the date at which the mathematical proof first closed. [E03, E05, E18]

M Descriptive statistics, process events and unified references

M.1 Individual runs and measurement rules

These 33 selected runs provide original-round time, role dispatches and tool returns. Each span runs from first dispatch to last role return, including within-window waits and recovery but excluding post-submission common evaluation. A-L1 and A-M1 are reconstructed estimates under the same endpoint definition, marked *. Time comparisons now cover all eleven groups. The table retains A-L7's original 58.86 minutes; its later 14.50-minute window gives a total of 73.37 minutes used in the main-text cumulative measure. Complete token comparisons still cover eight groups; see [N.5](#). Reconstruction evidence and both time summaries are in [P](#).

M.1.1 A

Group	Outcome	Minutes	Dispatches	Tool returns	Input M tokens
H1	Pass	182.16	16	484	95.568
L1	Pass	85.82*	8	136	—
L2	Pass	14.25	4	49	4.542
L3	Fail	69.78	12	150	15.728
L4	Pass	46.11	13	105	12.001
L5	Pass	76.96	10	182	—
L6	Pass	34.34	8	73	6.697
L7	Incomplete before con- tinuation; now pass	58.86	12	137	14.420
M1	Pass	136.89*	32	326	—
M2	Pass	59.54	7	138	13.880
M3	Pass	21.13	3	133	7.509

M.1.2 B

Group	Outcome	Minutes	Dispatches	Tool returns	Input M tokens
H1	Pass	126.17	7	513	63.346
L1	Pass	11.90	2	33	2.356
L2	Pass	30.09	3	93	6.397
L3	Fail	33.15	3	120	14.777
L4	Pass	15.89	2	81	5.025
L5	Pass	67.61	5	152	12.498
L6	Pass	21.98	3	55	5.777
L7	Pass	28.99	2	107	7.825
M1	Pass	17.64	3	63	4.426
M2	Pass	36.38	2	127	15.237
M3	Pass	41.54	3	111	9.240

M.1.3 C

Group	Outcome	Minutes	Dispatches	Tool returns	Input M tokens
H1	Pass	103.36	12	464	55.201
L1	Pass	12.16	3	44	2.571
L2	Pass	18.00	4	76	3.893
L3	Pass	39.90	7	108	7.893
L4	Pass	16.54	5	68	2.916
L5	Pass	28.70	5	104	10.651
L6	Pass	23.61	5	74	5.608
L7	Pass	32.27	7	106	9.129
M1	Pass	26.79	5	117	8.445
M2	Pass	36.53	6	134	12.499
M3	Pass	15.23	3	66	3.860

The original derived-metrics file and the supplied run index retain the metric-specific group membership and underlying values. [31]

M.2 Actual process-log coverage

All 244 role logs for the 41 existing solver roots were read, with 0 missing raw logs and 0 malformed nonempty lines. Native dispatch/item identity deduplicates tool returns; the 33 main roots are selected separately. The index retains 24 main-panel started items without completed returns. They are excluded from completed-return counts and do not indicate currently running tools.

Signal	A	B	C	Total
Compilation or API-probe returns containing Lean errors	180	291	322	793
Of those, name/type interface diagnostics	103	171	171	445
Executable command unavailable	76	58	22	156
File-access or permission errors	27	21	30	78
Explicit tool timeouts	2	1	0	3
Nonzero help-request returns	10	0	0	10
Of those, workspace copy/export failures	6	0	0	6
Explicit binding rejection during review application	1	0	0	1

Rules examine actual tool results: explicit timeout metadata; stderr for missing commands and file-access problems; an explicit runtime exception for copy/export failure; and actual review application plus mismatch text for binding rejection. Compiler diagnostics require a command-position Lean/Lake invocation or explicit build check, excluding echoed or written command text. Interface diagnostics are a subset concerning names and types. Deliberate probing and repeated attempts remain included, so return counts are not independent defects.

Host control, external endpoint evaluation and retrospective semantic judgment are outside this tool-return table. B-L2 host export failure/recovery and L5's unestablished adoption remain in G. Zero matched tool returns cannot

establish absence of faults in other layers. Categories overlap. `CLASSIFIED_PROCESS_EVENTS.jsonl` retains events, commands, source locations and matched excerpts; the complete extraction index remains `PROCESS_TOOL_EVENTS.jsonl` in the analysis directory.

M.3 Co-occurrence within a run

Category	Lean	Command	File	Timeout	Copy	Binding
Lean	33	33	27	3	4	1
Command	33	33	27	3	4	1
File	27	27	27	2	3	1
Timeout	3	3	2	3	1	1
Copy	4	4	3	1	4	1
Binding	1	1	1	1	1	1

Cells count main-panel runs containing both tool-signal categories; diagonals count runs containing each category. This is run-level co-occurrence, not simultaneity, a shared cause or a causal association.

M.4 Unified reference mapping

Ref.	Material	Original identifiers or files
[1]	Successive mathematical progress in inversion	E05
[2]	Editing, semantic review and registration	E01, E10, E15
[3]	Early orchestration and code editing	E01, E14
[4]	Gamma-to-Dirichlet proof responsibilities	E03
[5]	Proof obligations and parent reassessment	E02, E04, E11
[6]	Mathematical gates and MGF proof assembly	E02, E07, E10, E16
[7]	Total variation and its consumers	E08
[8]	CLT and coupon-collection chain	E18, E20
[9]	Target changes and review objects	E09, E18, E20
[10]	Historical comparisons, failing origins and selection	E18, E19
[11]	Saved-code checks and rule comparisons	E18, E19
[12]	Configurations and common task instructions	R01, R02, Y001
[13]	Corrected acceptance and current panel	Y001, Y002, Y003
[14]	Per-run durations, dispatches and tokens	ATTEMPT_CENSUS_DATA.json; DERIVED_RUN_METRICS.json
[15]	Evaluator repair and regression checks	Y005, Y006
[16]	Feedback semantic coverage and evidence index	Z016; EPISODE_SEMANTIC_COVERAGE_FINAL.json
[17]	H1 actual reuse, support and recovery	U07-U15
[18]	B/C-M1 adoption, build recovery and documentation	U01-U04
[19]	Assistance production, delivery, recovery and adoption	R02, R03; T03-T06; Z015, Z016
[20]	C-L1 readable review and submission	N10, N11
[21]	Corresponding L2 calls and consistent interpretation	Q017-Q027, Q098-Q107; Z020, Z021; X018
[22]	L7 condition repair, proof and disclosure	X003, X012, X053, X058; Y001, Y002
[23]	L3 tail and H1 textbook scope	X003, X012; Y001-Y003
[24]	Raw tool returns, process classification and co-occurrence	PROCESS_OBSERVATIONS.json; CLASSIFIED_PROCESS_EVENTS.jsonl
[25]	M2 raw-to-centered-moment repair	R02, R03, R14, R18; G.14.1
[26]	Previously captured local architecture, workflow and version	V01-V06, P01-P02; A.5
[27]	L1 mathematical routes, control, information and later revisions	N01-N13, R06-R08; G.5-G.10

Continued on next page

Ref.	Material	Original identifiers or files
[28]	M3 full-task adoption and confidence-format adjudication	U05-U06; G.12
[29]	Intermediate L3 premise removal and version-bound repair	R02, R03; T02; G.14.2
[30]	Common-evaluation method, settings and correction stages	Supplied A01–A40, AP01–AP11 and selected opinions; N.6
[31]	Run selection, input correspondence and missing usage	Supplied B01–B12 and RUN_INDEX; N.1/N.5
[32]	Historical target decisions and sampling metadata	Supplied C01–C07, E01–E02; C.4/D.1
[33]	L1 recovery and session-attribution conflict	Supplied D01–D05; G.6
[34]	Documented task lifecycle, repair, scheduling and canonical-state maintenance	Public architecture, workflow, workspace-state and dependency docs; A.5, E.3
[35]	Textbook–Mathlib interfaces and proof responsibility	Public interface policy, dependency trail and review criteria; E.3
[36]	Public implementation and workflow demonstration	Public architecture and workflow-demo docs; E.3
[37]	Candidate-state and six-service source correspondence	Revision 12 guide; supplied MAINTENANCE_SCOPE.csv, L2_SERVICE_AUDIT.csv, candidate/L1/H1 maps and Z015/Z016 originals
[38]	Independent audit of A/B/C versions, tools and information access	O.1; verification/revision13/abc_version_audit.md
[39]	13 September A-L7 continuation and L3 review-responsibility audit	O.2–O.3; verification/revision13/SOURCE_INDEX.json and the continuation and responsibility records linked there
[40]	Stepwise A-L3 review and application-gate audit; C-L1 information and submission	O.4; verification/revision14/SOURCE_INDEX.json; G.8–G.9

The process classifications are retained from the original `PROCESS_OBSERVATIONS.json` and its event files. E.2 and the accompanying source guide locate the original criteria, metrics, events and coverage records.

N Production acceptance mapping and experimental detail

N.1 Inputs, versions, candidate selection and time allowances

N.1.1 Shared task material and the observed input checks

The common asset rules require the same group’s seeds, textbook source, task sheet, library version and whitelisted modules to be supplied consistently. They permit legitimate mathematical-library facilities and equivalent proof routes, while excluding old candidates, production trees, Git history and other runs. Target-specific work must be proved rather than supplied through extra premises, renamed conclusions or private axioms. Authors also explain how the public declarations meet the task. [Appendix K; R21]

The supplied evidence checks the selected **common-evaluation bundles for all three configurations**, including C. Source and task manifests match in each of the eleven groups. Upstream manifests match in ten. For vector convergence (L7), A’s list additionally contains the continuous-mapping target as a one-line “Blank experimental target” file, with no implementation; B/C do not list that placeholder as upstream. This is a packaging difference, not completed proof work. The check establishes the frozen evaluator inputs, rather than reconstructing every earlier file accessible during solving. [31]

The separate variance-library investigation also connects the original A/B/C L2 containers to the same immutable image, read-only root and matching library files, with no mount overriding that library directory. That is an actual environment check for L2, distinct from the general protocol requirement. Runtime and recovery differences in other tasks remain part of their histories. [31]

M2 illustrates a substantive ambiguity in otherwise shared materials: its preparation sheet describes a centered fourth-moment bound, but the textbook source gives a raw fourth-moment bound. The source-facing theorem must derive

the centered bound. The later L7 textbook context and condition clarification were supplied retrospectively, not as initial solver input. G.14.1 and G.16 explain their mathematical consequences.

N.1.2 Which versions supply the main comparison

Execution family	Main-panel use	Other retained attempts or material
A: earlier development	Nine groups: L1–L7, M1 and M2	Runtime and host interventions remain in their accounts
A: later runtime	H1 and M3	H1’s partial and complete deliveries belong to the same continued root
A: separate vector-convergence repair	Not a replacement main entry	One supplementary L7 root
B: later transport-capable implementation (r4)	All eleven groups	Five original development roots and one capability-version root (r3) remain supplementary
C: later coordinator runtime (r3)	All eleven groups	One initial coordinator root remains supplementary

This gives 33 selected entries and eight additional roots, for 41 in all. Within a root, the saved aggregation takes the first complete candidate with a common decision in the bound list, or the first delivery with a common decision if no complete one exists. Roots without a common decision remain in the all-version inventory without an assigned common verdict. It then applies the version filter above. The code does not select a candidate for its favorable verdict; it is a retrospective aggregation rule, with no recovered evidence of a pre-solve freeze. Selection preserves A-H1’s same-root completion and A-L7’s separate repair identity. [31]

The version names correspond to operational differences. Earlier B lacked parts of the intended tool capability; its r3 capability repair still encountered helper and review-return failures. The r4 implementation supported readable feedback, although it also experienced the L2 export fault. Initial C’s checks ran but their opinions did not return; later C r3 delivered readable feedback in the innovation case. A retains author-selected search and editing alongside review gates, recovery in L1/H1, and the broader host stop in L7. These are actual conditions of the comparison, not matched repetitions differing only in organization. [R21; N01–N11; 31]

N.1.3 Recorded models, permissions and stopping

The saved main-panel requests and receipts identify gpt-5.6-sol, medium, for A/B authors and services and for C’s writers and checks; C’s coordinator uses gpt-6-astra, medium. These are the recorded execution settings; the evidence supplement holds the per-role records. [31]

Main-panel groups	Actual top-level allowance, all three configurations
L1	90 minutes
L2–L7	2 hours
M1–M3	4 hours
H1	10 hours

The clock begins at first actual dispatch and includes same-root assistance waiting and recovery. The allowances above come from the recorded runs rather than being inferred from the planned workload tiers. Actual stopping instructions take precedence: A-L7 was sealed with time remaining, while H1 continued within its original allowance. A’s early M2 prompt prohibited help and required a return after preparing review; those actions cannot be read as voluntary abandonment. Different historical ending events are retained in H, and the comparable span in M uses last role return. [R06, R07, R22; U10, U16; 31]

r3/r4 denote execution versions, whereas c1/c2 denote candidate versions within a run. The evidence index retains the per-run implementation records.

N.2 How these experiments support project acceptance evaluation

The report evaluates ProbabilityTheoryFormalization using two bodies of evidence. Historical reviews and code revisions examine actual proof responsibilities, assembly, caller maintenance and the reliability of acceptance records. The task comparison examines candidates, repairs and deliveries under A, B and C on the same mathematical groups. A uses the prescribed project workflow; B/C examine alternative organizations. This section states how experimental evidence relates to the project’s formal acceptance rules, rather than treating a common-evaluation pass or an alternative configuration as a verified production landing.

The current project contract connects three questions: whether a candidate meets the source statement and proof duties, whether independent review is bound to the exact code and dependencies, and whether the reviewed result can pass formal application into task completion. A successful build establishes technical readiness. Source-fidelity review also maps the source claims, assumptions, conclusions and essential proof steps, inspects listed direct downstream consumers, and assigns proof and completion classes. [P01, P02]

Evaluation question	Evidence already in this report	Relationship to formal project acceptance
Are the mathematical duties met?	Source tasks, implementations, and repairs in main Appendix R.2 and Appendix G ; common evaluations of complete or partial candidates	Experiments allow legitimate mathematically equivalent routes; production review additionally requires source-spine mapping and an eligible class
Does the judgment concern this candidate?	Candidate, source, contract, technical-fact and upstream bindings in K and N.6	Production review also binds the review request, prompt and rubric versions, and review basis; the records require an explicit correspondence
Can later tasks use the result?	Historical consumer repairs; actual H1 theorem calls and L2 definition-interface observations	In-group calls support local usability, but do not replace checks of listed production consumers
Has the project formally accepted it?	Builds, internal review registration, and common evaluation are reported separately	The production contract and application receipts govern formal acceptance; a report’s pass label cannot create it

Production review requires nonempty `proof_class` and `completion_class` fields. Together with the semantic verdict and source-task role, these feed the apply gate’s task-status projection; only an eligible `phase2_status=pass` denotes clean completion. `allowed_exception` is limited to explicitly listed task/class pairs and is not clean completion. Transmission failures, malformed results, and host-imposed stops do not automatically become allowed exceptions. [P01]

The panel measures experimental delivery, not the production catalog’s clean-completion rate. Builds, internal registrations and common decisions are reported where observed; a full per-candidate production-landing census is outside this study. Formal landing remains a separate question governed by the production classes, current review basis, downstream checks and application receipt.

N.2.1 Which artifacts the maintenance priorities concern

Follow-up. The table below records the earlier candidate states. A-L7 now also has a complete repaired delivery from the 13 September continuation; see [O.2](#).

The table maps the priorities to the frozen deliveries and their recorded subsequent states. L3/L7 statuses use the unchanged 12 September, 09:22 acceptance cutoff; L1/H1 rows retain their earlier event order. This is a bounded reading of supplied candidate, review and recovery records, not a fresh inspection of the live corpus. [22,23,27,33,37]

Observed object	Latest retained state in this study	Consequence for maintenance
L3: selected A, B r4 and C r3 deliveries	A/B omit the permitted eventual-tail interface; C delivers it. Reassessment withdraws A's earlier pass without new solving.	Complete the missing scope in those A/B artifacts before treating them as full deliveries; do not assign the same gap to C.
L7: original A, separate A repair, B r4 and C r3	Original A lacks continuous mapping. Its separate repair lacks composite measurability. B/C deliver complete, disclosed repaired statements.	Keep the two A candidates separate and retain their missing duties; B/C's repaired scope does not complete or replace them.
L1: early A and initial C; later B r4/C r3	A's opinion was host-recovered; its recovery-session attribution still conflicts. Initial C submitted without readable checks; later B/C received usable feedback.	Maintain the return path and distinguish repaired behavior from historical failure; the attribution conflict remains an evidence issue.
H1: A same-root continuation	The inversion helper product was recovered and adopted. The uniqueness review's binding was corrected without changing the proof; five targets reached internal delivery.	Preserve both artifact transmission and exact-input validation. Successful recovery is part of this result, not proof that the original faults never occurred.

For **all these rows**, a reliable experimental-candidate-to-current-canonical-object binding is absent from the supplied records. Current production adoption and post-adoption state therefore remain **unverified**, not failed and not implicitly passed. Internal experimental registration or a matching theorem name cannot supply that binding. The evidence guide maps the seven L3/L7 candidates and the L1/H1 records through `MAINTENANCE_SCOPE.csv`, `CANDIDATE_DELIVERY_INDEX.json`, `L1_MAP.json` and `H1_MAP.json`; it adds no production-landing verdict.

N.3 Completed coverage and the limits of retrospective correction

This section describes the original 12 September comparison. The additional 13 September continuation and audits are recorded in O and do not alter its denominators.

The comparison concerns three executed experimental configurations: Sol medium works through the existing workflow in A; a Sol medium lead author organizes B; an Astra medium coordinator organizes Sol medium authors and checking services in C. The coordinator model is part of C, so outcomes cannot isolate a coordination effect. Textbook proofs accompany the source statements; the experiment mainly evaluates implementation and collaborative completion.

The eleven groups were selected for mathematical and organizational work. B/C each added eight groups beyond L1, L2 and M1, completing sixteen first solves and their evaluations. A contains different development versions, an H1 same-root continuation and a separate L7 repair. The 33 main entries and 41 total solver roots have separate denominators; extra versions are not matched repetitions. [Z001, Z002]

First-pass records were consolidated at 03:07:37 on 12 September. L2/L3/H1 consistency assessment used 31 model calls. The targeted supplement ending at 05:17:25 reused valid L2 opinions and added two independent L7-C reviews without new solving. The evaluation repair completed at 09:22:31 reused this evidence, repaired parsing and adopted the current standard, adding no model evaluations, solver runs or builds. [X030, X031, Y007]

The current standard selects general expansion for L2, the task-permitted tail scope for L3, the textbook core for H1 and reasonable complete repairs for L7. It applies to every affected candidate, including changing A-L3 from pass to incomplete. This explicit retrospective correction makes existing outcomes usable for completion and trajectory analysis; it does not reclassify old solves as experiments begun under corrected contracts. Equal-condition repetitions have not been performed and are not prerequisites for interpreting the available material. [Y001-Y003]

N.4 What the final evidence covers

Analysis units. These units serve different questions; historical comparisons and saved-code pairs are not extra configuration trials.

Evidence unit	Role in the analysis
Main-panel delivery	Completion of the adopted obligations for one selected configuration–task entry
Solving run	One top-level attempt, including services, recovery and continuation under the same clock
Role dispatch	One attempted role invocation, including unsuccessful attempts
Tool return	Completed output visible to a model, including deliberate probes and repeated diagnostics
Service and finding	An assistance or checking episode, and the potentially multiple findings it contains
Historical comparison	An earlier/later relationship between judgments or candidates, with shared endpoints possible
Saved-code pair	Two frozen files selected for specific compilation, axiom and premise checks

The inventory below is the original experimental evidence set, before the separately reported 13 September continuation. It is not a combined denominator for the historical studies in C/D and the experiment.

The final delivery binds the identities and original evaluation sources of forty-one actual solver roots and indexes 127 internal service episodes and 121 individual finding records. Sixty-seven services have tool-result evidence of full feedback-text exposure. Missing exposure evidence remains unverified; it does not establish that an author failed to read feedback. [Z016, Z006]

The additional semantic and adoption audit at a common depth covers six L2 services. G.18 now presents every service’s saved validity, exposure and adoption result, including the unreconstructed links. The remaining 121 services have not received that same audit; they are not the 121 individual finding records. Earlier L1, M1, M2, M3 and H1 case studies in [main Appendix R.2](#) and [Appendix G](#) retain their own evidence and are not added to this uniform audit’s denominator.

The report consequently separates four layers: run and candidate identity, actual feedback exposure, validity of the feedback, and adoption that repairs the corresponding obligation. Indexing the first two layers cannot by itself produce a correct-feedback adoption rate or a mathematical-repair fraction.

N.5 Final denominators and costs by activity

The following accounts retain the original experimental collection and its dated supplements. The five calls for the 13 September A-L7 continuation and its evaluation are accounted for only in O.2, not added to the rows below.

The table separates actual calls by purpose across all versions identified in the final delivery. Tokens are cumulative recorded usage, not deduplicated textbook content. Authors and coordinators, internal services, post-submission evaluations, and engineering probes are separate categories. Input already includes cached input, and output already includes reasoning output; neither subset is added again. Values prefixed by “at least” are observed lower bounds, not estimates of complete totals. [Z011, Z002]

Cost category	Calls	Input incl. cache	Output incl. reasoning	Measurement scope
Authors and coordinators	117	≥408,139,733	≥1,782,393	Lower bound; missing calls

Continued on next page

Cost category	Calls	Input incl. cache	Output incl. reasoning	Measurement scope
Internal services	127	$\geq 121,219,401$	$\geq 1,132,494$	Lower bound; missing calls
Earlier provisional evaluation	11	4,240,524	69,275	Recorded usage complete
Original common evaluation	81	24,425,225	433,595	Recorded usage complete
Consistency reassessment	31	18,665,837	341,168	Recorded usage complete
Engineering model probes	3	204,798	3,604	Recorded usage complete

Five author/coordinator calls and six internal-service calls lack all four usage fields: input, cached input, output and reasoning output. Their locations are:

Run	Calls missing usage	What is missing	Effect on comparison
A-L1	2	One author call has a failed completion record without usage; one reviewer has no completion receipt	Time reconstructed; excluded from complete token comparison
A-M1	6	Two author and four service calls lack usage; four of these calls have no completion receipt	Time reconstructed; excluded from complete token comparison
A-L5	2	Two author calls have failed completion records with times but no usage	Span retained; excluded from common token comparison
A-L7 separate repair	1	A resumed mathematical-review call has a failed completion record without usage	Outside the main panel; affects all-version activity totals

Ten usage gaps affect three A main-panel runs, and one affects a supplementary repair. Complete token comparisons exclude L1, M1 and L5 and cover eight groups. A-L1 and A-M1 times have been reconstructed separately, so time comparisons including estimates cover all eleven groups. Recovering time does not recover missing tokens; see P. [31]

Complete all-version author and service totals remain unknown, so the table reports observed lower bounds. Missing-call proportions are not missing-token proportions, and no unsupported upper bound or imputation is used. Monetary cost and full engineering labor are not recorded; exports or recovery without a model invocation are not additional model calls.

[Appendix H](#) retains earlier-window solver measurements and must not be added again to this table. Root elapsed spans, sums of role durations, exact submission times, and last returns are distinct. Some historical times remain unknown or bounded only by observation. The final package connects roles and root identities so that resources and outcomes can be traced within versions, including unsuccessful attempts.

The 05:17 supplement adds only two L7-C independent reviews: cumulative input 1,425,757, including 1,254,912 cached tokens, and output 17,309; actual model duration sums to 481.00 seconds. There are zero new adjudications, solves or builds, and reused L2 reviews are not charged twice. Monetary cost is unavailable. This supplement is separate from the table's 31 reassessments at the 03:07 cutoff and from solver costs. Its 62 resource samples show at most two active containers and a maximum 5.520-second gap; atomic admission checks are recorded separately. Both containers and monitoring stopped on completion. [X020, X027]

The 09:22 evaluation repair added no model evaluations, solver runs, builds or containers. It implemented compatible parsing and current-rule synthesis, with eighteen recorded regression checks passing. Existing model costs are not counted again as repair-stage calls. [Y005-Y007]

N.6 Common evaluation: procedure, visible evidence and corrections

N.6.1 Evaluation roles and input

Common evaluation assesses submitted experimental artifacts after solving. It is separate from the help and review services used inside a run and from production review-apply. The supplied requests, bound program excerpt and execution receipts establish the procedures below. [30]

Across the 81 original common-evaluation calls, 31 consistency calls and two targeted L7 calls, requests and local receipts identify gpt-5.6-so1 with medium reasoning effort. The 114 recorded session identifiers are distinct, and no source-session continuation is recorded in that index. These settings and session separation support the stated evaluation configuration; they do not make repeated uses of one model statistically independent.

The original reviewer receives the source statement and argument, task and common rules, the candidate, permitted upstream/support files and neutral technical facts. The prompt requires source-faithfulness, obligation, premise, proof-route and dependency checks. Building establishes a technical fact; the reviewer must still assess the actual written mathematics. It cannot edit the subject, invoke a helper or treat the author’s own completion claim as proof of success.

At the request interface, the candidate is anonymous and configuration metadata and host paths are excluded. The initial consistency-review prompts also withhold old verdicts and other candidates’ opinions. This is **metadata masking**, not a demonstrated guarantee that code style, comments or file content contain no identifying clues. Later adjudicators intentionally receive additional evidence, described below.

N.6.2 Original dual-review and adjudication rule

Available validated results	Recorded next action or outcome
Fewer than two initial results	Await the missing result
Two pass votes or two fail votes	Finish with that verdict
Two inconclusive votes	Finish unresolved
Different initial votes	Request at most one fresh, read-only adjudicator
Valid adjudication gives pass or fail	Finish with the adjudicator’s verdict
Valid adjudication remains inconclusive	Finish unresolved

The adjudicator reads the same subject and two de-identified opinions. Its instruction is to resolve the concrete disagreement against the common task, not to choose by majority or confidence. Result validation checks candidate/request identity, the expected role, a fresh session, read-only subject evidence and required result fields. Repeated session or dispatch identity is rejected. These are the archived program rules, not a voting rule inferred from the total call count. [30]

For a complete candidate, the intended unit is the whole required task group. Explicit partial-material review instead assesses the saved material’s quality. The original A-L7 sequence exposed a mismatch because its adjudicator did not inherit the initial reviewers’ same partial-scope restriction. G.16 retains that discrepancy and the independent fact that the continuous-mapping branch was absent; a passing first target does not complete the group.

Malformed output is an execution/evaluation state rather than a mathematical counterexample. In A-M3, the second raw opinion says pass but supplies an object-valued confidence field. The host records an inconclusive execution result; paired with the other pass, this triggers adjudication, which passes. Similar confidence-type failures appear in other original endpoints. The original model text and the host-generated substitute remain separate records, so an inconclusive substitute is not attributed to the model as its mathematical vote.

N.6.3 Consistency reassessment and new input

The L2, L3 and H1 reassessments evaluate alternative interpretations of existing tasks. They preserve the submitted code, use two separate initial reviewers per candidate and complete the initial readings before checking cross-candidate rule application. At most one necessary adjudication per candidate can then be triggered by a disagreement on a required dimension or obligation, an invalid initial result, or inconsistent treatment of corresponding behavior. [30]

The adjudicator receives its candidate, its new initial opinions and a cross-candidate consistency summary, not the other candidates' source code. It can reject initial reasoning. For each interpretation, all required mathematical, premise, route, interface and delivery obligations must pass: a valid required failure makes that interpretation fail; otherwise unresolved evidence makes it unresolved; otherwise it passes. When interpretations change the result, the reassessment retains the criterion dispute rather than selecting the interpretation that favors a configuration.

Task	Interpretation or input change	When it entered evaluation
Variance and weak law (L2)	General mathematical expansion versus expansion proved in the candidate; frozen library correspondence checked	After the original evaluations, before the new independent consistency reviews
Scheffé and density convergence (L3)	Termwise-finite and extended-integral readings, followed by the task-permitted eventual-tail reading	The tail reading was added after the initial termwise-finite and extended-integral reviews exposed an excessive requirement; all four retained candidates received all three readings at adjudication
Inversion and applications (H1)	Standard-Cauchy versus arbitrary location-scale scope; source-code completion separated from explanatory delivery	Scope sensitivity was recorded before B/C H1 dispatch; the detailed reassessment rubric followed candidate completion. Original submission material was added after six initial reviews and before adjudication for all three candidates

The H1 material was the actual final message, original submission manifest and delivered explanations, not a newly written justification. Its inclusion addressed an incomplete evaluator view: anonymous Lean code alone could not show whether the author had supplied the required explanation. Source completeness and documentation remained separately assessed. The rule changes and material additions above have their actual timing; they were not all part of the original solve-time protocol.

N.6.4 Targeted continuous-mapping review and final criterion synthesis

The supplement completed at 05:17 supplies two further independent readings of the existing C-L7 candidate. Reviewers receive the original task, source, candidate and delivery explanation, plus explicitly retrospective textbook context. They separately assess mathematics, reasonableness of the repair and compliance with the original literal contract. Both valid opinions agree on all ten obligations, so no adjudication is needed. The candidate's original build evidence is reused. G.16 explains how mathematical correctness of a disclosed repair can coexist with non-compliance with the insufficient original conditions. [30]

At 09:22, the current acceptance selects general expansion for L2, eventual-tail scope for L3, the textbook standard-Cauchy core for H1 and reasonable, disclosed, fully proved repairs for L7. It reuses the existing opinions. The result processor accepts optional confidence metadata and determinable JSON/text escaping correspondences while preserving votes, mathematical symbols and reasons. Candidate or source identity mismatches still block use; an unresolvable quotation requires evidence rather than an invented match. [30]

This stage contains **both processing repair and a substantive acceptance choice**. The latter, not formatting alone, permits the repaired L7 statements. It also withdraws A-L3's former pass for a real tail-interface omission. The eighteen recorded regression checks test processing, not mathematical truth, and the stage adds no new model calls, solves or builds. Original outcomes and intermediate scope tables remain in K.8; K.9 identifies current acceptance.

O Evidence added on 13 September 2026

This appendix supplements the retained 12 September comparison and earlier appendices. Statements of “current” results in those dated records describe their original cutoff. First-batch results include the completed A-L7 continuation and give A 10/11, B 10/11 and C 11/11. Q gives the second batch and common reconciliation; earlier records support trajectory reconstruction. The 13 September additions did not include a new L3 solve or new L3 reviewer opinion; the L3 audit examined the existing decisions. This edition retains those dated records; it does not rerun the continuation or the recorded checks.

O.1 Configuration differences and actual exposure

The saved comparison examined frozen prompts, execution code, launch manifests and actual returns. The table identifies changes that matter for interpreting Section 2.3. It separates the configured material from evidence that an agent actually received or used it. The [source comparison](#) retains the detailed locators. [38]

Change	Content and affected runs	What the records establish
B’s access to project support	Early B trials advertised capabilities without the project engine and reference material being mounted. Main-panel B uses the later common reference package and working assistance interface.	An early B-L2 launch confirms the missing mounts; later capability and transport probes confirm working access. Early B trials are outside the main panel.
Feedback and helper-code return	Later B returns readable text and the actual inner patch to the caller. C changes export from read-only workspaces and saves role results before a later export check can fail.	B engineering probes read and apply a patch; a B-M3 record also shows a returned patch and application. Early C-L1 receives export errors despite completed checks; later C-L1 receives a readable pass and responds. A successful return does not establish every service’s success or adoption.
L4’s named-interface requirement	B’s final reference package removes a requirement that the distribution calculation call the exact discrete total-variation interface.	The common task still requires the discrete and continuous formulas and the example calculation. A-L4 actually receives a conditional instruction naming that interface. The source of that instruction and its effect on the chosen proof are unresolved.
M3/H1 organizational hints	B’s final references omit production module names and their parent-task membership for Skorokhod representation and inversion.	The removed rows suggest a decomposition but contain no Lean proof body. Whether A-M3 or A-H1 actually opened or used those rows has not been reconstructed.
C’s task expansion	The early pilot has a fixed allowance and review target. Later templates substitute the task allowance and target; the author and mathematical-consultation templates remain unchanged.	Selected C runs share the core backend and role-prompt contents, with launch copies admitting different tasks. This commonality concerns C’s selected implementations, not identical A/B/C contexts.

L4: an implementation constraint, not an additional theorem. The removed instruction forbids proving the Bernoulli-versus-Poisson calculation without using the declaration `thm_8_6_discrete`. The public task already requires both total-variation formulas and the calculation, while the common rules allow mathematically equivalent proof routes. Removing the named-interface mandate therefore broadens implementation choice without removing those mathematical obligations. Additional reminders to prove both branches were also removed, but their obligations remain in the public task. A-L4’s fifth author prompt names the interface conditionally; that is direct exposure evidence, not proof that the condition was followed or that this reference caused it. [38]

M3/H1: proof-organization clues without supplied proof code. The removed M3 names point to quantile definitions, the quantile law and space, inverse comparison and quantile convergence. The public M3 task already lists the probability space, quantiles, measurability, distributions, inverse images, exceptional levels and convergence. The H1 names point to Dirichlet, Fubini and kernel components of the inversion implementation; the public task requires fresh task-specific analytical support. These names could help organize or locate work, so an effect cannot be assumed absent. The saved evidence establishes their removal from B’s reference package, but not their use by the corresponding A authors or the size of any benefit. [38]

Implementation commonality has a precise scope. C’s template adaptation does not exhaust its changes: the return mechanism, persistence of results and readable artifact types also changed. Conversely, different launch-copy names among selected C runs do not imply different core role prompts or backend logic. B accesses a projected reference package; C’s coordinator uses the registered-artifact interface described in N.1. These are different information arrangements. The records support comparison of the executed configurations, with the concrete exposure limits above, rather than an isolated estimate of organizational effects. [38]

A-L7’s external stop changed the permitted next actions even with time remaining. O.2 and Appendix R.2.6 separately report that stop and the completed continuation; the earlier configuration comparison did not include the later continuation as if it had already occurred.

O.2 A-L7 continuation and artifact identity

The original author session 01a08d9c-6892-70c2-a44c-6fe9afa47be7 resumed on copies of its sealed state, with explicit permission to disclose a reasonable correction. The [structured result](#) and [evidence facts](#) record execution and validation. The [code difference](#) changes explanatory comments: the theorem signature and proof body were already in the sealed working candidate. This candidate is distinct from the incomplete delivery selected for the old main panel and from the earlier separately restarted repair.

The two fresh evaluations, [reviewer 1](#) and [reviewer 2](#), both pass mathematics, repair reasonableness and disclosure, and both fail literal original-contract identity. The added conditions are Borel measurability of the mapping, ambient continuity at each point of the set, and a measurable full-probability membership event for the limit. Composite measurability is proved. These conditions are sufficient repairs; the result does not establish a unique weakest repair.

The author ran from 11:04:09 to 11:18:39 Beijing time. Common evaluation ran separately, with approximately 6.7 and 6.0 minutes in parallel. Five calls used 7,641,026 input tokens (7,054,720 cached, already included) and 52,712 output tokens. Docker communication recovery and the execution-program version change are recorded in the original recovery material. One completed evaluator response required an offline dictionary/list structural conversion; the mathematical opinion was preserved. These later costs do not overwrite the original-round tables. The 870.275890-second solving window enters the cumulative measure, with independent common evaluation separate; see P.

O.3 L3 scope and actual reviewer exposure

The [updated responsibility audit](#) supersedes the earlier suggestion that A’s mathematical reviewer might not have received the tail wording. The actual returned tool output includes it. A’s three Math Gate rounds occurred in one reviewer session and passed; general review also accepted termwise integrability. B’s optional review used a fresh Sol session, not repeated self-review in the author session. Its response explicitly treated the tail alternative as optional. Neither author subsequently received an explicit repair test for this omission.

The distinction is therefore between an established scope and review failure under the adopted acceptance, and an untested claim about inability to construct a tail proof. The original task’s permissive wording and retrospective clarification of acceptance remain limitations. A/B still lack the required tail interface; their labels are not changed to pass. Recorded earlier fail-to-pass corrections for C-L2, B-H1 and B/C-L7 were already incorporated in the 12 September main panel.

The [source index](#) records the original absolute paths and SHA-256 hashes of the copied follow-up records. Those records preserve their original text; resolve any original relative links against the source locations in the index. The 13 September evidence update reports checks of these records and selected code differences, not a new Lean build or independent expert certification. Revision 18 retains that attributed account; its own editorial checks do not re-verify the linked raw records. Older evidence remains available through the retained evidence guide.

O.4 A-L3 review gates and the scope of A/C trajectory analysis

A's [second author instruction](#) applies a failed review and requests repair. All three [Math Gate rounds](#) pass. The [general re-review](#) accepts finite-integral encoding, lists no interface mismatch and checks a density caller already satisfying termwise integrability. The [fifth instruction](#) skips the passed target. The [program documentation](#) describes result-structure and binding checks, not fresh mathematical judgment; the [audit](#) connects these duties to execution. The [source index](#) records provenance.

C-L1's [early run](#) receives two check-export errors; later [r3](#) acknowledges a general pass, requests no mathematical consultation and submits unchanged code. [O.2](#) covers A-L7; [main Section 2.5](#) states the semantic-audit coverage limits.

O.5 A in M1 and B in H1: completion, timing and assessment scope

A-M1's [original final decision](#) records two passing reviewers and a final pass. The historical [run index](#) omitted the complete span because four role calls lacked completion receipts. This edition reconstructs approximately 136.89 minutes using the same first-dispatch to last-return definition as the other runs, marked as an estimate. Six calls separately lack usage data, so the group remains outside the complete token comparison. [Section 3.2](#) and [P](#) provide the time and evidence; the case selection in [Appendix R.2.4](#) does not affect A-M1's pass.

B-H1's [original adjudication](#) passes mathematics, public premises, the standard-Cauchy route and interfaces, while failing the arbitrary-location and positive-scale family route. B delivers the zero-location family with arbitrary positive scale, which includes the standard case, but no arbitrary-location reduction. The [adopted acceptance rule](#) of 12 September uses the standard-Cauchy core while retaining all five targets and proof obligations. The current synthesis therefore passes B-H1, with its arbitrary-location extension separately incomplete. This is not a new judgment or solve in this revision.

The original files are copied unchanged with a [source and checksum index](#), supporting the checks of [Sections 3.2, 4.3–4.5](#) and [5.4](#). The first-batch summary includes A-L7's completed continuation and gives A 10/11, B 10/11 and C 11/11; this edition's [Section 3.1](#) and [Appendix Q](#) separately report second-batch results. Earlier judgments and candidate states remain in their dated records.

P Revision 22 timing reconstruction and common definitions

This read-only check covered the timing databases for all 33 original runs and same-root recoveries. The other 31 complete spans reproduce the existing table as last role return minus first dispatch; native clock starts also equal first dispatch. Database hashes were unchanged. The [row-level check](#) retains timestamps, source paths and hashes.

A-L1 lacked one closing receipt and A-M1 lacked four. Their last retained role returns yield 85.822436 and 136.892199 minutes. All five corresponding containers stopped before the respective final return. The [reconstruction record](#) also retains controller evidence and final-candidate correspondence. Container stops support the estimates but do not recover the missing host receipts, so both values retain estimate markers.

Task group	A original round (minutes)	B original round (minutes)	C original round (minutes)
L1	85.82*	11.90	12.16
L2	14.25	30.09	18.00
L3	69.78	33.15	39.90
L4	46.11	15.89	16.54
L5	76.96	67.61	28.70
L6	34.34	21.98	23.61
L7	58.86	28.99	32.27
M1	136.89*	17.64	26.79
M2	59.54	36.38	36.53
M3	21.13	41.54	15.23

Continued on next page

Task group	A original round (minutes)	B original round (minutes)	C original round (minutes)
H1	182.16	126.17	103.36

* marks reconstructed estimates. All gaps between each original run’s endpoints remain included; waits and recovery time are not selectively subtracted by configuration.

A-L7’s continuation author window is 870.275890 seconds, or 14.504598 minutes. Its original round is 58.864038 minutes, giving 73.368636 minutes across the two windows. The [continuation result](#) supplies the measured seconds and calls. Window accumulation excludes the sealed period between separately authorized solving stages and independent post-submission common evaluation. Assistance and waiting within each window remain included; overlapping role durations are not added again.

Metric: median (range)	A	B	C
Original-round elapsed time (minutes)	59.5 (14.2-182.2)	30.1 (11.9-126.2)	26.8 (12.2-103.4)
Cumulative solving-window time, including continuation (minutes)	69.8 (14.2-182.2)	30.1 (11.9-126.2)	26.8 (12.2-103.4)
Original-round role dispatches	10 (3-32)	3 (2-7)	5 (3-12)
Original-round model-visible tool returns	137 (49-484)	107 (33-513)	104 (44-464)

Both time measures use all eleven groups. A’s original and cumulative medians are 59.543999 and 69.783549 minutes; B’s is 30.087101 and C’s is 26.792389. The [unified data](#) retain unrounded values and calculation rules. Revision 20’s nine-group summaries remain in that release and frozen evidence; new values do not overwrite historical records. Role and tool counts still describe the original round; the eight-group token coverage is unchanged.

Q Common evaluation and completed second-batch results

Q.1 Execution design and actual differences

The second batch completes 33 main runs in a registered ABC/BCA/CAB rotation, with at most two main runs active. Common acceptance occurs after solving; its later six-way concurrency does not change solving time. Separate work comprises 12 fixed-code diagnostics and 2 repairs. All 35 main or repair deliveries complete technical checks and final decisions. Model names and reasoning settings match the first batch: Sol at medium effort for authors and checkers, Astra at medium for C coordination.

Group	Added instruction
L2	Explicitly allows equivalent generic variance expansion; cross terms still must be eliminated
L3	Makes eventual-integrability coverage of the original sequence explicit before solving
L7	Allows a disclosed sufficient premise repair in advance, without requiring the weakest repair
M2	Requires raw-to-centered moment derivation explicitly; the subsequence-route instruction was present in both batches
H1	Clarifies all five targets and the standard Cauchy core, with parameter extensions separate

Source mathematics and supplied upstream files in the 214-item seed inventory, mathematical toolchain and dependencies are unchanged. A-L1/M1 change from fresh review after repair to same-session review. Eight groups

already using same-session review change their prohibition on revisiting passed goals; M2 initial review handoff is also unified. Some exact module hints are removed from A, and B regains both-branch completeness reminders. The client changes from 0.153.4 to 0.154.0-alpha.6.2 with different tool routing; the later first-batch A-L7 continuation already uses the newer client. All 11 B initial author prompts match, and C differs only in some numeric formatting. These are not model replacements.

The first batch selects 33 deliveries retrospectively from 41 development runs; the second registers 33 in advance. They are not random repetitions of one unchanged setup. Common rescoring changes the assessment of old artifacts, not what their authors knew while solving. Sources: [actual-condition comparison](#), [frozen protocol](#) and [original results](#).

Q.2 Common outcome criteria and reconciliation of 66 items

The common outcome policy is adopted after observing the original results, using textbook goals and the allowed mathematical scope. The table applies to all configurations in both batches. Previously corrected variance, tail, Cauchy and reasonable-premise-repair criteria remain; this edition makes acceptance of equivalent routes and modular interfaces explicit.

Group	Common mathematical requirement
L1	Innovation and partial sums, integrability, adaptation, zero conditional innovation mean and martingale from the original observations.
L2	Uncorrelated variance sum and weak law; generic variance expansion is allowed, cross terms and probability limit must be proved.
L3	Scheffe for eventual integrability of the original sequence, density total variation and distribution limits, including the specified normal boundaries.
L4	Both discrete and density total-variation formulas from the event supremum; Bernoulli and Poisson applications with endpoints.
L5	Maximal coupling, probability and marginal laws, attainment, with common mass zero and one.
L6	One- and two-sensor quadratic mean-square error, global optimality and valid boundary cases from source assumptions.
L7	Coordinate equivalences and continuous mapping, including output measurability; disclosed mathematically sufficient premise correction is accepted.
M1	Stopped martingale and constant expectations, stopped-value existence and all three optional-stopping cases with required domination and convergence.
M2	Full-sequence strong law from source raw fourth moments and independence; centering, moment expansion, summable tails and Borel-Cantelli. Equivalent direct full-sequence route is accepted.
M3	Constructed common probability space and quantiles, measurable marginals, inverse comparisons and almost-sure convergence, with endpoints and countable exceptional levels.
H1	All five goals, including inversion and uniqueness with task-specific analysis, standard Cauchy core and integer-support criterion; extra parameters optional.

Each row retains the correspondence between code, sources and assessed candidate. Applicable existing semantic opinions and build checks are reused. The program verifies 1241 bound files against the complete panels and rejects missing or duplicate rows, unexplained corrections and interface corrections without successful caller checks. This is retrospective common-policy reconciliation, not a fresh blind mathematical review of all 66 items. First-batch A-L7 uses its separately bound continuation result; the original candidate remains historical provenance.

Interface checks preserve the original task inputs: they instantiate delivered results, convert equivalent representations, or use general permitted-library facts to discharge redundant premises. The evaluator does not construct replacement inputs and add an omitted task-specific scope extension. This accepts the nonnegativity implication on C-L3's existing functions while retaining the missing tail argument in first-batch A/B-L3. Noncompliance with an instructed route remains recorded separately from the validity and completion of the mathematical result.

Panel	Passes recorded before this reconciliation	Common completed outcomes
1-A	10/11	10/11
1-B	10/11	10/11
1-C	11/11	11/11
2-A	10/11	11/11
2-B	10/11	11/11
2-C	10/11	11/11

For the first batch, the pre-reconciliation record already includes previous scoring corrections and the A-L7 continuation. For the second, it is the original final common acceptance before the three present changes. These are not original scores from a common stage; K.8.1 retains the earlier first-batch labels. The field `recorded_outcome` uses this definition, with each stage specified in `recorded_outcome_stage`.

The three changes discussed in the main text retain their reasons. All 66 rows are available as a [table](#) and a [record containing original decisions and code identities](#). The procedure and rules are supplied in `evaluate.py` and `policy.json`. Original results are preserved.

Q.3 Second-batch time and usage by item

Item	Original min	Recovery min	Total min	Endpoint correction sec
A-L1	15.21	0.00	15.21	0.000
B-L1	14.84	0.00	14.84	0.000
C-L1	15.27	0.00	15.27	8.856
A-L2	16.75	0.00	16.75	0.000
B-L2	16.30	0.00	16.30	0.000
C-L2	18.98	0.00	18.98	7.191
A-L3	25.64	18.71	44.35	0.000
B-L3	13.65	0.00	13.65	0.000
C-L3	23.34	0.00	23.34	9.664
A-L4	41.03	0.00	41.03	0.000
B-L4	18.79	0.00	18.79	0.000
C-L4	18.04	0.00	18.04	6.452
A-L5	32.72	0.00	32.72	0.000
B-L5	19.70	0.00	19.70	0.000
C-L5	30.09	0.00	30.09	9.257
A-L6	24.51	0.00	24.51	0.000
B-L6	34.13	0.00	34.13	0.000
C-L6	21.49	0.00	21.49	8.841
A-L7	21.61	0.00	21.61	0.000
B-L7	23.99	0.00	23.99	0.000

Continued on next page

Item	Original min	Recovery min	Total min	Endpoint correction sec
C-L7	22.01	0.00	22.01	7.409
A-M1	51.91	0.00	51.91	0.000
B-M1	23.70	0.00	23.70	0.000
C-M1	24.19	0.00	24.19	30.815
A-M2	51.17	0.00	51.17	0.000
B-M2	40.56	0.00	40.56	0.000
C-M2	22.91	0.00	22.91	7.644
A-M3	50.24	0.00	50.24	0.000
B-M3	20.21	0.00	20.21	0.000
C-M3	18.40	0.00	18.40	6.920
A-H1	135.59	0.00	135.59	0.000
B-H1	123.89	0.00	123.89	0.000
C-H1	86.42	0.00	86.42	8.582

C originally used submission as its endpoint; this table uses the last role return. The correction adds 111.631 seconds across 11 runs, with individual additions of 6.45–30.82 seconds. A and B original windows already match that endpoint. A-L3 recovery uses the same endpoint at 1122.899 seconds; a further 4.765–5.420 seconds belongs to outer bookkeeping. That overhead is not selectively added to one configuration. Pauses between separate windows are excluded from their sum; internal waiting is retained. M.1 and P give first-batch rows and the basis for its two estimates.

Original main runs	Input	Cached subset	Output	Reasoning subset	Role dispatches
A	143818952	136488448	968369	259453	63
B	97604774	93368448	674847	183177	48
C	87452495	83553536	603715	150635	52

A-L3 recovery separately records 6070965 input tokens; omitting it would make A's original main-run total an incomplete delivery cost. One capacity-error call in B-H1 has missing usage, making B's total a lower bound. The 35 independent final evaluations involve 75 evaluation identities plus 5 same-session format continuations, or 80 dispatches. Two initially invalid formatted opinions are not valid mathematical votes. Repair, diagnostic, acceptance and engineering use remain separately accounted in the original results; complete dollar cost is unavailable. Cached input and reasoning output are subsets and are not added twice.

Sources: [uniform endpoints](#), [usage and original acceptance](#), and [run index](#).

The following rows give recorded input and output tokens for each original second-batch main run; A-L3 recovery is separate. Cached and reasoning subsets and individual review records remain available in the original RESULTS and RUN_INDEX.

Item	Input tokens	Output tokens	Coverage
A-L1	4346485	34139	Complete
B-L1	3442740	33414	Complete
C-L1	2364969	36975	Complete
B-L2	3170506	33131	Complete
C-L2	3557210	38072	Complete
A-L2	5204890	35823	Complete
C-L3	6642131	52291	Complete
A-L3	6979609	55716	Complete

Continued on next page

Item	Input tokens	Output tokens	Coverage
B-L3	3290880	28458	Complete
A-L4	13021625	92806	Complete
B-L4	3743245	37467	Complete
C-L4	3549486	32717	Complete
B-L5	3683258	42041	Complete
C-L5	7443795	68637	Complete
A-L5	8362506	70600	Complete
C-L6	4908403	53253	Complete
A-L6	7076995	52459	Complete
B-L6	7124870	78857	Complete
A-L7	5153656	48426	Complete
B-L7	4417544	44101	Complete
C-L7	4048126	42890	Complete
B-M1	6772229	44414	Complete
C-M1	7410304	48813	Complete
A-M1	17107946	102084	Complete
C-M2	7572446	49372	Complete
A-M2	18589232	111849	Complete
B-M2	14785583	74322	Complete
A-M3	12650210	103408	Complete
B-M3	5051196	43196	Complete
C-M3	3602848	38797	Complete
B-H1	42122723	215446	Lower bound
C-H1	36352777	141898	Complete
A-H1	45325798	261059	Complete

Separate scope	Known input tokens	Known output tokens
A-L3 recovery	6070965	38740
12 diagnostic reviews	3578867	91652
2 repairs	2009465	25124
35 independent acceptance cases	22603252	376100

The independent-acceptance subtotal is a known lower bound: two B-L2 dispatches lack usage. They are separate from the missing-usage call in the B-H1 main run. These subtotals are not conflated with the original main-run stage, and complete monetary costs are unavailable.

Q.4 Fixed old L3 diagnostics and completed repairs

Both frozen task sheets allowed an eventually integrable tail formulation while requiring the nonnegative-function, integral-convergence and almost-everywhere-convergence argument controlling the negative part or minimum. The old diagnostic tasks retained that wording; the compulsory tail clause added to second-batch main tasks was not inserted into them.

The relevant original A instruction asked reviewers to verify that `hg_nonneg/hg_meas` had been removed from the public interface and that the almost-everywhere derivations were implemented. B asked for checks of Scheffe, the negative part, density, total variation and normal boundaries, including premise transfer, forbidden dependencies and uncovered requirements. A shared outer instruction required fresh independent read-only sessions, treated historical session identities only as provenance, and withheld the final reply of the review being replayed.

The complete added sentence, translated from the identical Chinese instruction for both sources, was: “Please explicitly check whether the public conclusion covers eventually integrable sequences and whether the conclusion for the original sequence is recovered, and explain how this scope check affects your review judgment.” Apart from that sentence, both conditions for a source retained the candidate, supplied material and outer instructions. The intervention explicitly made a previously optionally interpreted scope a checking requirement, without specifying a vote or a proof. The observed differences involve both attention to scope and interpretation of acceptance.

Exact original text: A original, A with added sentence, B original, B with added sentence, and old task sheet. The [input index](#) records checksums for all 12 actual prompts; the [complete replies](#) preserve individual reasons.

Old candidate	Instruction	Repeat	Discussed gap	Required repair	Verdict
A	original	1	False	False	Pass
A	original	2	False	False	Pass
A	original	3	False	False	Pass
A	explicit	1	True	True	Fail
A	explicit	2	True	True	Fail
A	explicit	3	True	True	Fail
B	original	1	True	True	Fail
B	original	2	True	False	Pass
B	original	3	True	False	Pass
B	explicit	1	True	True	Fail
B	explicit	2	True	True	Fail
B	explicit	3	True	True	Fail

Each row is a fresh Sol medium review session on a fixed candidate under one instruction. None of A’s original-instruction replies raises the tail gap. All three B replies discuss it, but only the first makes repair mandatory. Each candidate receives three failures under the explicit requirement. There are two code objects and two conditions per object, not twelve different mathematical tasks; separate sessions do not guarantee independent errors.

Repair source	Seconds	Technical check	Final review 1	Final review 2	Outcome
A	436.820	pass	pass	pass	pass
B	418.617	pass	pass	pass	pass

Fresh Sol medium repair sessions each receive a 7200-second allowance and the frozen old candidate. Both preserve the negative-part and dominated-convergence core, use integral linearity for sufficiently large indices, and transfer the limit by eventual equality. A supplies an existential cutoff interface; B supplies eventual integrability and an all-term compatibility theorem, with corresponding density and normal caller changes. Repair inputs are frozen before diagnostic results return: this tests repair under explicit requirements, not the causal effect of forwarding a particular diagnostic reply. All six B diagnostic sessions, repetitions 1, 2 and 3 under each instruction, reported missing dependency caches and could not rebuild within their sessions. This limitation occurs in both conditions and is not a candidate compilation failure. A’s six replies cite existing bound builds; those citations are not six fresh builds. Both later repair builds have separate success records. See the [diagnostic analysis](#) and [final results](#).

Q.5 Code evidence for the three disputed outcomes

A-M1’s existence theorem produces a stopped-value representative; its final theorem supplies all three expectation cases for any function with the correct representation certificate. The caller combines those results, using the general finite-integral argument already present inside the delivered increment proof for almost-sure finiteness in the third case. The C-L3 caller applies the general order-limit theorem to nonnegative terms and convergence, and obtains the required almost-everywhere strong measurability from integrability. It directly invokes the delivered eventual-integrability Scheffe theorem without adding a negative-part, dominated-convergence or tail argument. Both checks depend only on Lean’s usual propositional extensionality, choice and quotient axioms.

Files and outputs: [M1 caller](#), [L3 caller](#), [compile records](#), and [52 source-file matches](#). Original compiled-source volumes are mounted read-only; candidates are unchanged. B-M2's full-sequence proof and successful technical check already exist in the original delivery. Feedback and the unchanged code are located in the [run analysis](#) and [M1/M2 interpretation](#). The latter preserves the original evaluators' interpretation; this edition's common policy supersedes its characterization of the interface dispute as a mathematical gap.

Q.6 Checks and materials for Revision 23

Revision 23 records that Section 1 is unchanged and Section 2 receives only additions needed for the design and evaluation. Displayed mathematics, numeric tables, links and PDF layout are checked across both languages, with the validation record retained. External review is an additional editorial check, not mathematical re-evaluation of 66 items or certification of current production state.

R Detailed cases and historical analysis

The arguments from Chapters 4–6 of Revision 23 are retained here as R.1–R.3, with section numbers and cross-references updated for their new location.

R.1 How acceptance criteria changed the experimental verdicts

This chapter explains how task requirements determine outcome judgments. Appendix R.1.1–R.1.4 examine first-batch library calls, delivery scope and corrections to task assumptions. Appendix R.1.5 gives the mathematical grounds for three second-batch adjustments, and Appendix R.1.6 reports the old L3 diagnostics and repairs.

R.1.1 Corresponding library calls received inconsistent verdicts (L2)

L2 comprises textbook Theorems 11.4 and 11.5: variance additivity for pairwise uncorrelated random variables, followed by the related weak law of large numbers. Original common evaluation allowed A to call a basic expansion but rejected C for a corresponding library call. Checking the library relationship and applying one requirement consistently resulted in passes for A, B and C. [21]

Both calls provide the variance-covariance expansion that holds for generally correlated variables:

$$\text{Var}\left(\sum_i X_i\right) = \sum_i \sum_j \text{Cov}(X_i, X_j).$$

The candidates still had to use the given uncorrelatedness to eliminate cross terms, then complete the means, variance bound, Chebyshev inequality and limit required for the weak law. A's `variance_sum'` and C's `variance_fun_sum'` did not directly supply the whole task group; the latter is obtained from the former by a library conversion. This relationship supports the corrected verdict.

Current acceptance permits such proved basic expansions. A supplementary check asks whether the candidate also derives the expansion itself from the variance definition and linearity of expectation. B completed that additional work; it does not revoke A's or C's pass under the adopted criterion. The two checks were distinguished after results were observed. Their original G and S labels and obligation-level evidence remain in [Appendix G.15](#). [21]

This case shows why rejections from common evaluation also require scrutiny. A request for further work is useful feedback only if its requirement belongs to the task and applies consistently to both candidates.

R.1.2 A local repair can leave the task's scope incomplete (L3)

First-batch A and B complete the main Scheffe argument but require every term to be integrable. The adopted task scope requires integrability only eventually. The difference concerns actual inputs: on $(0, 1]$, take the first term to be $1/x$ and all later terms and the limit to be zero. The tail satisfies the requirement although the first term is not integrable. Covering this scope requires using the integral argument on the tail and recovering the original sequence's limit. [23]

The original task described the tail form as permitted; subsequent common acceptance made that scope explicit. First-batch C supplies the corresponding argument, whereas the original A and B deliveries do not. This assesses the deliveries under the common criterion while preserving the instructions available during solving. No additional extended-integral wrapper is required; Appendix G.17 gives the formal scope distinctions. [13,23,39]

A’s review had already prompted a real change: an early theorem assumed nonnegativity and measurability of the limit, and a later proof derived the needed properties. However, the route check treated the tail form as optional, and the general re-review accepted the all-term interface. Its density caller already had integrable terms, so successful use did not test the broader scope. B’s requested reviewer also recognized the tail version but treated it as an optional extension; the author then submitted. [29,39,40]

The acceptance program checks that an opinion matches the current code and materials; it does not judge mathematical scope again. A review that retains the narrower interpretation can therefore pass through a correct version check. This explains both the scope omission and why the established local repair did not complete the group. Appendix R.1.6 reports the subsequent explicit-requirement test and two successful repairs.

R.1.3 Cauchy averages (H1): why did B change from fail to pass?

B-H1 currently passes; its earlier failure remains traceable. H1 contains five connected duties, including an application to Cauchy sample averages. The dispute concerned whether that application required the standard distribution or the full family with arbitrary location and scale. The original adjudication assessed both scopes separately rather than declaring B’s proof generally incorrect. [23]

Check applied to B	Recorded assessment	Effect on current completion
Textbook standard-Cauchy core and the other four targets	Pass; B’s zero-location, arbitrary-positive-scale result includes the standard case	H1 counts as pass
Extension to arbitrary common location and positive scale	Arbitrary-location extension incomplete	Extension scope reported separately; not a core failure

The acceptance rule settled on 12 September requires all five targets and the inversion and uniqueness proof work, using the textbook-supported standard-Cauchy interpretation as the core. A delivers the standard distribution, B the zero-location family with arbitrary positive scale, and C the arbitrary-location and positive-scale family. All three therefore pass the core while differing in extension scope. This is the explicitly adopted acceptance choice; requiring arbitrary location instead would leave B’s existing delivery insufficient, but would change the current criterion. Appendix O.5 preserves both assessments and the adopted rule.

Two evaluation-processing issues compounded the scope dispute. B’s adjudication already accepted the standard core mathematically, but quotation escaping caused a peripheral check to reject the result. Elsewhere, an anonymous-code-only input omitted original submission notes that were relevant to a documentation judgment. The later correction restored those actual notes and resolved determinable quotation correspondence; it did not add a proof or write a missing explanation on the author’s behalf. These are defects in evaluation input and processing, not solver-side mathematical repairs. [15,23]; Appendix G.13.1

R.1.4 Continuous mapping (L7): wrong advice and insufficient assumptions

The continuous-mapping dispute contains two different mathematical problems. First, a reviewer recommended replacing ambient continuity at points of a set with continuity relative to that set. Take a one-point probability space and

$$S = \{0\}, \quad V = 0, \quad V_n = \frac{1}{n+1}, \quad f(x) = \begin{cases} 0, & x = 0, \\ 1, & x \neq 0. \end{cases}$$

The restriction of f to S is continuous, but $f(V_n) = 1$ does not converge to $f(V) = 0$. The approximating variables approach the set from outside it. This counterexample refutes that particular interface recommendation; it does not settle all obligations of the original candidate. [22]; Appendix G.16

Second, even retaining ambient continuity at every point of S does not guarantee measurability of every composite output. On the Lebesgue probability space $[1, 2]$, take a non-Lebesgue-measurable subset E , set $f = \mathbf{1}_E$ on the real line and $S = \{0\}$, and let $V_1(\omega) = \omega$ while $V_n = V = 0$ for $n \geq 2$. The function is zero near 0 and continuous there; all input variables are measurable and converge pointwise. Yet $f(V_1)$ is not measurable. A single initial term is enough to violate the demanded output property without changing convergence. These are preserved written counterexamples, not newly executed Lean checks. [22]

Current acceptance therefore permits reasonable, disclosed and fully proved repairs, with Borel measurability of the mapping and appropriate continuity at the limit. B/C complete such repairs. B's additional measurability condition on S is stronger than C's event-based formulation, but the criterion does not demand a unique weakest repair. C additionally preserves a numerical convergence core without the mapping-measurability premise. An external evaluator of A's repair also identified insufficient conditions; the evidence does not support exclusive discovery by C. [22]

The original selected A-L7 delivery and the earlier separate repair remain incomplete as historical artifacts. The 13 September continuation, however, completes the repaired task, including composite measurability. Its final proof already existed in the sealed working candidate; the new work clarifies the correction and completes review and delivery. Appendix R.2.6 describes that result and its changed conditions. [39]

R.1.5 Equivalent deliveries and three original rejections

The three second-batch rejections require different explanations. Reassessment uses the same mathematical goals. The evaluator may combine delivered results and permitted library facts or check equivalent premises, but may not supply an omitted task-specific proof. [42]

Item	Delivered mathematics	Common outcome assessment
A-M1	Existence of a stopped-value representative and expectation results for three conditions	Complete. An independent caller combines the results on the original inputs and checks all three cases
B-M2	Fourth-moment estimates and Borel-Cantelli give the full-sequence conclusion directly	Complete. Failure to use the stipulated subsequence-first route is recorded separately
C-L3	Eventual-integrability Scheffe theorem and its downstream applications	Complete. Nonnegativity follows from the termwise bounds and convergence; the required almost-everywhere strong measurability follows from integrability; the caller check passes

A-M1 supplies the representative's existence theorem rather than requiring the caller to prove optional stopping. C-L3's additional parameters follow on the same inputs from general library facts and exclude no valid inputs. Both independent callers pass in the original toolchain, with 52 candidate and upstream source files matched to their bindings. These files are evaluation witnesses, not part of the original authors' submissions.

The first-batch L3 gap is different: eventual integrability does not imply integrability at every index. Applying its narrower theorem requires constructing a tail input and recovering the original conclusion. The evaluator does not add that task-specific extension. The repair authors in Appendix R.1.6 actually completed it.

This common outcome assessment is retrospective. All 66 deliveries are reconciled against the same task requirements; applicable prior reviews and build evidence are retained, and disputed cases receive additional checks. Appendix Q.2 and Q.5 preserve original decisions, reasons for changes and route compliance. The procedure recognizes equivalent usable results while retaining substantive scope omissions.

R.1.6 Fixed old L3 candidates: noticing an omission and requiring its repair

All twelve fresh reviews have returned. Close reading distinguishes explicit discussion of the omitted tail scope, a mandatory repair and the final pass verdict. Each row contains three reviews of one source under one condition. The

intervention adds only the scope-checking sentence; [Appendix Q.4](#) gives the inputs and individual reasons. These are diagnostic judgments of old candidates, not acceptance votes for second-batch main runs. [43]

Fixed source and instruction condition	Recognized tail omission	Required repair	Final pass
A, original instruction	0/3	0/3	3/3
A, added scope check	3/3	3/3	0/3
B, original instruction	3/3	1/3	2/3
B, added scope check	3/3	3/3	0/3

For A, the distinction concerns recognition of the missing scope. All three original-instruction replies accept all-term integrability as the formal encoding of finite real integrals. They focus on the repaired nonnegativity and measurability premises and a successful density caller. All three added-instruction replies acknowledge those local achievements but identify that all-term integrability excludes sequences with finitely many nonintegrable initial terms. A conclusion written for the original sequence covers the required scope only if its inputs admit that scope. They require a tail interface and recovery of the original limit. [43]

For B, the distinction more often concerns whether a recognized issue blocks acceptance. All three original-instruction replies discuss eventual integrability. The first requires repair and fails; the other two treat the extension as optional greater generality and pass. All three added-instruction replies require repair. Pass counts alone would conceal the difference between knowing how to extend a theorem and judging that extension necessary. Original review materials differ between sources, and each condition has only three repetitions. The results support an interpretation involving attention and acceptance judgment in these cases, not a project-wide review success rate or an isolated model effect. [43]

Directed repair adds code evidence. Both fresh repair runs weaken the public assumption to eventual integrability, use integral linearity only on the tail and transfer the limit by eventual equality to recover the original sequence. The negative-part and dominated-convergence core remains. A uses an existential cutoff; B uses a filter-based eventual assumption and retains an all-term compatibility theorem. Downstream changes adapt callers; the existing density and Gaussian branches were not newly invented in these repairs. Both repairs pass builds and final common acceptance, taking 7.28 and 6.98 minutes respectively. They demonstrate repair of these two old deliveries under explicit requirements and remain separate from first-batch completion counts. Repair instructions were frozen separately before diagnostic results returned; success cannot be attributed to direct adoption of a particular diagnostic reply. [Appendix Q.4](#) locates the diffs and builds. [43]

R.2 Experimental trajectories: organization, repair and delivery

This chapter asks what authors and coordinators actually did, which feedback changed code, which feedback supported submission, and which execution faults delayed existing work. The comparison can inform project maintenance only when an outcome is connected to the work and decisions that produced it. These trajectories examine the second evaluation question: how review, assistance and control affect proof work and delivery. A supplies observations of the prescribed workflow; B/C supply observations of alternative organizations, not evidence that their arrangements are deployed production features.

The cases are organized around decisions that change a task’s course. A-L3 asks how review first produces a local repair and then mistakenly accepts complete scope ([Appendix R.1.2](#)). A-L7 asks why an existing proof does not reach the original selected delivery and what the continuation actually adds ([Appendix R.2.6](#)). C-L1 examines submission when the coordinator receives check errors or a readable pass. These sequences are compared with cases of demonstrated code repair and adoption. Each account uses its actual version and information state; changes between development runs are not treated as controlled interventions.

The established cases in this section concern the first batch and its earlier development runs. [Appendix R.2.8](#) adds second-batch feedback and delivery comparisons; historical material in [Appendix R.3](#) retains its own observation dates.

R.2.1 Innovation and martingales (L1): feedback available at submission

L1 asks the agent to build innovations from an observed process and prove that their partial sums form a martingale. With $X_0 = 0$ and $\mathcal{F}_n$ representing information through time n , the construction uses

$$Y_{n+1} = X_{n+1} - \mathbb{E}[X_{n+1} \mid \mathcal{F}_n], \quad S_n = \sum_{k=1}^n Y_k.$$

The required proof establishes integrability and adaptedness, derives zero conditional innovation mean, and then obtains $\mathbb{E}[S_{n+1} \mid \mathcal{F}_n] = S_n$ almost everywhere. A uses Mathlib’s natural filtration and a general zero-conditional-increment martingale criterion. B/C use the supplied one-indexed observation-history filtration and construct the local martingale definition’s fields. These routes perform corresponding mathematical work; use of the general criterion does not by itself omit the task. [27]; Appendix G.5–G.10

A’s difficulty after mathematical implementation is delivery of the review, not absence of passing opinions. Its initial prompt contains unrelated task identifiers, causing a detour. The author then reads the relevant library sources, works around missing file-writing tools and repairs concrete Lean interface errors. After candidate four builds, the review sequence becomes the major observed blockage. Seven general-review requests include one rejected at entry, five sessions with completion receipts and passing opinions, and a final started session without a completion receipt. The passing opinions do not reach a readable author-side result file through the normal path. The author searches, waits and retries, although the analyst can later read those opinions elsewhere. [27]

The outer execution record reports recovery of an existing review JSON into the required result location, and the controller subsequently observes a pass. The retained JSON matches the referenced reviewer’s output, although the recovery receipt names a different session. That attribution conflict remains unresolved; the observed progress is restoration of the acceptance path, not five successive improvements to the proof. [27,33]; Appendix G.6

Initial C encounters a related delivery fault but reaches a different submission state. The coordinator reads the writer’s candidate, builds it, and requests general and mathematical checks. Both checking models run and produce opinions, but readonly-filesystem export failures return errors rather than the opinions. The coordinator submits while explicitly disclosing that independent checking has not been obtained. Its later common dual evaluation passes the same candidate. That later evidence strengthens assessment of the artifact, not the coordinator’s information at its earlier decision. [27]; Appendix G.9.2

L1 observation	Mathematics at the selected point	Feedback actually available	Consequence
Initial A	Natural-filtration proof and successful candidate build	Several passing opinions exist but normal author-side delivery fails	Waiting and retries; host recovery before recorded acceptance
B: early trial	One-indexed filtration and local martingale proof	Helper fails; mathematical-review encoding fails; later general opinion is readable	Author continues and submits after readable checking
Initial C	Writer completes and coordinator builds the candidate	Two checking opinions exist but only export errors return	Coordinator submits with the checking gap disclosed
B: main-panel run	Completed proof with an unnecessarily broad time-index interface	Readable nonblocking advice identifies derivable time-zero assumptions	Author revises the public interface and checks the new code
C: main-panel run	Candidate already complete before review	Complete passing opinion is returned and acknowledged	Same candidate submitted without a mathematical edit

The main-panel B interface change is concrete. The earlier declaration requests measurability and integrability at every natural-number time, whereas the source only requires positive-time premises and gives $X_0 = 0$. B derives the time-zero properties internally and changes the final interface, while keeping helper lemmas with all-time premises.

A changed public theorem and corresponding proof show adoption of a specific piece of advice. Main-panel C instead uses a readable passing opinion to authorize submission of unchanged code. One is interface repair; the other is a checking result entering a decision. [20,27]

These runs establish a specific interaction: in A’s observed process, the need to apply a readable review makes a return-delivery failure block acceptance of existing mathematics; initial C’s submission discretion permits delivery with an explicitly unresolved checking state. Neither more opinions in inaccessible locations nor eventual endpoint agreement removes this operational difference. The later B/C versions show readable feedback supporting different useful actions, not a single universal “feedback repaired the proof” event.

In the main-panel C run, the readable opinion is a general review, with no mathematical-consultation call. The coordinator explicitly acknowledges the passing build and review, then submits the unchanged candidate. This is evidence of feedback entering a decision, not of a second check or a review-induced mathematical edit. [20]

R.2.2 Fourth-moment strong law (A-M2): move the missing bound into the proof

This case is the main-panel A-M2 run. M2 asks for the strong law for independent variables with common mean m and a uniform raw fourth-moment bound. The preparation summary refers to centered moments, while the supplied textbook gives raw moments. The public theorem must meet the source assumption and derive the centered bound internally. This difference is small to state but changes who is responsible for a necessary step. [25]

The author searches strong-law, independence and integral-product interfaces, reads implementations, and tests exact types in small files. It develops a finite-sum fourth-moment expansion, cancels terms using independence, derives a tail-probability bound and applies Borel-Cantelli. The first seven of eight initial build checks fail; the eighth passes. The version-eight candidate nevertheless asks the caller to supply a centered fourth-moment bound for $X_i - m$. [25]; Appendix G.14.1

A general reviewer identifies the raw-versus-centered gap. Subsequent code proves

$$(x - m)^4 \leq 8(x^4 + m^4), \quad \mathbb{E}(X_i - m)^4 \leq 8(c + m^4) \quad \text{when} \quad \mathbb{E}X_i^4 \leq c.$$

Version eleven moves that conversion inside the proof, accepts the raw-moment premise in its public declaration, and invokes the already proved centered-moment result as a helper. Version-bound re-review passes, the final source matches that version, and subsequent common evaluation passes. The chain includes an identifiable criticism, a change to the calling contract, a new derivation and verification of the changed candidate. It is stronger evidence of mathematical repair than merely observing review before acceptance. [25]

The build trajectory and semantic trajectory have different meanings. Some build failures concern types, function representations and integrability interfaces; their outer shell statuses are zero even though the output reports failure. The later semantic gap survives a successful build because Lean is checking the weaker statement actually written. A source comparison reveals the extra premise, and a code comparison reveals that it was genuinely removed rather than renamed. [25]

The first-turn prompt also prohibits assistance and requires a return after review preparation. The absence of help or that return cannot be interpreted as a voluntary decision that no further work was needed. Textbook guidance, author implementation, tool diagnostics, source clarification and review all participate in the recorded chain. The evidence establishes the repair without assigning its entire contribution to one reviewer or one autonomous decision.

R.2.3 Inversion and uniqueness (H1): prove support, deliver it and use it

H1 combines the bound $|e^{ix} - 1| \leq |x|$, characteristic-function inversion, uniqueness via inversion, a Cauchy-average application and an integer-valued criterion. This permits inspection of whether an intermediate result actually enters later proofs. The detailed adoption sequence below concerns A’s same-root completion; the endpoint panel also records B/C’s complete deliveries. [17,23]

The author first requests the whole inversion proof but restricts that support request to read-only work. The response locates a Dirichlet-integral gap without a usable patch. The author then asks for the integral’s limit and uniform bound, receives built support and incorporates it into the draft. A later helper completes the remaining inversion proof, but export fails. Recovery through the outer execution layer allows the author to read it, apply it, build and request independent internal review. A productive mathematical call and a failed delivery stage occur in the same sequence. [17]

The delivered inversion argument contains finite integral interchange, different pointwise limits of an oscillatory kernel, dominated convergence and endpoint half-masses:

$$\lim_{T \rightarrow \infty} \frac{1}{2\pi} \int_{-T}^T \frac{e^{-iat} - e^{-ibt}}{it} \phi_{\mu}(t) dt = \mu((a, b)) + \frac{1}{2}\mu(\{a\}) + \frac{1}{2}\mu(\{b\}), \quad a < b.$$

Uniqueness then calls this theorem to compare interval probabilities at suitable endpoints and uses a measure-determination argument. The standard-Cauchy application derives the characteristic function, applies the independent-sum product and scaling formulas, and invokes the newly completed uniqueness theorem. Those uses connect actual conclusions, not just files. [17]; Appendix G.13

Result or dependency	What the later proof does	Meaning for reuse
Inversion theorem	Uniqueness derives equal interval probabilities and equality in distribution	The earlier theorem is used in the mathematical argument
Uniqueness theorem	Cauchy-average application passes from equal characteristic functions to equal distributions	A second actual theorem use
Uniqueness file imported by the integer-valued criterion	Proof instead uses the real part of a unit-modulus exponential attaining its upper bound almost everywhere	An import is present, but this theorem is not invoked

The integer-valued result proves that $\phi_X(2\pi) = 1$ exactly when X is integer-valued almost surely. Its distinct proof route is useful negative evidence against counting every import as theorem reuse. Likewise, a uniqueness review initially passes but is rejected at application for an input-hash mismatch. The reviewer corrects the binding and explanation and the opinion is applied without a code change. That event repairs the evidence connection, not the inversion or uniqueness proof. [17]

The case does not show that smaller helper assignments are always superior. Request scope and permission changed together, and the broader remaining proof was also completed by a helper. B-M1 and A-M3 provide additional successful broad delegations. What H1 establishes is successful composition of analytical support, artifact delivery and downstream use under the recorded organization, including host recovery.

R.2.4 Stopping and representation (M1/M3): what broad delegation contributes

A, B and C all pass M1. This section selects B/C-M1 and A-M3 because these examined cases establish adoption of delegated work and distinguish later proof changes from documentation changes. A-M1 also passes, with a reconstructed time of approximately 136.89 minutes; see Section 3.2. Appendix O.5 records the present verification. B-M1 delegates both stopped-process and stopped-expectation tasks to a support author. The work is substantial: it must prove the stopped process is a martingale and cover the specified stopping-time cases. Under integrable stopping time and bounded increments, a bound such as

$$|X_{T \wedge n}| \leq |X_0| + cT$$

provides the integrable control needed to pass from finite stopping results to the limit. The implementation also handles measurability, integrability and a valid representative for potentially infinite stopping times. [18]; Appendix G.11

The helper produces a patch in an isolated workspace. The lead author reads it, applies the target changes, builds and requests mathematical review. The reconstructed target files match the final evaluated code. This is an established adoption chain for a whole-group implementation, not evidence that B is a single unaided author. The lead role contributes integration, tool recovery and checking even when the mathematical implementation is delegated. Its initial composite build command returns exit code zero while stderr says the command is missing; it locates the absolute executable and performs a genuine build. Counting only the first outer exit code would invent a successful validation. [18]

C-M1 also delegates the two targets and obtains a passing delivery, but the subsequent candidate change affects documentation rather than mathematics. The coordinator adds missing public-declaration and premise descriptions;

visible mathematical files remain unchanged and the internal reviews identify no corresponding proof defect. Two internal target checks reuse one session, so they are two calls, not two independent reviewers. The later common evaluation is separate. B/C have equal final pass labels and different contributions after the initial implementation. [18]

A-M3 supplies another broad-adoption case. The Skorokhod representation task constructs new variables on a common space using generalized inverse distribution functions and the same uniform input. It preserves marginal laws and proves almost-sure convergence, handling measurability, endpoints and exceptional levels. The helper's full-task implementation is actually adopted. A malformed confidence field later creates an inconclusive common-evaluation result and adjudication, without a new mathematical repair to the candidate. [28]; Appendix G.12

Together these cases oppose two simplistic explanations: successful help need not be a narrowly scoped lemma, and a new candidate version need not be a new proof. The correspondence between the delegated work, the returned artifact and the final code determines what was contributed.

R.2.5 Coupling and variance (A-L5/B-L2): produced code without established adoption

In the earlier A-L5 run, the author responsible for the whole task delegates the common minimum density, residual densities, joint measure, marginals and mismatch probability for maximal coupling. The helper produces a 534-line candidate and a passing build receipt. The parent receives paths and hashes for files on a machine outside its container, cannot access the text or patch in its own container, and continues local implementation. Helper and final parent code differ in naming and organization. Similar proof routes could arise from the same textbook and original request; they do not identify code adoption. [19]; Appendix G.1

This investigation changed the earlier handoff classification and the analysis framework's adoption criterion. It also distinguished an outer transfer diff from the inner patch that actually changes target files. An existing package, a checked patch and modified target code are separate observations. The helper's 554.094 seconds of process time remains part of incurred work even though adoption is not established. The evidence neither erases that cost nor proves the absence of every possible indirect influence. [19]

B-L2 stops at a different evidential point. Its support product is blocked by export permissions, then delivered through engineering recovery without another model call or container. The recovered artifact's existence does not establish that the author adopted it. A later read-only opinion concerns a byte-identical final candidate, and the author acknowledges receipt: this is evidence of checking an existing result before submission, not evidence that the recovered patch repaired the theorem. [19]; Appendix G.18

The difference matters for analysis as well as engineering. L5 establishes a concrete inaccessible-delivery problem. B-L2 establishes recovery but leaves adoption unconfirmed. The latter is not automatically a verified failure to adopt. A useful trajectory account identifies where observation stops instead of turning every missing link into either success or failure.

R.2.6 Vector convergence (A-L7): an external stop and a completed continuation

After the L7 Math Gate identifies interface and unresolved-obligation issues, the outer experiment execution imposes a broader instruction: no editing, assistance or new review, and the run is sealed. Approximately 3,668 seconds remain from its 7,200-second allowance. The stop is therefore neither recorded budget exhaustion nor an autonomous author decision to abandon the problem. The selected delivery lacks the continuous-mapping branch. A sealed working candidate and the selected delivery are different artifacts; the later finding of a proof in the former does not place it in the latter. [22,39]

On 13 September, the original author session resumes on copies of the sealed state, with explicit permission to use a reasonable, disclosed correction of the deficient source conditions. It completes in about 14.5 minutes, within the approximately 61 minutes remaining at sealing. Builds, internal mathematical checking and general review pass. Two fresh read-only Sol medium evaluator sessions agree that the two-target delivery is mathematically complete, the repair reasonable and its disclosure sufficient. Both separately record that it is not a literal implementation of the unrepaired original contract. [39]

The decisive code comparison shows that the final theorem signature and proof body were already present in the sealed candidate_v1. The continuation changes explanatory comments and completes internal review, disclosure

and delivery. It therefore demonstrates that the existing proof can be accepted under the explicit repaired-statement standard; it is not evidence that the whole proof was newly constructed during those 14.5 minutes. [39]

Material or stage	Recorded result	Interpretation
Original selected A-L7 delivery	Incomplete: continuous-mapping branch missing	Original main-panel entry
Earlier separately restarted repair	Incomplete: composite measurability missing	Earlier supplementary candidate; not the 13 September continuation
Sealed original working candidate, then resumed	Proof signature and body retained; explanation revised; complete repaired delivery passes	Successful continuation under explicit repair permission

The continuation also required Docker communication recovery and a changed execution-program version because the earlier installed binary was unavailable. The two common evaluations took about 6.7 and 6.0 minutes in parallel; they are separate from the author’s solving time. Five experimental model calls consumed 7,641,026 input tokens, including 7,054,720 cached tokens, and 52,712 output tokens. A result-processing error in one completed evaluation was repaired by a faithful offline structural conversion, without another model call or a changed opinion. Full records are indexed in Appendix O.2. The added solving time enters the cumulative measure in Section 3.2, while the original-round measure remains separate. [39]

R.2.7 Cross-case comparison: reconstructing outcomes as processes

Review can repair one problem while missing another. A-L3’s first review leads to internal derivations of two premises, while mathematical checking and general re-review still accept narrower scope. B’s chosen reviewer also clears that restriction. In M2, by comparison, review identifies the raw-to-centered-moment gap, code supplies the conversion, and re-review passes. Whether a scope alarm is raised and whether the author changes the declaration and proof explain more about the mechanism than the number of review calls. [25,29,39,40]

Checking contributes through information received before a decision. Early A-L1 waits and retries because opinions do not return normally, then continues acceptance after external recovery of the result file. Early C-L1 discloses missing requested opinions and submits; main-panel C-L1 submits the same candidate after receiving a general pass. Neither C review sequence produces a new mathematical edit. They establish different bases for submission; their eventual passes do not imply that the coordinator possessed the same evidence when deciding. [20,27]

Mathematical work, usable artifacts and completed delivery are distinct stages. H1’s helper proof enters subsequent theorems only after recovery, reading and integration; B-M1 adopts a helper’s complete group implementation. A-L7’s sealed candidate already contains the final proof body, and resuming the original author completes explanation, review and delivery. The continuation establishes that existing work can reach acceptance. It does not turn an external stop into author proof failure, or its duration into the cost of constructing the entire proof from scratch. [17,18,39]

These comparisons locate observable differences: scope judgment misses an obligation, the return path loses usable feedback, and an external instruction stops A-L7. Transport faults belong to the run’s environment and imposed restrictions to experiment execution; the agents’ decisions are assessed against the information available to them. The evidence does not decompose the independent causal contributions of the three configurations or establish that every repaired environment fault persists.

R.2.8 Second batch: what happened after feedback arrived

B-M2 receives an internal opinion explaining that its subsequence steps do not independently establish the transition to the full sequence. The final target file does not change after that opinion arrives. Its proof already obtains the strong law through full-sequence summable probabilities; the later subsequence and remainder statements depend on that full conclusion. This establishes noncompliance with the stipulated route while preserving the completed mathematics. [42,44]

A-M1’s internal review accepts the stopped-value representative and its certificate as a reasonable interface, whereas post-submission evaluation rejects the absence of a single overall wrapper. C-L3 retains a limit-nonnegativity param-

eter although its main proof already supports eventual integrability. These cases require the evaluator to distinguish a missing mathematical argument from a way of using established results. The independent callers in Appendix R.1.5 make that distinction checkable.

All three configurations also deliver second-batch H1 and M3. H1 includes all five targets and the analysis supporting inversion and uniqueness; M3 includes the common probability space, quantile distributions and almost-sure convergence. Together with the first-batch adoption cases, these successes motivate retaining both the mathematical achievements and the processes that produce them. [41,44]

R.3 Historical evidence: mathematical progress, review and maintenance

The project’s earlier reviews and corresponding code revisions provide the second body of evidence. They address questions that the eleven-group comparison does not settle: where the mathematical work accumulated, whether accepted statements could be used as intended, how changes reached dependent callers, and whether the archive supports a trustworthy account of progress. Their sources and sampling scope differ from those of the experimental panel; these records are neither additional trials nor an explanation of its completion counts. [4-11,32]

Appendix R.3.1–R.3.3 reconstruct source-facing proof work, assembly, caller migration and the conditions of accepted theorems. Appendix R.3.4–R.3.6 then ask what can be measured across review records after judgment identity, object binding, target comparability and follow-up selection are checked. Appendix R.3.7 examines saved code directly and tests whether a more elaborate evaluation method adds information for the chosen attributes. The historical cases explain specific changes; their mathematical detail is not treated as representative coverage of every relationship in the statistical analysis. [10,11,32]

R.3.1 Removing an axiom is not the same as completing the missing derivation

The normalized-Gamma-to-Dirichlet history contains two different shortcuts. An early proof uses public axioms in `gamma_beta_bridge`. A later version removes that bridge but directly defines the normalized Gamma density as the desired Dirichlet formula, making an equality available by simplification. The required derivation from the probability model is still missing. Subsequent work supplies the product distribution, normalization map, simplex support, projected density and change of variables. The improvement is the new mathematics connecting the model to the formula, not merely the absence of an axiom declaration. [4]; Appendix L.2.1

This case makes the interface distinction in Section 1.5 concrete. A proved conversion between two definitions can be legitimate support. A name standing for unproved mathematics, or a definition chosen to make the target tautological, does not serve the same purpose. Axiom checks and source correspondence address different failure modes.

The support later has two actual consumers, `prob_1_4` and `ex_1_2_2`. Shared density identification does not eliminate each task’s own statement, coordinate-chart work or Beta branch. A later change adding an already proved density conclusion as a fourth conjunct of the named parent theorem changes the public promise’s location. That is another kind of progress, different from initially establishing the density and from extracting it into shared files. [4]; Appendix B.4

R.3.2 Partial proofs, global assembly and downstream migration are separate work

In the moment-generating-function task `prob_14_8`, the route extends moment-generating functions to a complex strip, obtains tightness and analytic bounds, extracts limits and identifies the characteristic-function limit. The history contains genuine local results before the whole argument closes. Math Gate reviews repeatedly issued `stop` while recognizing these local results and identifying an assembly gap: limits on different compact pieces must be restrictions of a single global function. Constructing that function and proving restriction compatibility closes work that the mere list of completed local lemmas does not. [6]; Appendix L.2.3

The complete single-file proof precedes its later split into support modules. The consumer `prob_14_10` had already established approximation and error-control work and built the inputs for its upstream theorem; its eventual local change is only one call selecting the completed upstream conclusion. The upstream versions, however, change from private-axiom-dependent to internally proved. Small local diffs and multiple support files therefore give misleading accounts of where and when the mathematical work occurred. [6]; Appendix B.10

A total-variation repair exposes a different dependency. `def_8_5` keeps its formula but adds the source’s probability-measure requirement. Callers must provide that evidence where the expression is formed. In `def_10_5`, replacing “the measures are probability measures and the distances converge” by “if they are probability measures, the distances converge” would permit vacuous truth outside the intended domain. The repair instead passes the needed evidence through a dependent witness and local instance. Five substantive reviews involve only two main-file contents because caller changes, failed registration and expanded checks account for other events. [7]; Appendix L.2.4

Historically, some proof obligations, labeled **OBL**, became subtasks with their own candidates, reviews and propagation of completion status. Later rules absorbed them into parent or support files and required renewed assessment of the assembled parent. Decomposition still produces useful mathematics, but its pieces must be connected to the final public theorem and its callers. Sections 5 and 6 of the main report examine that connection. [5]

These histories make the maintenance responsibilities in Section 1.5 concrete: a local proof, its sound upstream basis, the source-facing public theorem and migrated callers can reach completion at different times. They also explain the later rule requiring the assembled parent to be checked after historical OBL absorption. That rule preserves useful decomposition while refusing to substitute a collection of old child passes for a fresh whole-task judgment. [5]

The division of responsibility developed from an earlier pipeline with prescribed planning, retrieval and candidate-selection stages, already including reranking, error feedback and experience injection. Later package-based editing gave the author more discretion while retaining explicit acceptance controls. Appendix L.1 traces that development; it is separate from the A/B/C comparison. [3]

R.3.3 A passing conditional theorem can still have no admissible instance

Two dependent tasks are involved: `thm_14_8` proves a general central limit theorem, and `ex_14_4_3` applies it to the coupon problem. The June artifacts are conditional proofs of the example; the August candidate belongs to the upstream general theorem. They are not successive versions of one file. We examine the example’s premises, the upstream proof and the connection required for application separately. [8]

The saved coupon example models waiting for new coupon types by independent positive-integer geometric variables. For N equally likely types and i already collected, the success probability is $p_{N,i} = (N - i)/N$. The analyzed example collects $m_N = \lfloor (N + 1)/2 \rfloor$ types. Its model sums the stage waiting times, with mean $\sum_{i < m_N} 1/p_{N,i}$ and variance $\sum_{i < m_N} (1 - p_{N,i})/p_{N,i}^2$. [8]

Four accepted June snapshots expose a contradiction before any limit theorem is applied. They use $N = n + 2$. The first row—the finite collection of waiting stages for one value of N —has $N = 2$, needs only one new type and contains a geometric stage of success probability one, hence zero variance. The same configuration requires a strictly positive normalization scale. A saved Lean probe derives a contradiction from the zero-variance identity and scale positivity using only the permitted foundational axioms. The external upstream central limit theorem premise is not needed for that contradiction. [8]; Appendix L.3

An August failed candidate for the upstream theorem already contains substantial core mathematics: it proves the Lindeberg and Lyapunov branches internally in a canonical product model, a product probability space whose coordinates have the prescribed distributions and independence. The remaining demand is to connect concrete independent rows to that model. Once that connection is provided, the upstream general theorem passes while the coupon example still needs the textbook normalization. With

$$Z_N = \frac{T_N - \mathbb{E}T_N}{\sqrt{\text{Var}(T_N)}}, \quad \frac{T_N - N \log 2}{\sqrt{N(1 - \log 2)}} = a_N Z_N + b_N,$$

the final step requires $a_N \rightarrow 1$, $b_N \rightarrow 0$ and distributional transfer under the changing affine map. Later work establishes bounded mean error and the variance limit and supplies that transfer. The core theorem, distributional connection and asymptotic normalization are distinct achievements. [8]

This is a concrete reason not to rank mathematical content by historical pass/fail. It is also a bounded result: the saved independent-stage model is analyzed; equivalence to the original sequential coupon-drawing stopping time was not separately delivered. That unproved connection is not silently supplied by the familiar interpretation of geometric waiting times.

Target revision must be distinguished from ordinary proof repair elsewhere too. In `prob_14_7`, independent approximating pairs and marginal convergence do not specify the displayed limits’ coupling: standard-normal independent pairs have sum variance two, but choosing the marginal limits as $X = Y = Z$ gives variance four. Adding limit inde-

pendence changes the accepted target. In prob_14_11, $m/N \rightarrow 1/2$ does not control centering error at the required square-root scale: $N = k^4$ and $m = N/2 + k^3$ preserve the proportion limit while the standardized mean error diverges. The accepted exactly centered result answers a revised question. [Appendix L.4](#) retains both arguments and the corresponding source decisions. [9]

R.3.4 Recover distinct judgments before measuring change

The preceding cases compare mathematical work with recorded judgments. Extending that comparison across the archive first requires identifying distinct opinions: one opinion can be stored in raw and normalized forms, a program can generate a failure record without a reviewer, and several genuine opinions can assess one candidate. The reconstruction distinguishes these cases using result and input digests, the recorded candidate and context, and the pointers connecting review to application. Similar wording alone is insufficient to merge records. [Appendix C.1](#) gives the rules and processing counts. [10]

This changes what the data can mean. A failed application is not automatically a mathematical rejection; saving one opinion twice does not supply two judgments. The reconstruction preserves separate branches for record identity and mathematical comparability, so the resulting quantities are not successive totals of the same kind of object. [10]

Checking only the main file also misses real changes. In 98 historical relations, that file is unchanged while other jointly supplied review objects differ; thirteen of those relations change from fail to pass. For thm_11_8, a later upstream source is recovered but the earlier one is not, establishing changed evidence without determining the mathematical difference. For thm_9_7, the registered early candidate differs from the candidate actually judged, so the relation is excluded from the bound comparison. These examples explain why a stable main file alone cannot establish reviewer inconsistency on fixed evidence. [9,10]

R.3.5 Acceptance increases, but close cases explain only part of that trend

The analysis compares earlier and later judgments for the same task. It first requires that each judgment be bound to the intended candidate. A further subset requires affirmative evidence that the mathematical targets are comparable. These are different checks: an intact record can still concern a revised task. [Appendix C.3-C.4](#) describes the selection, including the exclusion identified in [Appendix R.3.4](#) and the treatment of changed or unresolved targets. [10]

Historical selection	Comparisons / tasks	Earlier $\rightarrow$ later pass labels	Interpretation
Broad directly bound comparison	461 / 248	116 $\rightarrow$ 324	Growth in recorded acceptance within the selected archive
Affirmative target and object qualification	128 / 79	28 $\rightarrow$ 45	Growth in the purposefully examined, comparability-qualified subset
Broad set intersecting the detailed anomaly queue	163 comparisons	32 $\rightarrow$ 33	Only one net additional acceptance lies in this overlap
Broad set outside that queue	298 comparisons	84 $\rightarrow$ 291	The other 207 net additional acceptances lie here

Acceptance still increases after target and object qualification. The second row is a positive descriptive result for the examined subset. It is not a random sample of the broad comparison, so the difference between their trends cannot be attributed solely to target changes. Relations can share endpoints, and tasks with more relations receive more weight in a relation-level summary. The appendix reports both relation-weighted and task-weighted changes with their definitions. [10]

Detailed examples do not explain most of the broad increase. The last two rows locate the overlap with a separate queue of close mathematical and record investigations. They show why the preceding case histories cannot stand in for equally detailed explanation of the entire trend. They do not show that acceptance growth outside that queue is

wrong. Together, the selections distinguish the observed trend, the part with established comparability and the part whose underlying changes have received close investigation. [Appendix C.4.2](#) supplies the constructions and [C.3](#) the transition table. [10,32]

R.3.6 A diminishing-returns interpretation fails a selection check

The follow-up analysis starts from fixed failing endpoints in saved task histories and asks what happens at successive revisions. It checks target comparability and object binding at each step. Some paths end because their saved fragment ends; many others leave the strict comparison because target comparability has not been established. Such an exit does not itself establish task failure or a changed mathematical target. [Appendix C.5](#) gives the origin and exit rules. [10]

A tempting pattern appears in the observed steps: 22/69 qualified first steps reach acceptance, versus 3/31 later steps. But later steps are contributed by paths that have not already passed and remain observable. The saved analysis checks the tasks producing those later steps: only fifteen tasks do so, and their nineteen first steps contain just one pass. The lower later-step proportion is therefore entangled with which tasks enter later follow-up. The comparison does not isolate the effect of making another revision. [10]

This check changes the interpretation of the apparent decline: later observations disproportionately come from tasks that were already less successful at the first step. Broader follow-up reaches more recorded acceptances from the same origins, but relaxing comparability answers a different question about reachable records. [Appendix C.5-C.9](#) retains the counts, risk sets and broader paths needed to examine both results.

R.3.7 Saved-code checks distinguish compilation, proof shortcuts and definition behavior

Fourteen frozen pairs were selected for code checks: six development pairs and eight further pairs chosen by a seeded, stratified rule after excluding previously exposed tasks. Pair identities were fixed before later eligibility and execution outcomes. This is not a representative benchmark, but it preserves failed and unavailable measurements instead of replacing inconvenient pairs. [11]; [Appendix D.1](#)

The two axiom rules inspect the named target and its transitive dependencies. The permissive rule rejects the placeholder-proof axiom `sorryAx`; the strict rule permits only `propext`, `Classical.choice`, `Quot.sound`, or no axioms. Passing the strict rule implies passing the permissive rule. These are two rules chosen for this code study, not earlier and later project policies; neither establishes fidelity to the textbook. [11]

Of the 28 files, seventeen compile and sixteen yield the required named declaration. All sixteen extracted targets pass both saved axiom policies; the extra compiling file lacks the requested declaration. Across the eight further pairs, each side has six compiling files, but `def_13_2` changes from failure to success while `prob_10_1` changes the other way. An unchanged aggregate count therefore hides different artifact-level transitions. The historical runner has no cross-endpoint type-equality gate, so successful extraction on both sides still does not establish a common mathematical target. [11]

`prob_10_1` gives a concrete semantic finding: an early theorem takes precisely its desired conclusion as a premise and returns it. The later source removes that defect, but the file fails in the saved build environment. The evidence supports removal of a direct shortcut, not certification of a complete repaired implementation. These are separate outcomes worth reporting together.

For `def_2_1`, the source convention calls a set countable when it is in bijection with all natural numbers. The earlier library predicate also accepts finite sets. The empty set satisfies the latter but cannot have a bijection with the naturals, since zero would have no preimage. Saved Lean witnesses check the early predicate, the later predicate and the written reference, including agreement of the later predicate with the reference for all sets. This makes a disputed interpretation testable as a formal proposition. The interpretation linking source text to the reference remains an AI-assisted judgment, not independent expert certification; the witnesses nevertheless establish precise behavior of the formal predicates. [11]; [Appendix D.5](#)

The same study asks whether evaluating both candidates under both rules adds information beyond a fair fixed-rule comparison. For the inspected axiom and direct-premise attributes, effective rules coincide or have known implications, and the additional cells provide no extra answers. Seven constructed proof scenarios show that extra cells can add answers under deliberately arranged conditions; they validate the comparison program, not additional real-task success. The distinction between those scenarios and the historical negative result is retained in [Appendix](#)

D, together with all pair states and question counts. A more elaborate evaluation table is useful only when its added measurements resolve a question the simpler, logically informed comparison cannot already answer. [11]

S Tool feedback, subsequent actions, and use of results

S.1 Design and scope

This study follows how agents find mathematical resources, respond to checks, and use intermediate results in final proofs. It covers the second batch: eleven task groups, each run once under A, B, and C, giving 33 main runs. All passed the common mathematical acceptance procedure. The analysis adds process evidence while retaining those judgments and the original resource accounting.

Extraction covers all public tool records within this scope: 163 role dispatch records represent 135 distinct sessions and 4,233 events, comprising 4,054 actual tool calls and 179 tool discoveries. Every request has a matched response. Session and call identifiers remove duplicate records from continued dispatches and parallel-role copies. Final acceptance, dedicated review controls, additional repairs, and engineering continuations are outside the main-run denominator; the first batch remains separate. The [extraction account](#) and [source manifest](#) document inclusion and provenance.

The following six primary sources informed the design. Engineering articles, research releases, and preprints retain their actual publication types. The [acquisition manifest](#) records versions, reading scope, and file digests. These sources inform particular decisions; the operational labels were defined for this study.

Source	Use in this study
Anthropic, Demystifying evals for AI agents , engineering article	Assess interaction and outcome; allow valid alternative solutions
Anthropic, Quantifying infrastructure noise in agentic coding evals , engineering report	Record environmental constraints separately from task performance
Anthropic, How we built our multi-agent research system , engineering article	Check intermediate results, handoffs, and subsequent use
Microsoft Research, Magentic-One , preprint v1	Examine problems in successful runs and repeated actions in context
Google DeepMind and collaborators, Towards Autonomous Mathematics Research / Aletheia , preprint v3	Separate retrieval, applicability checks, and completed proofs
METR, RE-Bench research release , research announcement	Preserve resource accounting and distinguish parallel attempts from time budgets

S.2 Two fixed starts per run

Each run contributes its earliest proof or interface diagnostic and its earliest active retrieval for a concrete mathematical need, ordered by actual event time. A diagnostic must concern types, names, syntax, instances, or proof goals after source loading. Missing inputs, module-loading failures, and tools that never start are recorded separately as preparation problems. Retrieval requires an actively chosen keyword, declaration type, or source passage; prescribed bulk reading of upstream inputs does not automatically qualify. Development review narrowed the diagnostic rule to v1.1; the earlier version remains available. This is a retrospective design, documented in the [annotation protocol](#).

Annotations connect searches, edits, checks, and handoffs around an identifiable need. A shared filename, session, or arbitrary later passing check does not establish continuity. Cross-role connections require dispatch, return, and candidate evidence. The two starts concern the same need in 25 runs. Accordingly, 66 start records form two sets of 33; the records must not be added as independent recoveries. This fixed-first selection covers early processes in every run rather than enumerating all retrieval and recovery episodes.

The first diagnostics occurred during implementation in 19 runs and interface probes in 14. Among processes followed from first retrieval, 29 have at least one related declaration explicitly used in final source, 2 have source presence only, 1 was replaced, and 1 remains unknown. Adoption concerns a resource or implementation in the followed process, not every initial search hit. Source presence alone does not establish use by the final theorem.

The table is generated from the [adjudicated annotations](#), using the task identifiers from the main text. Recorded progress stops at different levels: a located candidate declaration, partial implementation, or a passing local check. These counts therefore do not rank configurations. For B-L6, no passing receipt was observed within the selected diagnostic segment, while the overall task passed. The available connection does not establish that this particular trial subsequently recovered.

Run	Diagnostic setting	Diagnostic progress	Retrieval progress	Final retrieval use	Shared need
A-L1	Implementation	Passed locally	Passed locally	Explicit use	Yes
B-L1	Implementation	Passed locally	Passed locally	Explicit use	Yes
C-L1	Implementation	Passed locally	Candidate located	Explicit use	Yes
A-L2	Probe	Passed locally	Passed locally	Explicit use	No
B-L2	Probe	Implementation advanced	Passed locally	Explicit use	Yes
C-L2	Probe	Alternative interface	Candidate located	Explicit use	Yes
A-L3	Implementation	Passed locally	Passed locally	Explicit use	Yes
B-L3	Implementation	Passed locally	Passed locally	Explicit use	No
C-L3	Implementation	Passed locally	Candidate located	Explicit use	No
A-L4	Probe	Candidate located	Passed locally	Explicit use	Yes
B-L4	Probe	Implementation advanced	Implementation advanced	Explicit use	Yes
C-L4	Probe	Implementation advanced	Candidate located	Explicit use	Yes
A-L5	Probe	Candidate located	Passed locally	Explicit use	Yes
B-L5	Implementation	Passed locally	Passed locally	Explicit use	Yes
C-L5	Implementation	Passed locally	Candidate located	Explicit use	No
A-L6	Implementation	Passed locally	Passed locally	Explicit use	Yes
B-L6	Probe	No pass in segment	Candidate located	Explicit use	No
C-L6	Implementation	Passed locally	Candidate located	Explicit use	No
A-L7	Probe	Passed locally	Passed locally	Explicit use	Yes
B-L7	Probe	Implementation advanced	Passed locally	Source presence	Yes
C-L7	Implementation	Passed locally	Candidate located	Explicit use	No
A-M1	Implementation	Passed locally	Passed locally	Source presence	Yes
B-M1	Implementation	Passed locally	Passed locally	Explicit use	Yes
C-M1	Implementation	Passed locally	Candidate located	Explicit use	Yes
A-M2	Probe	Candidate located	Passed locally	Replaced	Yes
B-M2	Probe	Implementation advanced	Passed locally	Explicit use	Yes
C-M2	Implementation	Passed locally	Candidate located	Explicit use	Yes
A-M3	Probe	Passed locally	Passed locally	Explicit use	No
B-M3	Implementation	Passed locally	Passed locally	Explicit use	Yes
C-M3	Implementation	Passed locally	Candidate located	Explicit use	Yes
A-H1	Implementation	Passed locally	Passed locally	Explicit use	Yes
B-H1	Probe	Passed locally	Candidate located	Unknown	Yes
C-H1	Implementation	Passed locally	Passed locally	Explicit use	Yes

S.3 Review and evidence

Automatic classification supplies coarse candidates for retrieval, editing, and checking. Review reads actual inputs and necessary output context, accounting for standard-input checks, separate targets within compound commands, replayed warnings, and a successful build followed by an unrelated command failure. Same-file check sequences aid navigation; they are not recovery labels. All 13 real-event regression cases and 5 synthetic boundary cases passed. These are targeted program checks, not a population accuracy estimate; see the [regression results](#).

The 44 records for A and C received non-blind review by other agents. The principal investigator reviewed all 22 B records and requested supplementary checks; the results were then consolidated into 66 adjudicated records. Initial labels, reviews, and changes are preserved separately. All assessors were model agents; there was no human-expert reference set or unbiased accuracy estimate. Review focused on first eligible events, continuity of need, and final-source evidence, rather than inferring adoption from imports or name occurrences.

A-M2 and A-M3 distinguish local success from delivery. Their first support actors obtained passing local checks, but their returns contained empty patches, no applied patch, and empty artifact lists. When the authors later inspected their drafts, these still contained 200 and 65 lines of input material respectively. Subsequent delegation produced verifiable nonempty patches and copies into author files; A-M2 also had an intervening export failure. Its first candidate was ultimately replaced. In A-M3, the first support candidate was replaced, while the author's earlier retrieval process could still be connected to later use. The [A review records](#) supply calls, file digests, and original line references.

B-M3 illustrates a connected process. The author searched for distribution-function and quantile resources, expanded the query, and delegated support implementation. The support actor encountered mismatches between the project's distribution function and the library expression; direct conversion still failed. Establishing equality of the two functions then enabled rewriting the limit, monotonicity, and right-continuity goals, and the support file passed. Final source retained this bridge and the quantile inverse-image result. This establishes local application and use without treating that stage as completion of the entire representation theorem. The two B-M3 [adjudicated records](#) identify the chain and final source locations. The [event index](#) contains requests, responses, and their original file and line references, cross-referenced by the [source manifest](#).

S.4 Environment observations, prototype checks, and time

Actual error output explicitly reported command lookup failure in 173 calls across all 33 runs; 49 nevertheless had a zero outer exit code. Multiple such messages in one call count once, while command categories can overlap. Lookup failures for `apply_patch` appeared in 66 calls across 33 runs and 64 sessions. Outer success therefore cannot replace reading subcommand feedback, and recurrence across runs does not establish persistent disregard within one session. Lookup failures require attention to paths and spelling; for example, a failed `lake` lookup does not establish that the image lacked the installed tool. These observations describe command lookup failures, not every environment fault or a measured time penalty; see the [environment summary](#).

An isolated research prototype provides startup tool information and argument validation without changing production. An isolated environment probe checked tool availability. Static replay covered 3,896 historical calls: it rejected the 9 invalid under the original rules and accepted the other 3,887, agreeing with the original argument guards. Another 29 synthetic boundary checks passed. Replay executed no historical commands; valid arguments do not imply execution or mathematical success. The [prototype account](#) and [replay summary](#) document these checks. No model-effect comparison has been run, so these results establish neither fewer agent errors nor a speed improvement.

All run times retain their original values and accounting, including ordinary waiting. Intervals between interleaved events are elapsed time; absent exclusive attribution, they are not pure retrieval time. Parallel-role durations are not summed as total run time. This appendix does not remove intervals to shorten reported expenditure.

T Evidence locations for the cases expanded in revision 27

This appendix supplies compact evidence entry points for the expanded main text. It selects 65 existing call events and 12 adjudicated records from 6 runs, preserved unchanged in the [event subset](#) and [annotation subset](#). The [index](#) gives call identifiers, roles, times, physical source lines, and final-source locations. This is a rereading subset, with no additional runs, reclassification, or expansion of Appendix S's statistical scope.

T.1 B-M3: an existing pointwise lemma followed by function rewriting

The main author’s first retrieval appears at original author-record lines 13–14, the expanded query at 15–16, and delegation at 20–21. The table refers to one support-author record, whose full path and call identifiers appear in the B-M3 index entries. Each range gives the call line and return line.

Original record lines	Action and return to check
21–22	Read distribution-function and related representation source
41–42	Write and check the draft; the pointwise-equality lemma already exists, but function targets fail
43–44	Attempt direct conversion; the return reports that expressions are not definitionally equal
45–46	Read the current file and conditionally complete order source
47–48	Probe supremum-related candidates, including unknown names
49–50	Adjust simplification and supremum proofs; function-target errors remain
51–52	Derive a function equality from the existing pointwise lemma and rewrite; other bounds are also revised, and checking passes with one style warning

The [final support file](#) retains the pointwise lemma at lines 17–18, uses it for the distribution function’s left-end limit at 22–24 and other properties at 27–29, 32–34, and 38–40, and states the quantile/distribution-function correspondence at 61–62. These locations support the main text’s conclusions about the local result and its retention. The pointwise proof already present in the draft must be distinguished chronologically from the later function equality.

T.2 B-M1: received advice, index revision, and final calls

The review advice returned to the main author at 21:49:01.258 UTC on 13 September 2026, at line 14 of the original author record. The author queried the stopped-process interface again at 21:49:16.818 and wrote the implementation at 21:52:38.908. After index-conversion revisions, the passing check returned at 21:55:48.220. These times locate the sequence; no exclusive retrieval duration is calculated.

The [final file](#) handles finite-index conversions at lines 19 and 21, uses the stopped-submartingale interface at 56, handles the negative process at 61–62, and combines the properties into a martingale at 65. The actual returned advice and implementation locations also appear in the [existing follow-up check](#).

T.3 A-L1 and B-L6: checking different endpoints against their targets

A-L1’s index entries preserve checks, reads, edits, and rechecks of the same task file. The [final file](#) retains observation-history measurability at line 49, uses finite-sum integrability at 121, and begins the final martingale theorem at 140. The main text uses these locations to check retention of the corresponding work.

B-L6’s three-term square-integral trial appears in its first-diagnostic entry; adoption of Gaussian-moment material appears in its first-retrieval entry. Their starts differ. The [final file](#) uses a distribution-map equality at line 23, transfers the mean integral at 25 and the square integral at 33, and begins the single-sensor formula at 44. These support the respective adoption conclusions without supplying a pass for the three-term trial.

T.4 Correspondence and rereading scope

The mathematical and historical accounts in Section 4 correspond to Appendix R.1–R.3 and its original sources. This appendix only adds locations for the processes expanded in Section 5. The empty patches and candidate replacements in A-M2 and A-M3 continue to rely on the [existing A review](#). Full calls, initial annotations, review results, and final source are inherited from revision 26. This edition compares each subset item with the full records. The work here is evidence rereading and explanation in the report; proof experiments have not been rerun.